THEIVANAI AMMAL COLLEGE FOR WOMEN

(An Autonomous Institution, Affiliated to Annamalai University, Chidambaram - Tamil Nadu)

(Accredited with 'A+' Grade (4th Cycle) by NAAC, Recognized under 2(f) and 12(B) by UGC, Approved by AICTE, New Delhi)

Villupuram-Tamil Nadu

DEPARTMENT OF COMPUTER SCIENCE

This is to certify that Ms **SUKAINA FIRDOUS J (20524UCSC047),** a student of III B.Sc. Computer Science, has successfully completed the practice-based practical learning component of **Advanced Database Management System – Practical (UCSR513)** during the academic year 2026–2027.

The practical examination will be held on **16.10.2026**

Course In-charge: Dr. S Manimekalai

Head of the Department: Ms. K Manohari

CONTENT

S.NO	EXPERIMENT	PAGE NO	FACULTY SIGNATURE
1	**DDL Commands** • CREATE • ALTER • DROP • TRUNCATE		
2	**DML Commands** • INSERT • UPDATE • DELETE		
3	**DQL Commands** • SELECT • WHERE • ORDER BY • GROUP BY • HAVING		
4	**Transactions** • COMMIT • ROLLBACK • SAVEPOINT		
5	**Joins** • Inner Join • Left Join • Right Join • Full Join • Self Join		
6	**SQL Functions** • Aggregate Functions • String Functions • Date Functions		
7	**Subqueries and Nested Queries**		
8	**Views, Indexes, and Sequences**		
9	**Stored Procedures and Functions**		
10	**Triggers**		
11	**Cursor Implementation**		
12	**PL/SQL Exception Handling**		
13	**Database Normalization (1NF, 2NF, 3NF, BCNF)**		
14	**Mini Project**		
	Annexures: • **Khan Academy** • **Hacker Rank**		
	• **Code Chef**		

EX. NO:01 Creation of a DDL operations using Movies Table
DATE:04.08.2026

Aim

To create a **Movies** table, insert movie records, display all records, and retrieve movies released from the year 2000 onwards.

Software Used

Oracle /MySQL/sqlite3

Description

DDL (Data Definition Language), it is used to create and modify the structure of database objects in database

DDL COMMANDS:

- Create
- Alter
- Drop
- Truncate
- Rename

CREATE: Create command is used to create a new table and also use constraints.

Syntax

```
CREATE TABLE table_name (
   column1 datatype constraints,
   column2 datatype constraints,
   column3 datatype constraints
);
```

Table Structure

```
SQL> CREATE TABLE movies (
        id INTEGER PRIMARY KEY,
```

name TEXT,

release_year INTEGER

);

Table Created

ALTER: Alter command is used to alter a table or make some changes in a table. It has 3 methods such as ADD, MODIFY, DROP

ADD: Add command is used to insert a new attribute/column with data type.

Syntax:

ALTER TABLE table_name ADD column_name datatype;

Query:

SQL> ALTER TABLE movies ADD director VARCHAR(100);

Table Altered

MODIFY: Modify command is used to modify or change the data type and to increase the data size in the table not the attribute/column.

Syntax:

ALTER TABLE table_name MODIFY column_name datatype;

Query:

SQL>ALTER TABLE movies MODIFY name VARCHAR(150);

ALTER TABLE movies ALTER COLUMN name VARCHAR(150);

DROP: Drop command is used to delete the attribute/column in a table.

Syntax:

ALTER TABLE table_name DROP COLUMN column_name;

Query:

SQL> ALTER TABLE movies DROP COLUMN director;

RENAME: Rename command is used to re-change the table name.

SQL> RENAME TABLE movies TO films;

Table renamed.

(OR)

ALTER TABLE movies RENAME TO films;

Column Rename

ALTER TABLE movies RENAME COLUMN name TO movie_name;

TRUNCATE: Truncate command is used to delete a value in the table.

SYNTAX:

TRUNCATE TABLE tablename;

Query:

TRUNCATE TABLE movies;

DROP: Deletes the table and all its data permanently.

SYNTAX:

DROP TABLE tablename;

Query:

DROP TABLE movies;

Explanation

- Create a table named **movies**.
- Insert movie details into the table.
- Display all movie records.
- Retrieve movies released in or after the year **2000** and sort them by release year.

Create Table

```
CREATE TABLE movies (
        id INTEGER PRIMARY KEY,
        name TEXT,
        release_year INTEGER
```

Table Created.

Sample Data

```
INSERT INTO movies VALUES (1, 'Avatar', 2009);
INSERT INTO movies VALUES (2, 'Titanic', 1997);
INSERT INTO movies VALUES (3, 'Star Wars: Episode IV - A New Hope', 1977);
INSERT INTO movies VALUES (4, 'Shrek 2', 2004);
INSERT INTO movies VALUES (5, 'The Lion King', 1994);
INSERT INTO movies VALUES (6, 'Disney''s Up', 2009);
```

Records Inserted.

SQL Query

Query 1: Display all movie records

```
SQL> SELECT * FROM movies;
```

Resultant Query

id	name	release_year
1	Avatar	2009
2	Titanic	1997
3	Star Wars: Episode IV - A New Hope	1977
4	Shrek 2	2004
5	The Lion King	1994
6	Disney's Up	2009

Query 2: Display movies released from the year 2000 onwards

```
SQL> SELECT *
FROM movies
WHERE release_year >= 2000
ORDER BY release_year;
```

Resultant Query

id	name	release_year
4	Shrek 2	2004
1	Avatar	2009
6	Disney's Up	2009

RESULT:

Thus, the Movies table was created successfully, records were inserted, all movie details were displayed, and the movies released from the year 2000 onwards were retrieved successfully.

EX. NO:02 **Creation of a DML operations using students Table**

DATE:07.08.2026

Aim

To create a **students** table and perform DML operations such as select, insert, update, and delete on it.

Software Used

Oracle /MySQL/sqlite3

Description

DML (Data Manipulation Language), it is used to manipulate the data present in the database.

DML COMMANDS:

- Insert
- Update
- Delete
- Select

SELECT: It is used to retrieve data from the table database.

Syntax

SELECT column1,column2,….

FROM table_name

WHERE condition;

Query

SELECT*FROM students;

INSERT: Used to add new data into a table.

Syntax

INSERT INTO table_name(column1,column2) VALUES(value1,value2);

Query

INSERT INTO students(id, name, age) VALUES(1,'Arun',22);

1 row inserted.

UPDATE: Used to modify existing data in a table

Syntax

UPDATE table_name

SET column1 = value1,column2 = value2

WHERE condition;

Query

UPDATE students

SET age = 21

WHERE id = 1;

1 row updated.

DELETE: Used to remove data from a table.

Syntax

DELETE FROM table_name

WHERE condition;

Query

DELETE FROM students

WHERE id = 2;

1 row deleted.

Explanation

- Select a table from the **students.**
- Insert students details into the table.
- Update existing data values in a table.
- Delete specific rows from a table.

Create Table

```
CREATE TABLE students (
    Id INT PRIMARY KEY,
    name VARCHAR(50),
    age INT,
    course VARCHAR(30)
);
```

Table Created

Sample Data

INSERT INTO students VALUES (1, 'Arun', 22,’CSE’);

INSERT INTO students VALUES (2, 'Ravi', 20,’ECE’);

Records Inserted

SQL Query

Query 1: Display all students records

SQL> SELECT * FROM students; **Resultant Query**

id	name	age	course
1	Arun	22	CSE
2	Ravi	20	ECE

Query 2: Update the students records

```
UPDATE students
SET age = 21
WHERE id = 1;
```

1 row updated

Updated Record

id	name	age	course
1	Arun	21	CSE
2	Ravi	20	ECE

Query 3: Delete the students records

```
DELETE FROM students
WHERE id = 2;
```

1 row deleted.

Remaining Records

id	name	age	course
1	Arun	21	CSE

RESULT:

Thus, the Students table was created successfully, records were inserted, all students details were displayed, students records were updated and deleted.

EX. NO:03 **Creation of a DQL operations using Book Table**

DATE:11.08.2026

Aim

To study and perform DQL commands such as select, where, order by, group by and having on the book table.

Software Used

Oracle /MySQL/sqlite3

Description

DQL (Data Query Language), it is used to retrieve and fetch data from a database table.

DQL COMMANDS:

- Select
- Where
- Order By
- Group By
- Having

SELECT: It is used to specifies the exact columns to retrieve data from the table database.

Syntax

SELECT column1,column2,….

FROM table_name

WHERE condition;

Query

SELECT*FROM book;

WHERE: Filters rows before grouping based on conditions.

Syntax

SELECT column1,column2,….

FROM table_name

WHERE condition;

Query

SELECT*FROM book

WHERE price>500;

ORDER BY: Sorts the final results in ascending or descending order.

Syntax

SELECT column1,column2,….

FROM table_name

ORDER BY column1 ASC;

Query

SELECT*FROM book

ORDER BY price DESC;

GROUP BY: Aggregates rows with identical values into summary rows.

Syntax

SELECT column1,COUNT(*)

FROM table_name

GROUP BY column1;

Query

```
SELECT publisher, COUNT(*)
FROM book
GROUP BY publisher;
```

HAVING: Filters grouped summary rows after the GROUP BY clause.

Syntax

```
SELECT column1,COUNT(*)
FROM table_name
GROUP BY column1;
HAVING condition;
```

Query

```
SELECT publisher, COUNT(*)
FROM book
GROUP BY publisher;
HAVING COUNT(*) > 1;
```

Explanation

- Create the book table.
- Insert sample records into the table.
- Use SELECT to display all records.
- Use WHERE to filter records.
- Use ORDER BY to sort records.
- Use GROUP BY to GROUP BY to group records.
- Use HAVING to filter grouped records.

Create Table

```
CREATE TABLE book (
    Book_id INT PRIMARY KEY,
    title VARCHAR(50),
    author VARCHAR(100),
    publisher VARCHAR(30),
    year INT,
    price DECIMAL(10,2)
);
```

Sample Data

```
INSERT INTO book (book_id, title, author, publisher, year, price)
VALUES (1, 'Python Basics', 'John Smith', 'ABC Press', 2022, 450);
INSERT INTO book (book_id, title, author, publisher, year, price)
VALUES (2, 'DBMS Guide', 'A. Kumar', 'XYZ Publications', 2021, 550);
INSERT INTO book (book_id, title, author, publisher, year, price)
VALUES (3, 'SQL Mastery', 'John Smith', 'ABC Press', 2023, 650);
INSERT INTO book (book_id, title, author, publisher, year, price)
VALUES (4, 'Java Fundamentals', 'R. Ravi', 'Tech Books', 2020, 400);
```

SQL Query

Query 1: SELECT to display all book records.

```
SQL> SELECT * FROM book;
```

Resultant Query

book_id	title	author	publisher	year	price
1	Python Basics	John Smith	ABC Press	2022	450
2	DBMS Guide	A.Kumar	XYZ Publications	2021	550
3	SQL Mastery	John Smith	ABC Press	2023	650
4	Java Fundamentals	R.Ravi	Tech Books	2020	400

Query 2: WHERE to filter records.

```
SQL> SELECT*FROM book
WHERE price>500;
```

Resultant Query

Book_id	title	author	publisher	year	price
2	DBMS Guide	A.Kumar	XYZ Publications	2021	550
3	SQL Mastery	John Smith	ABC Press	2023	650

Query 3: ORDER BY to filter records.

```
SELECT*FROM book
ORDER BY price DESC;
```

Resultant Query

book_id	title	author	publisher	year	price
3	SQL Mastery	John Smith	ABC Press	2023	650
2	DBMS Guide	A.Kumar	XYZ Publications	2021	550
1	Python Basics	John Smith	ABC Press	2022	450
4	Java Fundamentals	R.Ravi	Tech Books	2020	400

Query 4: GROUP BY to group records.

```
SELECT publisher, COUNT(*)
FROM book
GROUP BY publisher;
```

Resultant Query

publisher	COUNT(*)
ABC Press	2
XYZ Publications	1
Tech Books	1

Query 5: Use HAVING to filter grouped records.

```
SELECT publisher, COUNT(*)
 FROM book
 GROUP BY publisher;
 HAVING COUNT(*) > 1;
```

Resultant Query

publisher	COUNT(*)
ABC Press	2

RESULT:

Thus, the DQL Command Queries were successfully used to display, select, where, order by, and group by data in the above students table.

EX. NO:04 **Creation of a Transactions operations using College Management**
DATE:15.08.2026 **Table**

AIM

To study and perform transaction control commands like commit, rollback, and savepoint on the College Management table.

Software Used

Oracle /MySQL/sqlite3

Description

TCL (Transactional Control Language), it is used to manage changes made by data manipulation statements to ensure database integrity.

TCL COMMANDS:

- Commit
- Rollback
- Savepoint

COMMIT: Permanently saves all data changes made during the current transaction to the database.

Syntax

SQL>COMMIT;

Query

```
UPDATE college_management
SET fees = 55000
WHERE id = 1;
COMMIT;
```

Transaction Committed.

ROLLBACK: Undoes all changes made during the current transaction and reverts the database to its last committed state.

Syntax

ROLLBACK;

Query

SQL>ROLLBACK;

Transaction rolled back.

ROLLBACK TO SAVEPOINT: ROLLBACK TO SAVEPOINT is used to Creates a temporary checkpoint within a transaction to allow a partial rollback to that specific point.

Syntax

ROLLBACK TO SAVEPOINT savepoint_name;

Query

SQL>ROLLBACK TO SAVEPOINT sp1;

Rollback to savepoint completed.

EXPLANATION

- Create the college_management table.
- Insert sample records into the table.
- Use SAVEPOINT to mark a point in the transaction.
- Use ROLLBACK to undo changes.
- Use COMMIT to save changes permanently.
- Display the output after each operation.

Create Table

```
CREATE TABLE college_management (
id INT PRIMARY KEY,
student_name VARCHAR(50),
course VARCHAR(50),
fees INT
);
```

Table Created.

Sample Data

```
INSERT INTO college_management VALUES (1, 'Arun', 'CSE', 50000);
INSERT INTO college_management VALUES (2, 'sukaina', 'ECE', 45000);
```

Records Inserted.

SQL Query

Query 1: A Commit used to save changes permanently in the database.

```
UPDATE college_management
SET fees = 55000
WHERE id = 1;
COMMIT;
```

Resultant Query

Commit complete.

Query 2: A Rollback undoes changes.

```
ROLLBACK TO sp1;
```

Resultant Query

Rollback complete.

Query 3: Savepoint creates a checkpoint.

SAVEPOINT savepoint_name;

Resultant Query

Savepoint created.

RESULT:

Thus, the TCL Command Queries were successfully executed used to display the savepoint, rollback and commit.

EX. NO:05 Creation of a Joins operations using Employee and Dept Table

DATE:18.08.2026

AIM

To perform different types of SQL JOIN operations (INNER JOIN, LEFT JOIN, RIGHT JOIN, FULL JOIN,OUTER JOIN and SELF JOIN) to retrieve data from one or more related tables based on a common column.

Software Used

Oracle /MySQL/sqlite3

Description

Joins it is used to retrieve the data from one or more related tables based on a common columns.

JOINS COMMANDS:

- INNER JOIN
- LEFT JOIN
- RIGHT JOIN
- FULL JOIN
- SELF JOIN

INNER JOIN: Returns only the matching records from both tables.

Syntax

SELECT column_name

FROM table1

INNER JOIN table2

ON table1.common_column = table2.common_column;

LEFT JOIN: Returns all records from the left table and matching records from the right table.

Syntax

SELECT column_name

FROM table1

LEFT JOIN table2

ON table1.common_column = table2.common_column;

RIGHT JOIN: Returns all records from the right table and matching records from the left table.

Syntax

SELECT column_name

FROM table1

RIGHT JOIN table2

ON table1.common_column = table2.common_colum

FULL JOIN: Returns all records from both tables, including matching and non-matching Records.

Syntax

SELECT column_name

FROM table1

FULL OUTER JOIN table2

ON table1.common_column = table2.common_column;

SELF JOIN: Joins a table with itself to compare or retrieve related records.

Syntax

```
SELECT A.column_name, B.column_name
FROM table_name A
JOIN table_name B
 ON A.common_column = B.common_column
```

EXPLANATION

- Open the Oracle SQL environment (or SQL Developer).
- Create the required tables (Employee and Department).
- Insert sample records into the tables.
- Verify the inserted data using the SELECT statement.
- Execute the required JOIN query (INNER, LEFT, RIGHT, FULL OUTER and SELF JOIN).
- Observe the output and compare the retrieved records.
- Verify that the output matches the expected result.
- End the SQL session.

CREATE TABLE

```
-- Create Employee Table
CREATE TABLE Employee (
   Emp_ID NUMBER PRIMARY KEY,
   Name VARCHAR2(30),
   Dept_ID NUMBER
);
```

```
-- Create Department Table
CREATE TABLE Department (
    Dept_ID NUMBER PRIMARY KEY,
    Department VARCHAR2(30)
);
```

Tables Created.

SAMPLE DATA

```
-- Insert Sample Data into Employee
INSERT INTO Employee VALUES (101, 'Arun', 1);
INSERT INTO Employee VALUES (102, 'Bala', 2);
INSERT INTO Employee VALUES (103, 'Charan', 3);
INSERT INTO Employee VALUES (104, 'Divya', NULL);
-- Insert Sample Data into Department
INSERT INTO Department VALUES (1, 'HR');
INSERT INTO Department VALUES (2, 'Sales');
INSERT INTO Department VALUES (4, 'Finance');
```

Records Inserted.

SQL Query

Query 1: The inner join returns only the matching records from both tables.

```
SELECT E.Emp_ID, E.Name, D.Department
FROM Employee E
INNER JOIN Department D
ON E.Dept_ID = D.Dept_ID;
```

Resultant Query

Emp_ID	Name	Department
101	Arun	HR
102	Vishnu	Sales

Query 2: Display all records from the left table.

```
SELECT E.Emp_ID, E.Name, D.Department
FROM Employee E
LEFT JOIN Department D
ON E.Dept_ID = D.Dept_ID;
```

Resultant Query

Emp_ID	Name	Department
101	Arun	HR
102	Vishnu	Sales

Query 2: Display all records from the left table.

```
SELECT E.Emp_ID, E.Name, D.Department
FROM Employee E
LEFT JOIN Department D
ON E.Dept_ID = D.Dept_ID;
```

Resultant Query

Emp_ID	Name	Department
101	Arun	HR
102	Vishnu	Sales
103	Charan	NULL
104	Divya	NULL

Query 3: Display all records from the right table.

```
SELECT E.Emp_ID, E.Name, D.Department
FROM Employee E
RIGHT JOIN Department D
ON E.Dept_ID = D.Dept_ID;
```

Resultant Query

Emp_ID	Name	Department
101	Arun	HR
102	Vishnu	Sales
NULL	NULL	Finance

Query 4: Display all records from the right table.

```
SELECT E.Emp_ID, E.Name, D.Department
FROM Employee E
FULL OUTER JOIN Department D
ON E.Dept_ID = D.Dept_ID;
```

Resultant Query

Emp_ID	Name	Department
101	Arun	HR
102	Vishnu	Sales
103	Charan	NULL
104	Divya	NULL
NULL	NULL	Finance

Query 5: Joins a table itself.

```
SELECT
E.Name AS Employee,
M.Name AS Manager
FROM Employee E
LEFT JOIN Employee M
ON E.Manager_ID = M.Emp_ID;
```

Resultant Query

Employee	Manager
Arun	NULL
Bala	Arun
Charan	Arun
Divya	Vishnu



RESULT:

Thus, the SQL JOIN operation was executed successfully, and the output was obtained as expected.

EX. NO:06 Creation of a SQL Functions operations using Employee Table

DATE:20.08.2026

AIM

To perform calculations on multiple rows using SQL Aggregate Functions.

Software Used

Oracle /MySQL/sqlite3

Description

SQL Functions are built-in functions used to perform calculations and handle date values efficiently in a database.

SQL FUNCTION COMMANDS:

- AGGREGATE FUNCTIONS
- STRING FUNCTIONS
- DATE FUNCTIONS

AGGREGATE FUNCTIONS: To perform calculations on multiple rows and return a single value.

Common aggregate functions are:

- COUNT()
- SUM()
- AVG()
- MAX()
- MIN()

Syntax

SELECT AGGREGATE_FUNCTION(column_name)

FROM table_name;

STRING FUNCTIONS: It is used to manipulate and modify text(string) values.

Syntax

```
SELECT function_name(column_name)
FROM table_name;
```

Common String functions are:

- UPPER()
- LOWER()
- LENGTH()
- SUBSTR()

DATE FUNCTIONS: It is used to retrieve and manipulate date and time values.

Common Date functions are:

- CURRENT_DATE
- YEAR()
- MONTH()
- DAY()

Syntax

```
SELECT function_name(date_column)
FROM table_name;
```

EXPLANATION

- Create the required table.
- Insert sample records into the table.
- Execute Aggregate Functions (COUNT(), SUM(), AVG(), MAX(), MIN()) to perform calculations.
- Execute String Functions (UPPER(), LOWER(), LENGTH(), SUBSTR()) to manipulate text data.
- Execute Date Functions (CURRENT_DATE, YEAR(), MONTH(), DAY()) to retrieve and manipulate date values.
- Display the output and verify the results.

CREATE TABLE

```
CREATE TABLE Employee (
    EmpID INT,
    Name VARCHAR(30),
    Department VARCHAR(20),
    Salary INT
);
```

Table created.

SAMPLE DATA

```
INSERT INTO Employee VALUES (101,'Arun','IT',30000);
INSERT INTO Employee VALUES (102,'Priya','HR',25000);
INSERT INTO Employee VALUES (103,'Kumar','IT',40000);
INSERT INTO Employee VALUES (104,'Meena','Sales',35000);
INSERT INTO Employee VALUES (105,'Ravi','HR',28000);Insert Sample Data
INSERT INTO Employee VALUES (101,'Arun','IT',30000);
INSERT INTO Employee VALUES (102,'Priya','HR',25000);
INSERT INTO Employee VALUES (103,'Kumar','IT',40000);
INSERT INTO Employee VALUES (104,'Meena','Sales',35000);
INSERT INTO Employee VALUES (105,'Ravi','HR',28000);
```

SQL QUERY

Query 1: To perform calculations on multiple rows and return a single value.

```
SELECT
COUNT(*) AS Total_Employees,
SUM(Salary) AS Total_Salary,
AVG(Salary) AS Average_Salary,
MAX(Salary) AS Highest_Salary,
MIN(Salary) AS Lowest_Salary
FROM Employee;
```

Resultant Query

Total_Employees	Total_Salary	Average_Salary	Highest_Salary	Lowest_Salary
5	158000	31600	40000	25000

Query 2: It is used to manipulate and modify text(string) values.

```
SELECT
Name,
UPPER(Name) AS Upper_Name,
LOWER(Name) AS Lower_Name,
LENGTH(Name) AS Length_Name,
SUBSTR(Name,1,3) AS First_Three_Letters
FROM Student;
```

Resultant Query

Name	Upper_Name	Lower_Name	Length_Name	First_Three_Letter
Arun	ARUN	arun	4	aru
Priya	PRIYA	Priya	5	Pri
Kumar	KUMAR	Kumar	5	Kum

Query 3: It is used to retrieve and manipulate date and time values.

```
SELECT
OrderID,
OrderDate,
CURRENT_DATE AS Today_Date,
YEAR(OrderDate) AS Year,
MONTH(OrderDate) AS Month,
DAY(OrderDate) AS Day
FROM Orders;
```

Resultant Query

OrderID	OrderDate	Today_Date	Year	Month	Day
1	2026-08-01	2026-08-07	2026	8	1
2	2026-08-05	2026-08-07	2026	8	5
3	2026-08-07	2026-08-07	2026	8	7



RESULT:

Thus, the SQL Functions (Aggregate Functions, String Functions, and Date Functions) were executed successfully, and the desired output was obtained.

EX. NO:07 **Creation of Subqueries and Nested Queries using**

DATE:22.08.2026 **Hospital Management Table**

AIM

To study and implement subqueries and nested queries in SQL using a Hospital Management System database and retrieve information based on conditions from another query.

Software Used

Oracle /MySQL/sqlite3

Description

Subqueries and Nested Queries are queries written inside another SQL query to retrieve data based on the result of an inner query from the Hospital_Management table.

SQL FUNCTION COMMANDS:

- SUBQUERY
- NESTED QUERY

SUBQUERY: A subquery is a query written inside another SQL query to obtain intermediate results.

Syntax

```
SELECT column_name
FROM table_name
WHERE column_name operator
    (SELECT column_name
    FROM table_name
    WHERE condition);
```

NESTED QUERY: A nested query is a query placed within another query to retrieve data based on the result of the inner query.

Syntax

```
SELECT column_name
FROM table_name
WHERE column_name IN
    (SELECT column_name
     FROM table_name
     WHERE condition)
```

EXPLANATION

- Create the **Hospital Management** database.
- Create the Patient and Doctor tables.
- Insert sample patient and doctor records.
- Use a **subquery** to retrieve data based on the result of another query.
- Use nested queries with IN, =, >, <, AVG(), MAX(), etc.
- Execute the queries and observe the output.
- Verify that the result is obtained correctly.

CREATE TABLE

```
CREATE TABLE Hospital_Management (
    Patient_ID INT,
    Patient_Name VARCHAR(30),
    Age INT,
    Gender VARCHAR(10),
    Disease VARCHAR(30),
    Doctor_Name VARCHAR(30),
    Fees INT
);
```

Table Created.

SAMPLE DATA

```
INSERT INTO Hospital_Management VALUES
(101, 'Arun', 25, 'Male', 'Fever', 'Dr. Kumar', 500);
INSERT INTO Hospital_Management VALUES
(102, 'Priya', 32, 'Female', 'Diabetes', 'Dr. Ravi', 800);
INSERT INTO Hospital_Management VALUES
(103, 'Rahul', 45, 'Male', 'Heart Disease', 'Dr. Kumar', 1500);
INSERT INTO Hospital_Management VALUES
(104, 'Divya', 28, 'Female', 'Fever', 'Dr. Meena', 600);
INSERT INTO Hospital_Management VALUES
(105, 'Karthik', 50, 'Male', 'Diabetes', 'Dr. Ravi', 1000);
```

Records Inserted.

SQL QUERY

Query 1: Fees greater than average fees

```
SELECT Patient_Name, Fees
FROM Hospital_Management
WHERE Fees > (SELECT AVG(Fees)
    FROM Hospital_Management);
```

Resultant Query

Patient_Name	Fees
Priya	800
Rahul	1500
Karthik	1000

Query 2: Highest Fees patient

```
SELECT Patient_Name, Fees
FROM Hospital_Management
WHERE Fees = (SELECT MAX(Fees)
              FROM Hospital_Management);
```

Resultant Query

Patient_Name	Fees
Rahul	1500

Query 3: Lowest fees patient

```
SELECT Patient_Name, Fees
FROM Hospital_Management
WHERE Fees = (SELECT MIN(Fees)
              FROM Hospital_Management);
```

Resultant Query

Patient_Name	Fees
Arun	500

Query 4: Patients older than average age

```
SELECT Patient_Name, Age
FROM Hospital_Management
WHERE Age > (SELECT AVG(Age)
             FROM Hospital_Management);
```

Resultant Query

Patient_Name	Age
Rahul	45
Karthik	50

Query 5: Patients having the same disease as Arun

```
SELECT Patient_Name, Disease
FROM Hospital_Management
WHERE Disease = (SELECT Disease
        FROM Hospital_Management
        WHERE Patient_Name = 'Arun');
```

Resultant Query

Patient_Name	Disease
Arun	Fever
Divya	Fever

Query 6: Patients treated by the same doctor as Priya

```
SELECT Patient_Name, Doctor_Name
FROM Hospital_Management
WHERE Doctor_Name = (SELECT Doctor_Name
        FROM Hospital_Management
        WHERE Patient_Name = 'Priya');
```

Resultant Query

Patient_Name	Doctor_Name
Priya	Dr.Ravi
Karthik	Dr.Ravi

Query 7: Patients whose fees are greater than Arun's fees

```
SELECT Patient_Name, Fees
FROM Hospital_Management
WHERE Fees > (SELECT Fees
        FROM Hospital_Management
       WHERE Patient_Name = 'Arun');
```

Resultant Query

Patient_Name	Fees
Priya	800
Rahul	1500
Divya	600
Karthik	1000

Query 8: Youngest patient

```
SELECT Patient_Name, Age
FROM Hospital_Management
WHERE Age = (SELECT MIN(Age)
        FROM Hospital_Management);
```

Resultant Query

Patient_Name	Age
Arun	25

RESULT:

Thus, Subqueries and Nested Queries were successfully executed using the Hospital_ Management table, and the required patient details were obtained successfully.

EX. NO:08 **Creation of a View, Indexes and Sequences operations using**

DATE:25.08.2026 **Furniture Table**

AIM

To create and manipulate views, Indexes and sequences on the **Furniture** table for efficient data access and automatic generation of unique values.

Software Used

Oracle /MySQL/sqlite3

Description

View, Index, and Sequence are database objects used to simplify data access, speed up data retrieval, and automatically generate unique values in the Furniture table.

SQL FUNCTION COMMANDS:

- View
- Indexes
- Sequences

VIEW: A View is a virtual table created from one or more tables.

Syntax

```
CREATE VIEW view_name AS
SELECT column1, column2, ...
FROM table_name
WHERE condition;
```

INDEXES: An Index is used to speed up data retrieval from a table.

Syntax

```
CREATE INDEX index_name
ON table_name(column_name);
```

SEQUENCES: A Sequence is used to automatically generate unique numbers.

Syntax

```
CREATE SEQUENCE sequence_name
START WITH initial_value
INCREMENT BY increment_value;
```

EXPLANATION

- Create the Furniture table and insert sample records.
- Create a View to display selected columns from the Furniture table.
- Create an Index on the FNAME column to improve data searching.
- Create a Sequence to generate unique furniture IDs automatically.
- Execute SELECT and INSERT queries to verify the View, Index, and Sequence operations.
- Display the output of all operations.

CREATE TABLE

```
CREATE TABLE Furniture (
    FID NUMBER(3),
    FNAME VARCHAR(20),
    CATEGORY VARCHAR2(20),
    PRICE NUMBER(8,2)
);
```

Table Created.

SAMPLE DATA

```
INSERT INTO Furniture VALUES (101, 'Chair', 'Seating', 1500);
INSERT INTO Furniture VALUES (102, 'Table', 'Table', 3500);
INSERT INTO Furniture VALUES (103, 'Sofa', 'Seating', 12000);
INSERT INTO Furniture VALUES (104, 'Bed', 'Bedroom', 18000);
CREATE VIEW Furniture_View AS
SELECT FID, FNAME, PRICE
FROM Furniture;
SELECT * FROM Furniture;
```

Records Inserted.

SQL QUERY

Query 1: To display a View is a virtual table created from one or more tables.

```
CREATE TABLE Furniture (
    FID NUMBER(3),
    FNAME VARCHAR2(20),
    CATEGORY VARCHAR2(20),
    PRICE NUMBER(8,2)
);
```

Resultant Query

FID	FNAME	PRICE
101	Chair	1500
102	Table	3500
103	Sofa	12000
104	Bed	18000

Query 2: An Index is used to speed up data retrieval from a table.

```
CREATE INDEX Furniture_Index
ON Furniture (FNAME);
SELECT*FROM Furniture
WHERE FNAME = 'Chair';
```

Resultant Query

FID	FNAME	CATEGORY	PRICE
101	Chair	Seating	1500

Query 3: : A Sequence is used to automatically generate unique numbers.

```
CREATE SEQUENCE Furniture_Seq

START WITH 105

INCREMENT BY 1;

INSERT INTO Furniture VALUES(Furniture_Seq.NEXTVAL,'Cupboard','Storage',8500);

SELECT*FROM Furniture;
```

Resultant Query

FID	FNAME	CATEGORY	PRICE
101	Chair	Seating	1500
102	Table	Table	3500
103	Sofa	Seating	12000
104	Bed	Bedroom	18000
105	Cupboard	Storage	8500
106	Desk	Table	5000

RESULT:

Thus, the View,Index and Sequence were successfully created and executed on the Furniture table, and the required outputs were obtained successfully.

EX. NO:09 **Creation of Stored Procedures and Functions operations using**

DATE:27.08.2026 **Accessories Table**

AIM

To study and implement Stored Procedures and Functions in PL/SQL to perform specific operations on database tables and to understand how procedures and functions can be created, executed, and used to process data efficiently.

Software Used

Oracle /MySQL/sqlite3

Description

Stored Procedures and Functions are PL/SQL programs stored in the database to perform specific tasks. A procedure performs an operation, while a function performs an operation and returns a value.

SQL FUNCTION COMMANDS:

- Stored Procedures
- Functions

STORED PROCEDURES: A stored procedure is a PL/SQL program stored in the database to perform a specific task.

Syntax

```
CREATE OR REPLACE PROCEDURE procedure_name
(
   parameter_name IN datatype
)
IS
BEGIN
   statements;
END;
```

FUNCTIONS: A function is a PL/SQL program that performs a specific task and returns a value.

Syntax

```
CREATE OR REPLACE FUNCTION function_name
(
   parameter_name IN datatype
)
RETURN datatype
IS
BEGIN
   statements;
   RETURN value;
END;
/
```

EXPLANATION

- Create a table and insert the required records.
- Create a **Stored Procedure** using CREATE OR REPLACE PROCEDURE.
- Pass required parameters to the procedure.
- Write SQL/PLSQL statements inside the procedure to perform the required operation.
- Execute the procedure using the EXEC command.
- Create a **Function** using CREATE OR REPLACE FUNCTION.
- Return a value from the function using the RETURN statement.
- Execute the function using a SELECT statement.
- Verify the output obtained from both the procedure and function.

Create Table

```
CREATE TABLE ACCESSORIES (
        A_ID NUMBER,
        A_NAME VARCHAR2(30),
        PRICE NUMBER
);
```

Table Created.

Sample Data

```
INSERT INTO ACCESSORIES VALUES (1,'MOUSE',500);
INSERT INTO ACCESSORIES VALUES (2,'KEYBOARD',800);
INSERT INTO ACCESSORIES VALUES (3,'HEADPHONE',1200);
```

Records Inserted.

SQL Query

Query 1: A stored procedure is a PL/SQL program stored in the database to perform a specific task.

```
CREATE OR REPLACE PROCEDURE SHOW_ACCESSORY
(
   P_ID IN NUMBER
)
IS
   V_NAME ACCESSORIES.A_NAME%TYPE;
   V_PRICE ACCESSORIES.PRICE%TYPE;
BEGIN
   SELECT A_NAME, PRICE
   INTO V_NAME, V_PRICE
   FROM ACCESSORIES
   WHERE A_ID = P_ID;
```

```
    DBMS_OUTPUT.PUT_LINE('Accessory Name: ' || V_NAME);


        DBMS_OUTPUT.PUT_LINE('Price: ' ||
V_PRICE); END;
/
```

Resultant Query

Accessory Name	Price
Necklace	2500

Query 2: A function is a PL/SQL program that performs a specific task and returns a value.

```
 CREATE OR REPLACE FUNCTION GET_DISCOUNT
(
   P_PRICE IN NUMBER
)
RETURN NUMBER
IS
BEGIN
   RETURN P_PRICE * 10 / 100;
END;
/
```

Resultant Query

A_NAME	PRICE	DISCOUNT
MOUSE	500	50
KEYBOARD	800	80
HEADPHONE	1200	120

RESULT:

Thus, the Stored Procedure and Function were successfully created and executed using the ACCESSORIES table.

EX. NO:10 **Creation of trigger operations using Hospital Emergency**

DATE:30.08.2026 **Priority Table**

AIM

To create a trigger that automatically assigns an appropriate **priority number** based on the emergency level when a new patient is inserted into the **HOSPITAL_EMERGENCY_PRIORITY** table.

Software Used

Oracle /MySQL/sqlite3

Description

A **Trigger** is a special PL/SQL program that executes **automatically** when a specified event occurs on a table.

SQL FUNCTION COMMANDS:

- INSERT
- UPDATE
- DELETE

Syntax

```
CREATE OR REPLACE TRIGGER trigger_name
BEFORE INSERT OR UPDATE
ON table_name
FOR EACH ROW
BEGIN
   statements;
END;
/
```

CREATE PROGRAM

```
CREATE OR REPLACE TRIGGER SET_EMERGENCY_PRIORITY
BEFORE INSERT ON HOSPITAL_EMERGENCY_PRIORITY
FOR EACH ROW
BEGIN

   IF UPPER(:NEW.EMERGENCY_LEVEL) = 'CRITICAL' THEN
      :NEW.PRIORITY := 1;

   ELSIF UPPER(:NEW.EMERGENCY_LEVEL) = 'URGENT' THEN
      :NEW.PRIORITY := 2;

   ELSIF UPPER(:NEW.EMERGENCY_LEVEL) = 'NORMAL' THEN
      :NEW.PRIORITY := 3;

   END IF;

END;
/
```

INSERT TABLE NEW PATIENTS

```
INSERT INTO HOSPITAL_EMERGENCY_PRIORITY
(
   PATIENT_ID,
   PATIENT_NAME,
```

```
    AGE,
    EMERGENCY_LEVEL
)
VALUES
(
    104,
    'RAHUL',
    55,
    'CRITICAL'
);
SELECT * FROM HOSPITAL_EMERGENCY_PRIORITY;
```

OUTPUT

PATIENT_ID	PATIENT_NAME	AGE	EMERGENCY_LEVEL	PRIORITY
101	ARUN	45	CRITICAL	1
102	PRIYA	30	URGENT	2
103	KUMAR	60	NORMAL	3
104	RAHUL	55	CRITICAL	1

RESULTS

Thus, the trigger was successfully created and executed to automatically assign the priority number according to the patient's emergency level.

EX. NO:11 Creation of Cursor Implementation using Library Table

DATE:02.09.2026

AIM

To create and implement a cursor in PL/SQL to retrieve and display records from the LIBRARY table.

Software Used

Oracle /MySQL/sqlite3

Description

A cursor is used to retrieve and process the records of the LIBRARY table one row at a time.

SQL FUNCTION COMMANDS:

DECLARE – Define the cursor and its SQL query.

OPEN – Execute the cursor query and make the result set available.

FETCH – Retrieve one row at a time.

CLOSE – Release the cursor after processing all rows.

Syntax

```
DECLARE
  CURSOR cursor_name IS
    SELECT column1, column2
    FROM table_name;

  variable1 table_name.column1%TYPE;
  variable2 table_name.column2%TYPE;
BEGIN
```

```
    OPEN cursor_name;

    LOOP
        FETCH cursor_name INTO variable1, variable2;

        EXIT WHEN cursor_name%NOTFOUND;

        DBMS_OUTPUT.PUT_LINE(variable1 || ' ' || variable2);
    END LOOP;

    CLOSE cursor_name;
END;
/
```

EXPLANATION

- Create the LIBRARY table with suitable attributes.
- Insert sample records into the table.
- Declare an explicit cursor in the PL/SQL block.
- Open the cursor.
- Fetch each record from the cursor.
- Display the fetched values using DBMS_OUTPUT.PUT_LINE.
- Continue fetching until all records are processed.
- Close the cursor.
- Display the result.

CREATE TABLE

```
CREATE TABLE LIBRARY
(
   BOOK_ID NUMBER(5),
   BOOK_NAME VARCHAR2(30),
   AUTHOR VARCHAR2(30),
   PRICE NUMBER(8,2)
);
```

SAMPLE DATA

```
INSERT INTO LIBRARY VALUES(101, 'DBMS', 'Korth', 450);
INSERT INTO LIBRARY VALUES(102, 'Oracle', 'Scott', 550);
INSERT INTO LIBRARY VALUES(103, 'Java', 'Herbert', 600);
INSERT INTO LIBRARY VALUES(104, 'Python', 'Mark Lutz', 700);
SELECT * FROM LIBRARY;
```

OUTPUT

BOOK_ID	BOOK_NAME	AUTHOR	PRICE
101	DBMS	Korth	450
102	Oracle	Scott	550
103	Java	Herbert	600
104	Python	Mark Lutz	700

QUERY: Cursor Implementation

```
DECLARE
   CURSOR C_LIBRARY IS
      SELECT BOOK_ID, BOOK_NAME, AUTHOR, PRICE
      FROM LIBRARY;
```

```
    V_ID     LIBRARY.BOOK_ID%TYPE;
    V_NAME  LIBRARY.BOOK_NAME%TYPE;
    V_AUTHOR LIBRARY.AUTHOR%TYPE;
    V_PRICE  LIBRARY.PRICE%TYPE;

BEGIN
    OPEN C_LIBRARY;

    LOOP
        FETCH C_LIBRARY
        INTO V_ID, V_NAME, V_AUTHOR, V_PRICE;

        EXIT WHEN C_LIBRARY%NOTFOUND;

        DBMS_OUTPUT.PUT_LINE(
            'Book ID: ' || V_ID ||
            ' | Book Name: ' || V_NAME ||
            ' | Author: ' || V_AUTHOR ||
            ' | Price: ' || V_PRICE
        );
    END LOOP;

    CLOSE C_LIBRARY;
END;
/
```

OUTPUT

Book ID: 101 | Book Name: DBMS | Author: Korth | Price: 450

Book ID: 102 | Book Name: Oracle | Author: Scott | Price: 550

Book ID: 103 | Book Name: Java | Author: Herbert | Price: 600

Book ID: 104 | Book Name: Python |Author: Mark Lutz | Price: 700

PL/SQL procedure successfully completed.



RESULT

Thus, the **explicit cursor was successfully created and implemented** in PL/SQL to retrieve and display the records from the LIBRARY table **row by row**.

EX. NO:12 **Creation of PL/SQL Exception Handling using Student**

DATE:04.09.2026 **Table**

AIM

To create a PL/SQL program using exception handling to manage errors that occur while accessing data from a student table.

Software Used

Oracle /MySQL/sqlite3

Description

PL/SQL Exception Handling is used to handle errors occurring while accessing data from a student table.

Syntax

```
BEGIN
   -- PL/SQL statements
EXCEPTION
   WHEN exception_name THEN
      -- Exception handling statements
END;
/
```

EXPLANATION

- Create a relational table named STUDENT.
- Insert student records into the table.
- Use a PL/SQL block to search for a student using SELECT INTO.
- If the student ID does not exist, handle the NO_DATA_FOUND exception.
- Display the appropriate message using DBMS_OUTPUT.PUT_LINE.

CREATE TABLE

```
CREATE TABLE STUDENT (
   STUDENT_ID NUMBER,
   STUDENT_NAME VARCHAR2(30),
   MARK NUMBER
);
```

SAMPLE DATA

```
INSERT INTO STUDENT VALUES (101, 'RAVI', 85);
INSERT INTO STUDENT VALUES (102, 'ANU', 90);
INSERT INTO STUDENT VALUES (103, 'SUKAI', 75);
```

OUTPUT

Student_ID	Student_Name	Mark
101	RAVI	85
102	ANU	90
103	SUKAI	75

PL/SQL Exception Handling Program

```
SET SERVEROUTPUT ON;

DECLARE
   V_NAME STUDENT.STUDENT_NAME%TYPE;
   V_MARK STUDENT.MARK%TYPE;
BEGIN
   SELECT STUDENT_NAME, MARK
   INTO V_NAME, V_MARK
   FROM STUDENT
```

```
    WHERE STUDENT_ID = 105;

    DBMS_OUTPUT.PUT_LINE('Student Name: ' || V_NAME);
    DBMS_OUTPUT.PUT_LINE('Mark: ' || V_MARK);

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Student ID not found');
END;
/
```

OUTPUT

Student ID not found

RESULT

Thus, the PL/SQL Exception Handling program was successfully executed, and the NO_DATA_FOUND exception was successfully handled when the student record was not found.

EX. NO:13 Creation of Database Normalization (1NF, 2NF, 3NF, BCNF)

DATE:07.09.2026

AIM

To implement database normalization using 1NF, 2NF, 3NF and BCNF in a Municipality Management database to reduce data redundancy and improve data consistency.

Software Used

Oracle /MySQL/sqlite3

Description

Database normalization organizes data into 1NF, 2NF, 3NF and BCNF to reduce data redundancy and improve data consistency.

The major normal forms covered are:

- First Normal Form(1NF)
- Second Normal Form(2NF)
- Third Normal Form(3NF)
- Boyce-Codd Normal Form(BCNF)

Syntax

```
CREATE TABLE table_name (
    column1 datatype,
    column2 datatype,
    column3 datatype
);
```

EXPLANATION

- Create the MUNICIPALITY_MANAGEMENT table.
- Insert municipality-related records.
- Convert the table into **1NF** by maintaining atomic values.
- Convert 1NF into **2NF** by removing partial dependencies.
- Convert 2NF into **3NF** by removing transitive dependencies.
- Convert 3NF into **BCNF** by ensuring every determinant is a candidate key.
- Display the normalized tables and verify the output.

1NF – FIRST NORMAL FORM

Rule: Each column should contain only atomic/single values.

QUERY

```
CREATE TABLE MUNICIPALITY_MANAGEMENT (
   CITIZEN_ID NUMBER,
   CITIZEN_NAME VARCHAR2(30),
   SERVICE VARCHAR2(30)
);

INSERT INTO MUNICIPALITY_MANAGEMENT VALUES (101,'RAVI','WATER');
INSERT INTO MUNICIPALITY_MANAGEMENT VALUES (101,'RAVI','TAX');
INSERT INTO MUNICIPALITY_MANAGEMENT VALUES (102,'ANU','WATER');
SELECT * FROM MUNICIPALITY_MANAGEMENT;
```

OUTPUT

Citizen_ID	Citizen_Name	Service
101	RAVI	WATER
101	RAVI	TAX
102	ANU	WATER

2NF – SECOND NORMAL FORM

Rule: Table must be in 1NF and non-key attributes must depend on the whole primary key.

QUERY

```
CREATE TABLE CITIZEN_2NF (
   CITIZEN_ID NUMBER PRIMARY KEY,
   CITIZEN_NAME VARCHAR2(30)
);

 CREATE TABLE SERVICE_2NF (
   CITIZEN_ID NUMBER,
   SERVICE VARCHAR2(30)
);


INSERT INTO CITIZEN_2NF VALUES (101,'RAVI');
INSERT INTO CITIZEN_2NF VALUES (102,'ANU');
INSERT INTO SERVICE_2NF VALUES (101,'WATER');
INSERT INTO SERVICE_2NF VALUES (101,'TAX');
INSERT INTO SERVICE_2NF VALUES (102,'WATER');
SELECT * FROM CITIZEN_2NF;
SELECT * FROM SERVICE_2NF;
```

OUTPUT

CITIZEN_2NF

CITIZEN_ID	CITIZEN_NAME
101	RAVI
102	ANU

SERVICE_2NF

CITIZEN_ID	SERVICE
101	WATER
101	TAX
102	WATER

3NF – THIRD NORMAL FORM

Rule: Table must be in 2NF and should not contain transitive dependency.

QUERY

```
CREATE TABLE CITIZEN_3NF (CITIZEN_ID NUMBER PRIMARY
KEY,CITIZEN_NAME VARCHAR2(30),AREA_ID NUMBER);

CREATE TABLE AREA_3NF (AREA_ID NUMBER PRIMARY KEY,AREA_NAME
VARCHAR2(30));

INSERT INTO CITIZEN_3NF VALUES (101,'RAVI',10);

INSERT INTO CITIZEN_3NF VALUES (102,'ANU',20);

INSERT INTO AREA_3NF VALUES (10,'MADURAI');

INSERT INTO AREA_3NF VALUES (20,'PONDI');


SELECT * FROM CITIZEN_3NF; SELECT * FROM AREA_3NF;
```

OUTPUT

CITIZEN_3NF

CITIZEN_ID	CITIZEN_NAME	AREA_ID
101	RAVI	10
102	ANU	20

AREA_3NF

AREA_ID	AREA_NAME
10	MADURAI
20	PONDI

BCNF – BOYCE-CODD NORMAL FORM

Rule: Every determinant must be a candidate key.

QUERY

```
CREATE TABLE MUNICIPAL_SERVICE (
   SERVICE_ID NUMBER PRIMARY KEY,
   SERVICE_NAME VARCHAR2(30),
   OFFICER_ID NUMBER
);

CREATE TABLE MUNICIPAL_OFFICER (
   OFFICER_ID NUMBER PRIMARY KEY,
   OFFICER_NAME VARCHAR2(30)
);

INSERT INTO MUNICIPAL_SERVICE VALUES (1,'WATER',501);
INSERT INTO MUNICIPAL_SERVICE VALUES (2,'TAX',502);

INSERT INTO MUNICIPAL_OFFICER VALUES (501,'KUMAR');
INSERT INTO MUNICIPAL_OFFICER VALUES (502,'PRIYA');
```

```
SELECT * FROM MUNICIPAL_SERVICE;

SELECT * FROM MUNICIPAL_OFFICER;
```

OUTPUT

MUNICIPAL_SERVICE

SERVICE_ID	SERVICE_NAME	OFFICER_ID
1	WATER	501
2	TAX	502

MUNICIPAL_OFFICER

OFFICER_ID	OFFICER_NAME
501	KUMAR
502	PRIYA

RESULT

Thus, the **MUNICIPALITY MANAGEMENT database** was successfully normalized into **1NF, 2NF, 3NF and BCNF**, reducing data redundancy and maintaining data consistency.

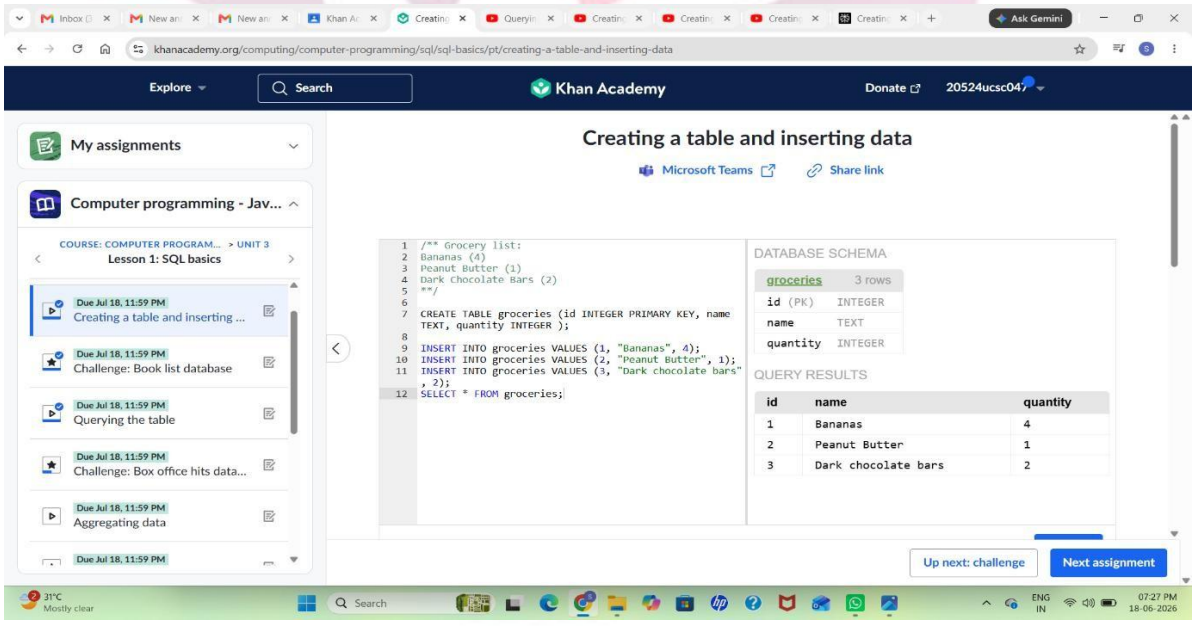
KHAN ACADEMY

1. CREATING A TABLE AND INSERTING DATA

PROGRAM

```
CREATE TABLE groceries (id INTEGER PRIMARY KEY, name TEXT, quantity INTEGER );
INSERT INTO groceries VALUES (1, "Bananas", 4);
INSERT INTO groceries VALUES (2, "Peanut Butter", 1);
INSERT INTO groceries VALUES (3, "Dark
chocolate bars", 2);
SELECT * FROM groceries;
```

OUTPUT



2. BOOK LIST DATABASE

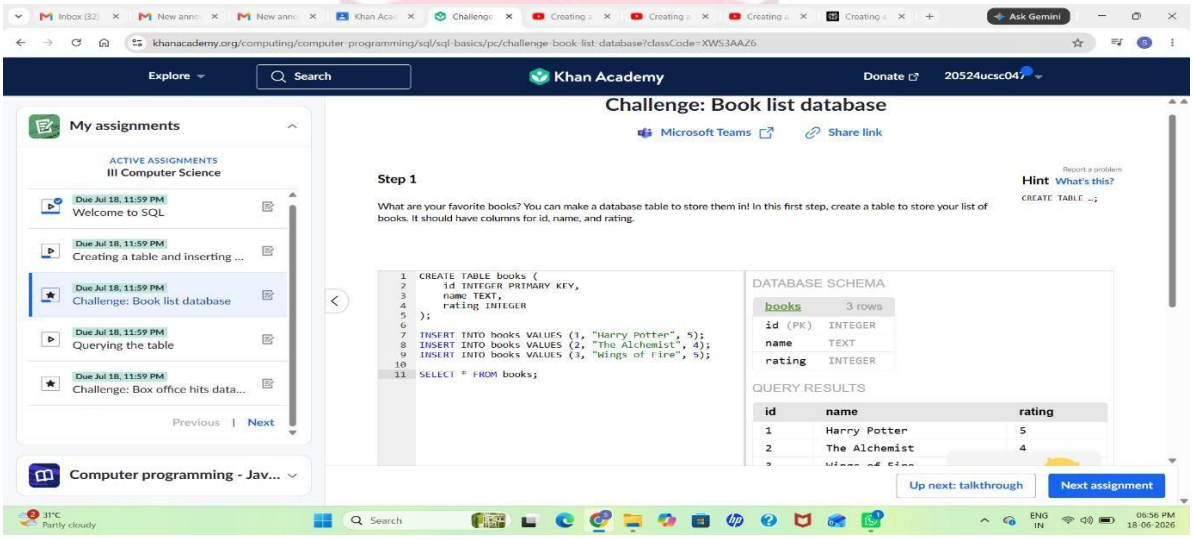
PROGRAM

```
CREATE TABLE books (
   id INTEGER PRIMARY KEY,
   name TEXT, rating
   INTEGER
);

INSERT INTO books VALUES (1, "Harry Potter", 5); INSERT
INTO books VALUES (2, "The Alchemist", 4); INSERT INTO
books VALUES (3, "Wings of Fire", 5);

SELECT * FROM books;
```

OUTPUT



3. QUERYING THE TABLE

PROGRAM

CREATE TABLE groceries (id INTEGER PRIMARY KEY, name TEXT, quantity INTEGER, aisle INTEGER);

INSERT INTO groceries VALUES (1, "Bananas", 4, 7); INSERT

INTO groceries VALUES(2, "Peanut Butter", 1, 2);

INSERT INTO groceries VALUES(3, "Dark Chocolate Bars", 2, 2); INSERT

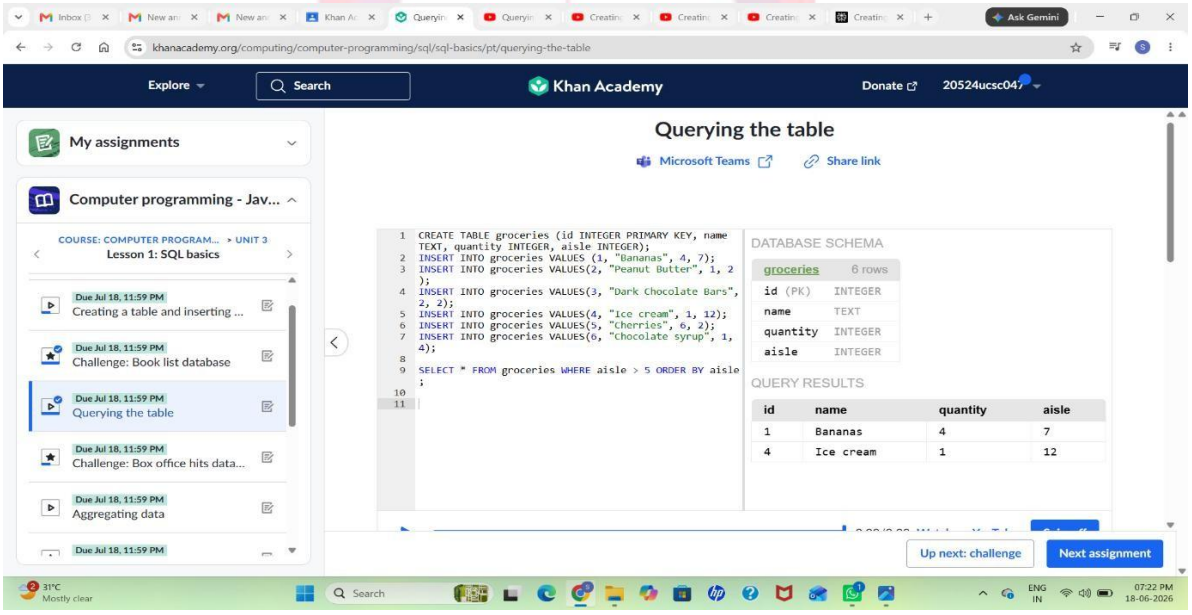
INTO groceries VALUES(4, "Ice cream", 1, 12);

INSERT INTO groceries VALUES(5, "Cherries", 6, 2); INSERT INTO

groceries VALUES(6, "Chocolate syrup", 1, 4);

SELECT * FROM groceries WHERE aisle > 5 ORDER BY aisle;

OUTPUT



4. BOX OFFICE HITS DATABASE

PROGRAM

CREATE TABLE movies (id INTEGER PRIMARY KEY, name TEXT, release_year INTEGER);

INSERT INTO movies VALUES (1, "Avatar", 2009); INSERT

INTO movies VALUES (2, "Titanic", 1997);

INSERT INTO movies VALUES (3, "Star Wars: Episode IV - A New Hope", 1977); INSERT INTO

movies VALUES (4, "Shrek 2", 2004);

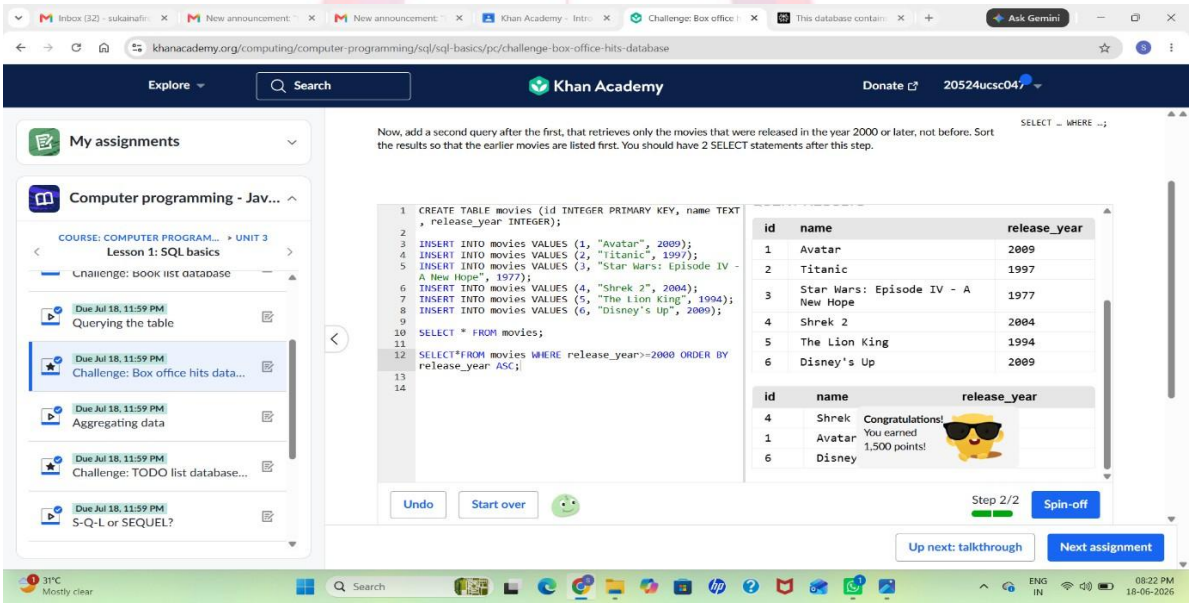
INSERT INTO movies VALUES (5, "The Lion King", 1994); INSERT

INTO movies VALUES (6, "Disney's Up", 2009);

SELECT * FROM movies;

SELECT*FROM movies WHERE release_year>=2000 ORDER BY release_year ASC;

OUTPUT



5. AGGREGATING DATA

PROGRAM

CREATE TABLE groceries (id INTEGER PRIMARY KEY, name TEXT, quantity INTEGER, aisle INTEGER);

INSERT INTO groceries VALUES (1, "Bananas", 56, 7); INSERT

INTO groceries VALUES(2, "Peanut Butter", 1, 2);

INSERT INTO groceries VALUES(3, "Dark Chocolate Bars", 2, 2); INSERT

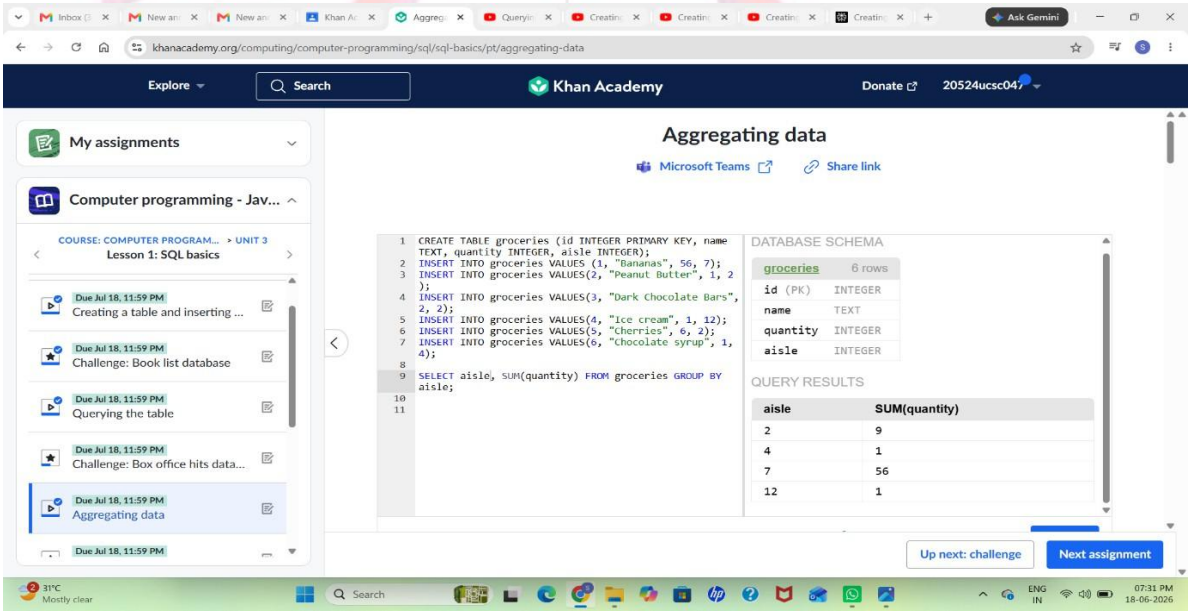
INTO groceries VALUES(4, "Ice cream", 1, 12);

INSERT INTO groceries VALUES(5, "Cherries", 6, 2); INSERT INTO

groceries VALUES(6, "Chocolate syrup", 1, 4);

SELECT aisle, SUM(quantity) FROM groceries GROUP BY aisle;

OUTPUT



6. TODO LIST DATABASE STATS

PROGRAM

CREATE TABLE todo_list (id INTEGER PRIMARY KEY, item TEXT, minutes INTEGER); INSERT INTO todo_list VALUES (1, "Wash the dishes", 15);

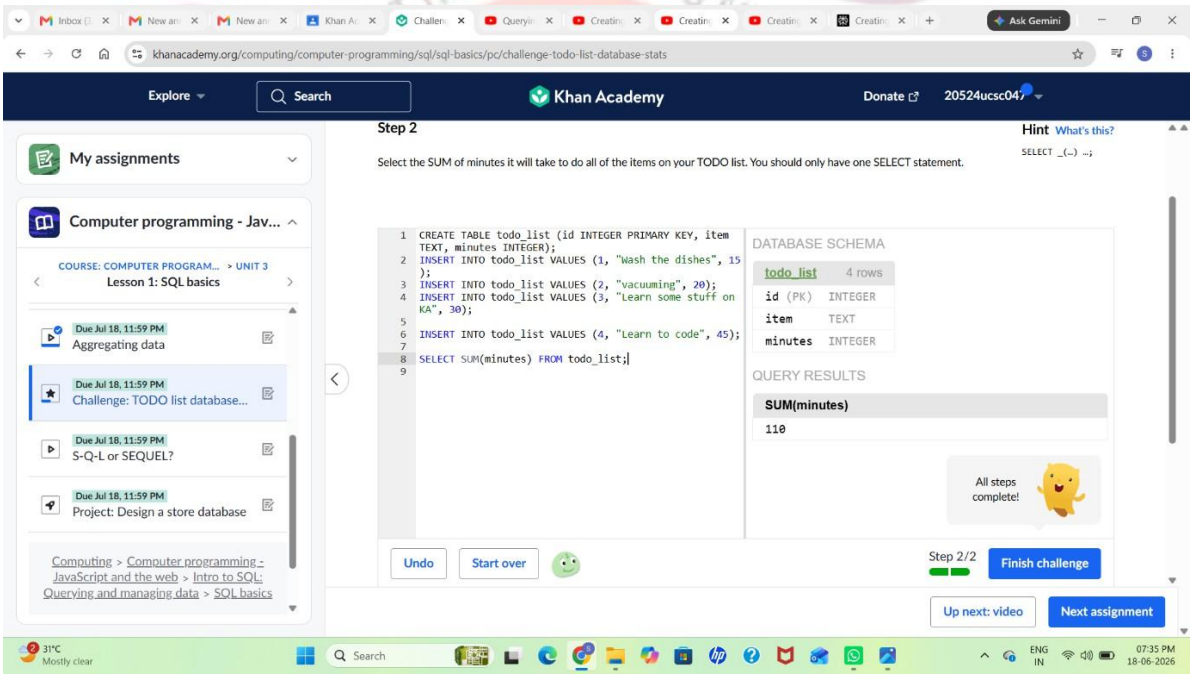
INSERT INTO todo_list VALUES (2, "vacuuming", 20);

INSERT INTO todo_list VALUES (3, "Learn some stuff on KA", 30);

INSERT INTO todo_list VALUES (4, "Learn to code", 45);

SELECT SUM(minutes) FROM todo_list;

OUTPUT



7. PROJECT: DESIGN A STORE DATABASE

PROGRAM

```
CREATE TABLE books (
   id INTEGER PRIMARY KEY,
   name TEXT, author
   TEXT, genre TEXT,
   price REAL, stock
   INTEGER
);
```

```
INSERT INTO books VALUES (1, "Harry Potter 1", "J.K. Rowling", "Fantasy", 12.99, 50);
INSERT INTO books VALUES (2, "Harry Potter 2", "J.K. Rowling", "Fantasy", 13.99, 45);
INSERT INTO books VALUES (3, "Harry Potter 3", "J.K. Rowling", "Fantasy", 14.99, 40);
INSERT INTO books VALUES (4, "The Hobbit", "J.R.R. Tolkien", "Fantasy", 10.99, 30);
INSERT INTO books VALUES (5, "Lord of the Rings", "J.R.R. Tolkien", "Fantasy", 19.99, 25);
INSERT INTO books VALUES (6, "The Alchemist", "Paulo Coelho", "Fiction", 9.99, 60); INSERT
INTO books VALUES (7, "Atomic Habits", "James Clear", "Self-Help", 15.99, 55);
INSERT INTO books VALUES (8, "Rich Dad Poor Dad", "Robert Kiyosaki", "Finance", 11.99,
35);
INSERT INTO books VALUES (9, "Think and Grow Rich", "Napoleon Hill", "Finance", 8.99, 40);
INSERT INTO books VALUES (10, "Deep Work", "Cal Newport", "Self-Help", 13.99, 20);
INSERT INTO books VALUES (11, "1984", "George Orwell", "Dystopian", 7.99, 70);
INSERT INTO books VALUES (12, "Animal Farm", "George Orwell", "Dystopian", 6.99, 65);
INSERT INTO books VALUES (13, "The Great Gatsby", "F. Scott Fitzgerald", "Classic", 8.99, 30);
INSERT INTO books VALUES (14, "To Kill a Mockingbird", "Harper Lee", "Classic", 9.99, 25);
INSERT INTO books VALUES (15, "The Diary of a Young Girl", "Anne Frank", "Biography",
10.99, 50);
```

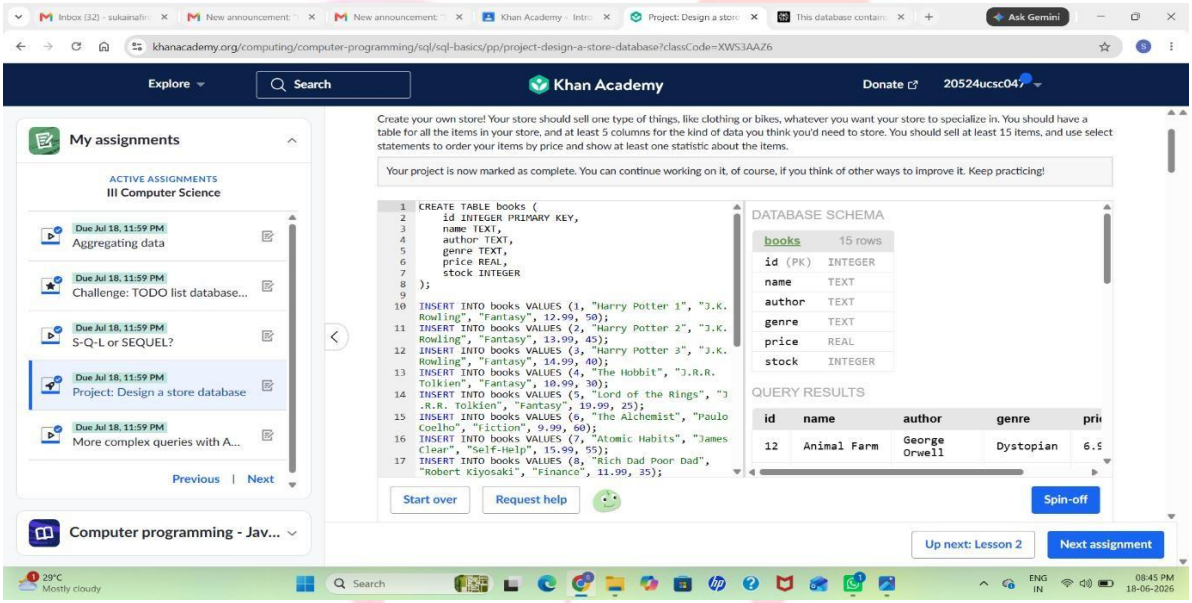
SELECT * FROM books ORDER BY price ASC;

SELECT MAX(price) FROM books; SELECT

MIN(price) FROM books; SELECT AVG(price)

FROM books;

OUTPUT



8. MORE COMPLEX QUERIES WITH AND/OR

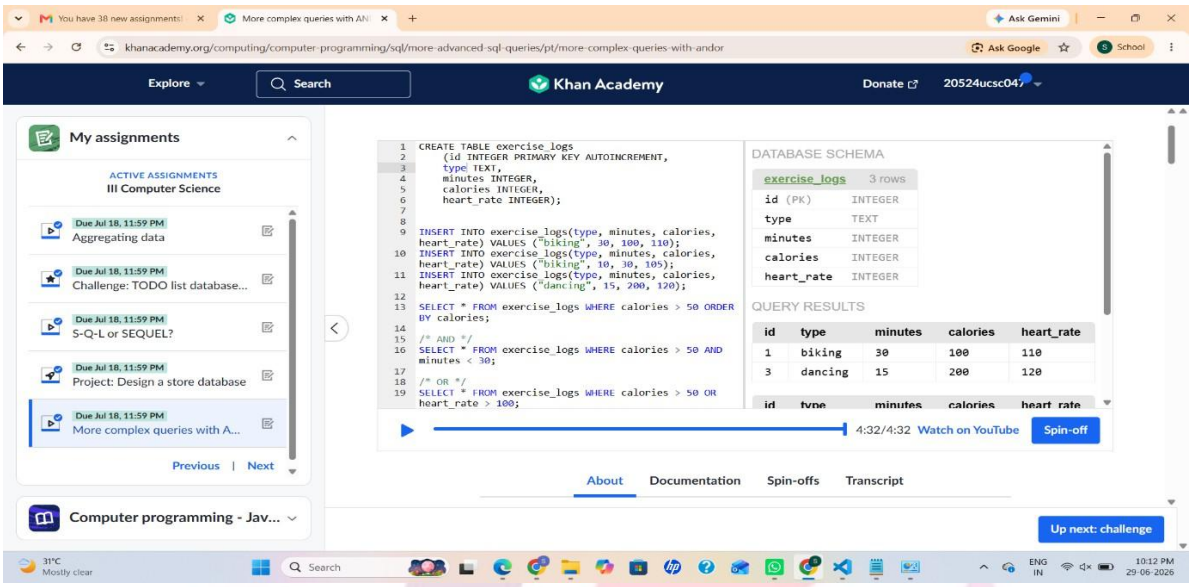
PROGRAM

```
CREATE TABLE exercise_logs
   (id INTEGER PRIMARY KEY AUTOINCREMENT,
   type TEXT, minutes
   INTEGER, calories
   INTEGER,
   heart_rate INTEGER);


INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("biking", 30, 100, 110);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("biking", 10, 30, 105);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("dancing", 15, 200,
120);


SELECT * FROM exercise_logs WHERE calories > 50 ORDER BY calories;
/* AND */
SELECT * FROM exercise_logs WHERE calories > 50 AND minutes < 30;
/* OR */
SELECT * FROM exercise_logs WHERE calories > 50 OR heart_rate > 100;
```

OUTPUT


Explore
Search
Khan Academy
Donate
My assignments
ACTIVE ASSIGNMENTS
III Computer Science
Due Jul 18, 11:59 PM
Aggregating data
Due Jul 18, 11:59 PM
Challenge: TODO list database...
Due Jul 18, 11:59 PM
S-Q-L or SEQUEL?
Due Jul 18, 11:59 PM
Project: Design a store database
Due Jul 18, 11:59 PM
More complex queries with A...
Previous | Next
Computer programming - Jav...
CREATE TABLE exercise_logs
(id INTEGER PRIMARY KEY AUTOINCREMENT,
type TEXT,
minutes INTEGER,
calories INTEGER,
heart_rate INTEGER);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("biking", 30, 100, 110);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("biking", 10, 30, 105);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("dancing", 15, 200, 120);
SELECT * FROM exercise_logs WHERE calories > 50 ORDER BY calories;
/* AND */
SELECT * FROM exercise_logs WHERE calories > 50 AND minutes < 30;
/* OR */
SELECT * FROM exercise_logs WHERE calories > 50 OR heart_rate > 100;
DATABASE SCHEMA
exercise_logs 3 rows
id (PK) INTEGER
type TEXT
minutes INTEGER
calories INTEGER
heart_rate INTEGER
QUERY RESULTS
id type minutes calories heart_rate
1 biking 30 100 110
3 dancing 15 200 120
4:32/4:32 Watch on YouTube
Spin-off
About
Documentation
Spin-offs
Transcript
Up next: challenge

9. KARAOKE SONG SELECTOR

PROGRAM

```
CREATE TABLE songs (
   id INTEGER PRIMARY KEY,
   title TEXT, artist
   TEXT, mood TEXT,
   duration INTEGER, released
   INTEGER);


INSERT INTO songs (title, artist, mood, duration, released) VALUES
   ("Bohemian Rhapsody", "Queen", "epic", 60, 1975);
INSERT INTO songs (title, artist, mood, duration, released) VALUES
   ("Let it go", "Idina Menzel", "epic", 227, 2013);
INSERT INTO songs (title, artist, mood, duration, released) VALUES ("I
   will survive", "Gloria Gaynor", "epic", 198, 1978);
INSERT INTO songs (title, artist, mood, duration, released) VALUES
   ("Twist and Shout", "The Beatles", "happy", 152, 1963);
INSERT INTO songs (title, artist, mood, duration, released) VALUES ("La
   Bamba", "Ritchie Valens", "happy", 166, 1958);
INSERT INTO songs (title, artist, mood, duration, released)
   VALUES ("I will always love you", "Whitney Houston", "epic", 273, 1992); INSERT
INTO songs (title, artist, mood, duration, released)
   VALUES ("Sweet Caroline", "Neil Diamond", "happy", 201, 1969); INSERT
INTO songs (title, artist, mood, duration, released)
   VALUES ("Call me maybe", "Carly Rae Jepsen", "happy", 193, 2011);

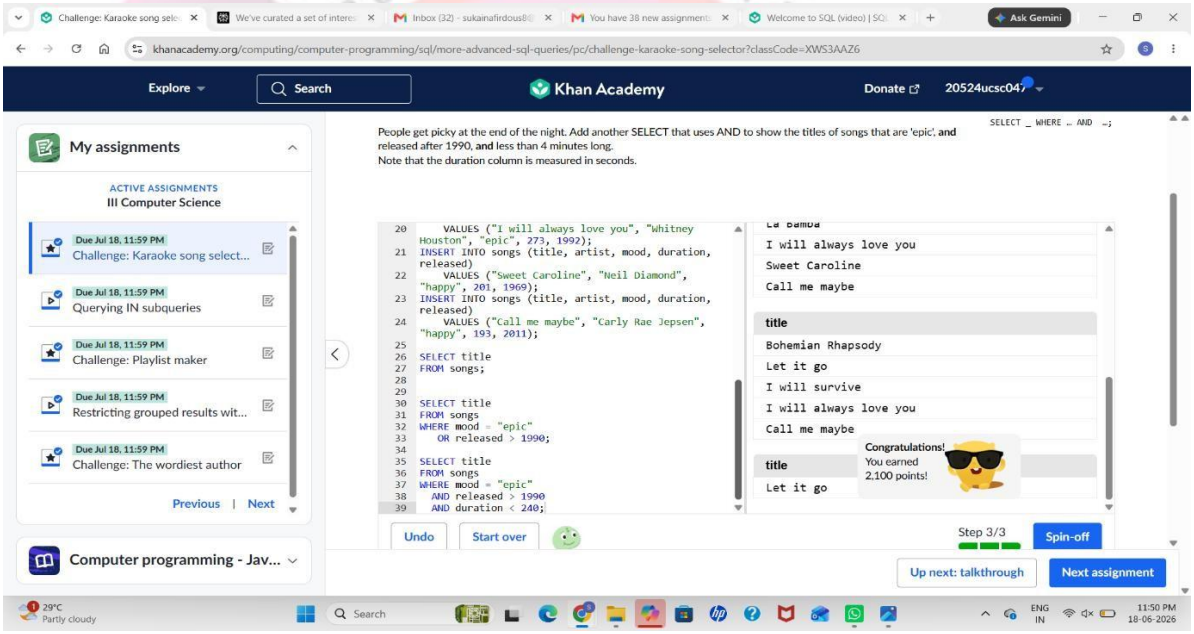

SELECT title
```

```
FROM songs;

SELECT title FROM
songs
WHERE mood = "epic" OR
  released > 1990;

SELECT title FROM
songs
WHERE mood = "epic" AND
 released > 1990 AND duration <
 240;
```

OUTPUT



10. QUERYING IN SUBQUERIES

PROGRAM

```
CREATE TABLE exercise_logs
   (id INTEGER PRIMARY KEY AUTOINCREMENT,
   type TEXT, minutes
   INTEGER, calories
   INTEGER,
   heart_rate INTEGER);


INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("biking", 30, 100, 110);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("biking", 10, 30, 105);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("dancing", 15, 200,
120);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("tree climbing", 30, 70,
90);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("tree climbing", 25, 72,
80);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("rowing", 30, 70, 90);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("hiking", 60, 80, 85);


SELECT * FROM exercise_logs WHERE type = "biking" OR type = "hiking" OR type = "tree
climbing" OR type = "rowing";


/* IN */
SELECT * FROM exercise_logs WHERE type IN ("biking", "hiking", "tree climbing", "rowing");



CREATE TABLE drs_favorites (id
```

```
INTEGER PRIMARY KEY,
    type TEXT, reason
    TEXT);

INSERT INTO drs_favorites(type, reason) VALUES ("biking", "Improves endurance and flexibility.");
INSERT INTO drs_favorites(type, reason) VALUES ("hiking", "Increases cardiovascular health.");

SELECT type FROM drs_favorites;

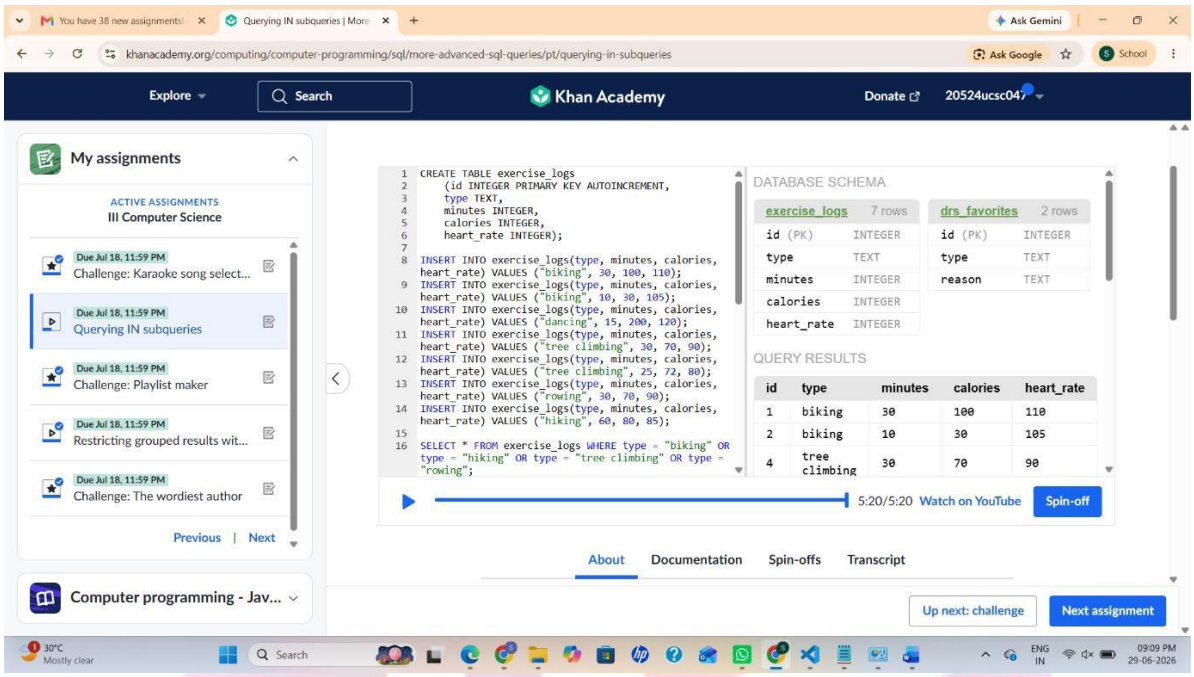
SELECT * FROM exercise_logs WHERE type IN ( SELECT
    type FROM drs_favorites);

SELECT * FROM exercise_logs WHERE type IN (
    SELECT type FROM drs_favorites WHERE reason = "Increases cardiovascular health");

/* LIKE */

SELECT * FROM exercise_logs WHERE type IN (
    SELECT type FROM drs_favorites WHERE reason LIKE "%cardiovascular%");
```

OUTPUT


Querying IN subqueries | More
khanacademy.org/computing/computer-programming/sql/more-advanced-sql-queries/pt/querying-in-subqueries
Explore
Search
Khan Academy
Donate
20524ucsc047
My assignments
ACTIVE ASSIGNMENTS
III Computer Science
Due Jul 18, 11:59 PM
Challenge: Karaoke song select...
Due Jul 18, 11:59 PM
Querying IN subqueries
Due Jul 18, 11:59 PM
Challenge: Playlist maker
Due Jul 18, 11:59 PM
Restricting grouped results wit...
Due Jul 18, 11:59 PM
Challenge: The wordiest author
Previous | Next
Computer programming - Jav...
CREATE TABLE exercise_logs
(id INTEGER PRIMARY KEY AUTOINCREMENT,
type TEXT,
minutes INTEGER,
calories INTEGER,
heart_rate INTEGER);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("biking", 30, 100, 110);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("biking", 10, 30, 105);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("dancing", 15, 200, 120);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("tree climbing", 30, 70, 90);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("tree climbing", 25, 72, 80);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("rowing", 30, 70, 90);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("hiking", 60, 80, 85);
SELECT * FROM exercise_logs WHERE type = "biking" OR type = "hiking" OR type = "tree climbing" OR type = "rowing";
DATABASE SCHEMA
exercise_logs 7 rows
id (PK) INTEGER
type TEXT
minutes INTEGER
calories INTEGER
heart_rate INTEGER
drs_favorites 2 rows
id (PK) INTEGER
type TEXT
reason TEXT
QUERY RESULTS
id type minutes calories heart_rate
1 biking 30 100 110
2 biking 10 30 105
4 tree climbing 30 70 90
5:20/5:20 Watch on YouTube
Spin-off
About
Documentation
Spin-offs
Transcript
Up next: challenge
Next assignment

11. PLAYLIST MAKER

PROGRAM

```
CREATE TABLE artists (
   id INTEGER PRIMARY KEY AUTOINCREMENT,
   name TEXT, country
   TEXT, genre TEXT);


     INSERT INTO artists (name, country, genre)
 VALUES ("Taylor Swift", "US", "Pop"); INSERT
          INTO artists (name, country, genre)
   VALUES ("Led Zeppelin", "US", "Hard rock"); INSERT
INTO artists (name, country, genre)
   VALUES ("ABBA", "Sweden", "Disco");
INSERT INTO artists (name, country, genre)
   VALUES ("Queen", "UK", "Rock");
INSERT INTO artists (name, country, genre) VALUES
   ("Celine Dion", "Canada", "Pop");
INSERT INTO artists (name, country, genre) VALUES
   ("Meatloaf", "US", "Hard rock");
INSERT INTO artists (name, country, genre) VALUES
   ("Garth Brooks", "US", "Country");
INSERT INTO artists (name, country, genre) VALUES
   ("Shania Twain", "Canada", "Country");
INSERT INTO artists (name, country, genre)
   VALUES ("Rihanna", "US", "Pop");
```

```
INSERT INTO artists (name, country, genre) VALUES
("Guns N' Roses", "US", "Hard rock");

INSERT INTO artists (name, country, genre)
VALUES ("Gloria Estefan", "US", "Pop");
 INSERT INTO artists (name, country, genre)
VALUES ("Bob Marley", "Jamaica", "Reggae");

CREATE TABLE songs (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    artist TEXT, title
    TEXT);

INSERT INTO songs (artist, title)
    VALUES ("Taylor Swift", "Shake it off");
INSERT INTO songs (artist, title)
    VALUES ("Rihanna", "Stay");
INSERT INTO songs (artist, title)
    VALUES ("Celine Dion", "My heart will go on"); INSERT
INTO songs (artist, title)
    VALUES ("Celine Dion", "A new day has come"); INSERT
INTO songs (artist, title)
    VALUES ("Shania Twain", "Party for two"); INSERT
INTO songs (artist, title)
    VALUES ("Gloria Estefan", "Conga");
INSERT INTO songs (artist, title)
    VALUES ("Led Zeppelin", "Stairway to heaven");

INSERT INTO songs (artist, title)
```

VALUES ("ABBA", "Mamma mia");

INSERT INTO songs (artist, title) VALUES

("Queen", "Bicycle Race");

INSERT INTO songs (artist, title)
VALUES ("Queen", "Bohemian Rhapsody"); INSERT

INTO songs (artist, title)

VALUES ("Guns N' Roses", "Don't cry");

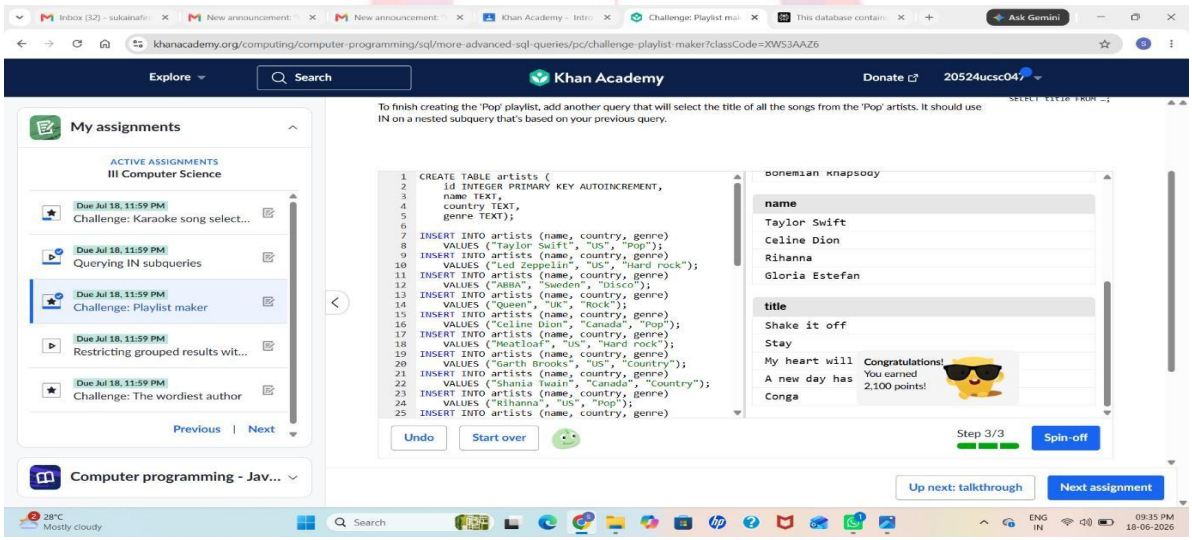
SELECT title FROM songs WHERE

artist = "Queen";

SELECT name FROM artists WHERE genre = 'Pop';

SELECT title FROM songs WHERE artist IN (SELECT name FROM artists WHERE genre = 'Pop');

OUTPUT



12. RESTRICTING GROUPING RESULTS WITH HAVING

PROGRAM

```
CREATE TABLE exercise_logs
   (id INTEGER PRIMARY KEY AUTOINCREMENT,
   type TEXT, minutes
   INTEGER, calories
   INTEGER,
   heart_rate INTEGER);


INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("biking", 30, 115, 110);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("biking", 10, 45, 105);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("dancing", 15, 200,
120);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("dancing", 15, 165,
120);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("tree climbing", 30, 70,
90);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("tree climbing", 25, 72,
80);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("rowing", 30, 70, 90);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("hiking", 60, 80, 85);


SELECT * FROM exercise_logs;


SELECT type, SUM(calories) AS total_calories FROM exercise_logs GROUP BY type;

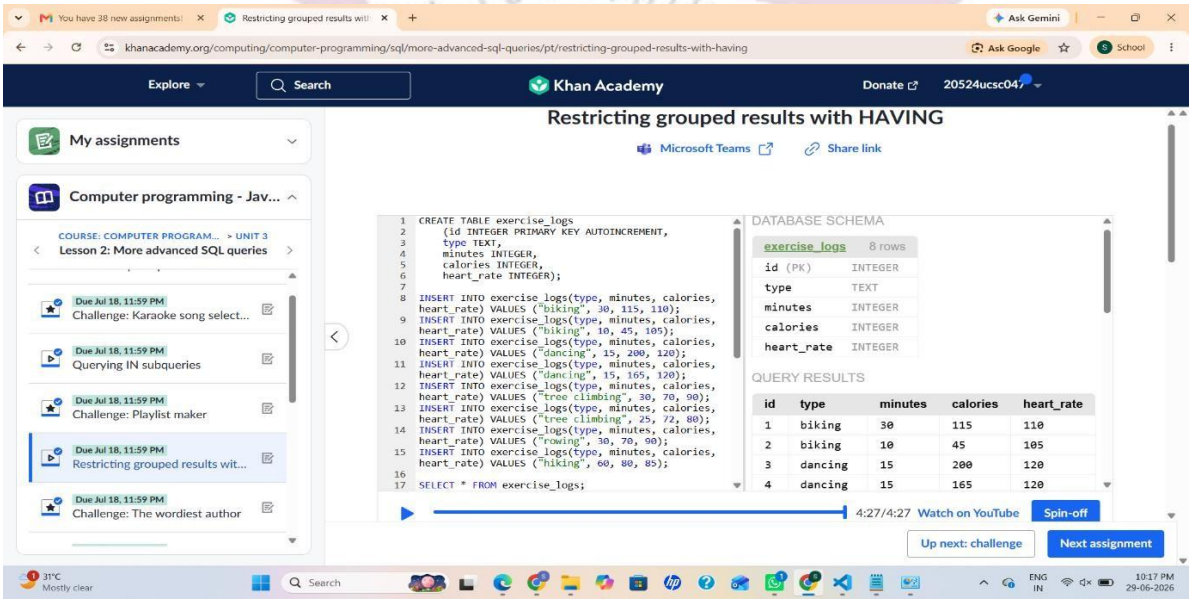

SELECT type, SUM(calories) AS total_calories FROM exercise_logs
```

```
GROUP BY type
HAVING total_calories > 150
;
```

```
SELECT type, AVG(calories) AS avg_calories FROM exercise_logs
GROUP BY type
HAVING avg_calories > 70
;
```

```
SELECT type FROM exercise_logs GROUP BY type HAVING COUNT(*) >= 2;
```

OUTPUT



13. THE WORDIEST AUTHOR

PROGRAM

```
CREATE TABLE books (
   id INTEGER PRIMARY KEY AUTOINCREMENT,
   author TEXT, title
   TEXT,
   words INTEGER);


INSERT INTO books (author, title, words)
   VALUES ("J.K. Rowling", "Harry Potter and the Philosopher's Stone", 79944);
INSERT INTO books (author, title, words)
   VALUES ("J.K. Rowling", "Harry Potter and the Chamber of Secrets", 85141);
INSERT INTO books (author, title, words)
   VALUES ("J.K. Rowling", "Harry Potter and the Prisoner of Azkaban", 107253);
INSERT INTO books (author, title, words)
   VALUES ("J.K. Rowling", "Harry Potter and the Goblet of Fire", 190637); INSERT
INTO books (author, title, words)
   VALUES ("J.K. Rowling", "Harry Potter and the Order of the Phoenix", 257045); INSERT
INTO books (author, title, words)
   VALUES ("J.K. Rowling", "Harry Potter and the Half-Blood Prince", 168923);
INSERT INTO books (author, title, words)
   VALUES ("J.K. Rowling", "Harry Potter and the Deathly Hallows", 197651);


INSERT INTO books (author, title, words)
   VALUES ("Stephenie Meyer", "Twilight", 118501); INSERT
INTO books (author, title, words)
   VALUES ("Stephenie Meyer", "New Moon", 132807);
```

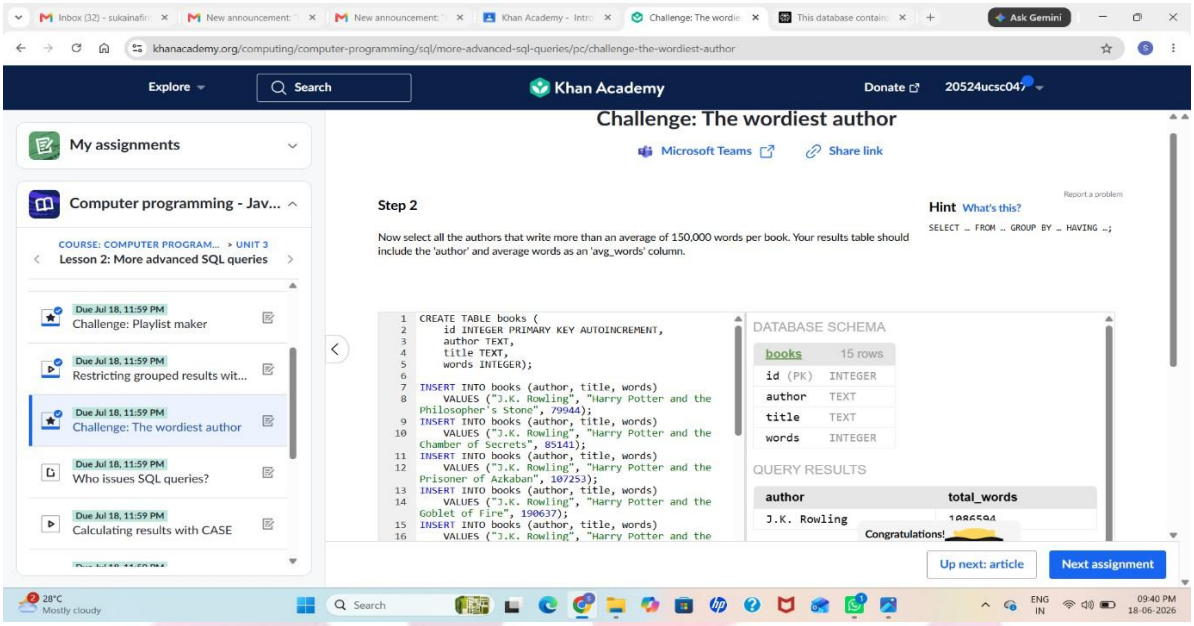
```
    INSERT INTO books (author, title, words)
    VALUES ("Stephenie Meyer", "Eclipse", 147930); INSERT
INTO books (author, title, words)
    VALUES ("Stephenie Meyer", "Breaking Dawn", 192196);

INSERT INTO books (author, title, words)
    VALUES ("J.R.R. Tolkien", "The Hobbit", 95022); INSERT
INTO books (author, title, words)
    VALUES ("J.R.R. Tolkien", "Fellowship of the Ring", 177227); INSERT
INTO books (author, title, words)
    VALUES ("J.R.R. Tolkien", "Two Towers", 143436);
INSERT INTO books (author, title, words)
    VALUES ("J.R.R. Tolkien", "Return of the King", 134462);

SELECT author, SUM(words) AS total_words FROM
books
GROUP BY author
HAVING total_words > 1000000;

SELECT author, AVG(words) AS avg_words FROM books
GROUP BY author
HAVING avg_words > 150000;
```

OUTPUT


Challenge: The wordiest author
Step 2
Now select all the authors that write more than an average of 150,000 words per book. Your results table should include the 'author' and average words as an 'avg_words' column.
Hint
DATABASE SCHEMA
QUERY RESULTS
author
total_words
J.K. Rowling
Congratulations!
Up next: article
Next assignment

14. CALCULATING RESULTS WITH CASE

PROGRAM

```
CREATE TABLE exercise_logs
   (id INTEGER PRIMARY KEY AUTOINCREMENT,
   type TEXT, minutes
   INTEGER, calories
   INTEGER,
   heart_rate INTEGER);


INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("biking", 30, 100, 110);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("biking", 10, 30, 105);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("dancing", 15, 200,
120);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("dancing", 15, 165,
120);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("tree climbing", 30, 70,
90);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("tree climbing", 25, 72,
80);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("rowing", 30, 70, 90);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("hiking", 60, 80, 85);


SELECT * FROM exercise_logs;

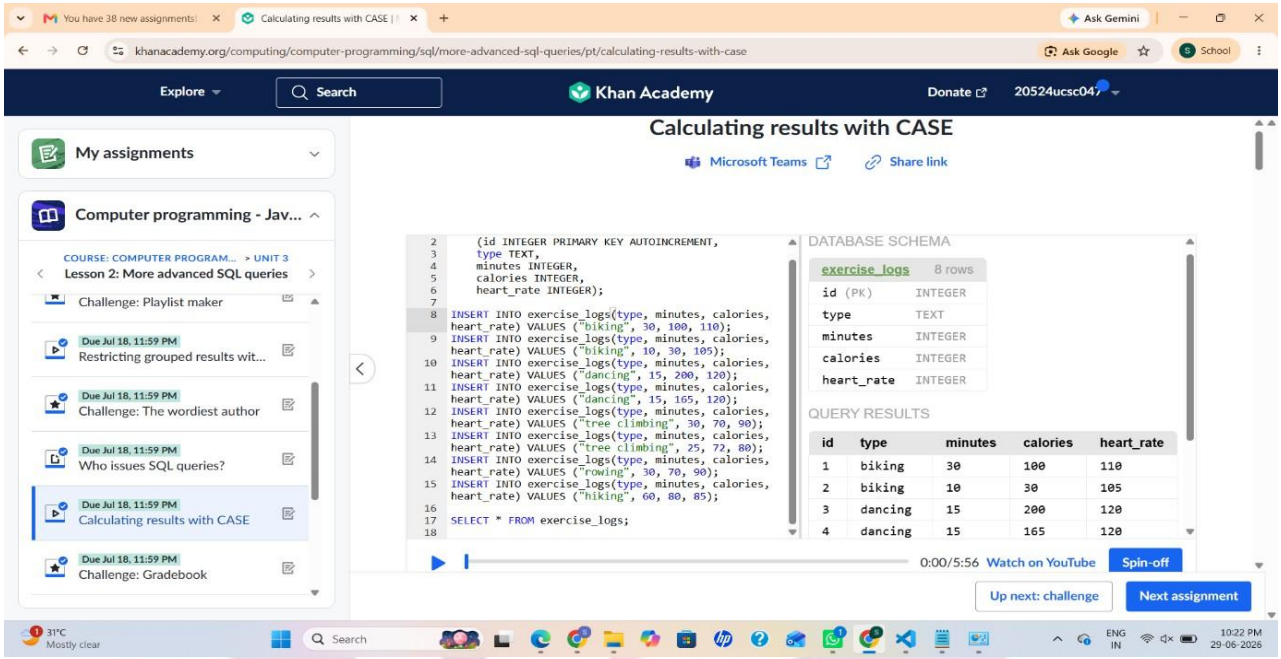

SELECT COUNT(*) FROM exercise_logs WHERE heart_rate > 220 - 30;
/* 50-90% of max*/
```

```
SELECT COUNT(*) FROM exercise_logs WHERE heart_rate
>= ROUND(0.50 * (220-30))
   AND  heart_rate <= ROUND(0.90 * (220-30));


/* CASE */
SELECT type, heart_rate, CASE
       WHEN heart_rate > 220-30 THEN "above max"
       WHEN heart_rate > ROUND(0.90 * (220-30)) THEN "above target" WHEN
       heart_rate > ROUND(0.50 * (220-30)) THEN "within target" ELSE "below
       target"
   END as "hr_zone" FROM
exercise_logs;


SELECT COUNT(*), CASE
       WHEN heart_rate > 220-30 THEN "above max"
       WHEN heart_rate > ROUND(0.90 * (220-30)) THEN "above target" WHEN
       heart_rate > ROUND(0.50 * (220-30)) THEN "within target" ELSE "below
       target"
   END as "hr_zone" FROM
exercise_logs  GROUP  BY
hr_zone;
```

OUTPUT


Calculating results with CASE
Khan Academy
My assignments
Computer programming - Jav...
Lesson 2: More advanced SQL queries
Challenge: Playlist maker
Restricting grouped results wit...
Challenge: The wordiest author
Who issues SQL queries?
Calculating results with CASE
Challenge: Gradebook
Microsoft Teams
Share link
DATABASE SCHEMA
exercise_logs 8 rows
id (PK) INTEGER
type TEXT
minutes INTEGER
calories INTEGER
heart_rate INTEGER
QUERY RESULTS
id type minutes calories heart_rate
1 biking 30 100 110
2 biking 10 30 105
3 dancing 15 200 120
4 dancing 15 165 120
0:00/5:56 Watch on YouTube
Spin-off
Up next: challenge
Next assignment

15.GRADEBOOK

PROGRAM

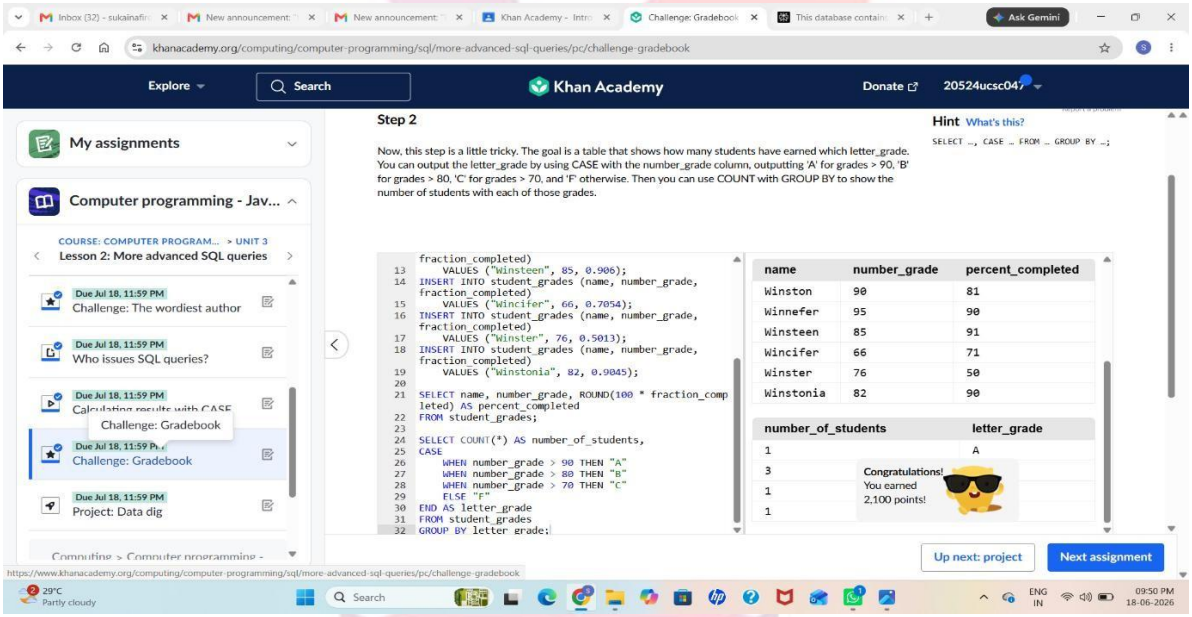
```
CREATE TABLE student_grades (
   id INTEGER PRIMARY KEY AUTOINCREMENT,
   name TEXT, number_grade
   INTEGER,
   fraction_completed REAL);

INSERT INTO student_grades (name, number_grade, fraction_completed) VALUES
   ("Winston", 90, 0.805);
INSERT INTO student_grades (name, number_grade, fraction_completed) VALUES
   ("Winnefer", 95, 0.901);
INSERT INTO student_grades (name, number_grade, fraction_completed) VALUES
   ("Winsteen", 85, 0.906);
INSERT INTO student_grades (name, number_grade, fraction_completed) VALUES
   ("Wincifer", 66, 0.7054);
INSERT INTO student_grades (name, number_grade, fraction_completed) VALUES
   ("Winster", 76, 0.5013);
INSERT INTO student_grades (name, number_grade, fraction_completed) VALUES
   ("Winstonia", 82, 0.9045);

SELECT name, number_grade, ROUND(100 * fraction_completed) AS percent_completed FROM
student_grades;
```

```
SELECT COUNT(*) AS number_of_students, CASE
    WHEN number_grade > 90 THEN "A"
    WHEN number_grade > 80 THEN "B" WHEN
    number_grade > 70 THEN "C" ELSE "F"
END AS letter_grade FROM
student_grades GROUP BY
letter_grade;
```

OUTPUT



16. PROJECT: DATA DIG

PROGRAM

```
CREATE TABLE astronauts ( Name
  TEXT PRIMARY KEY, Year
  INTEGER,
  GroupNum INTEGER, Status
  TEXT, Birth_Date TEXT,
  Birth_Place TEXT, Gender
  TEXT, Alma_Mater TEXT,
  Undergraduate_Major TEXT,
  Graduate_Major TEXT, Military_Rank
  TEXT, Military_Branch TEXT,
  Space_Flights INTEGER,
  Space_Flight_hr INTEGER,
  Space_Walks INTEGER,
  Space_Walks_hr REAL, Missions
  TEXT,
  Death_Date TEXT,
  Death_Mission TEXT
);

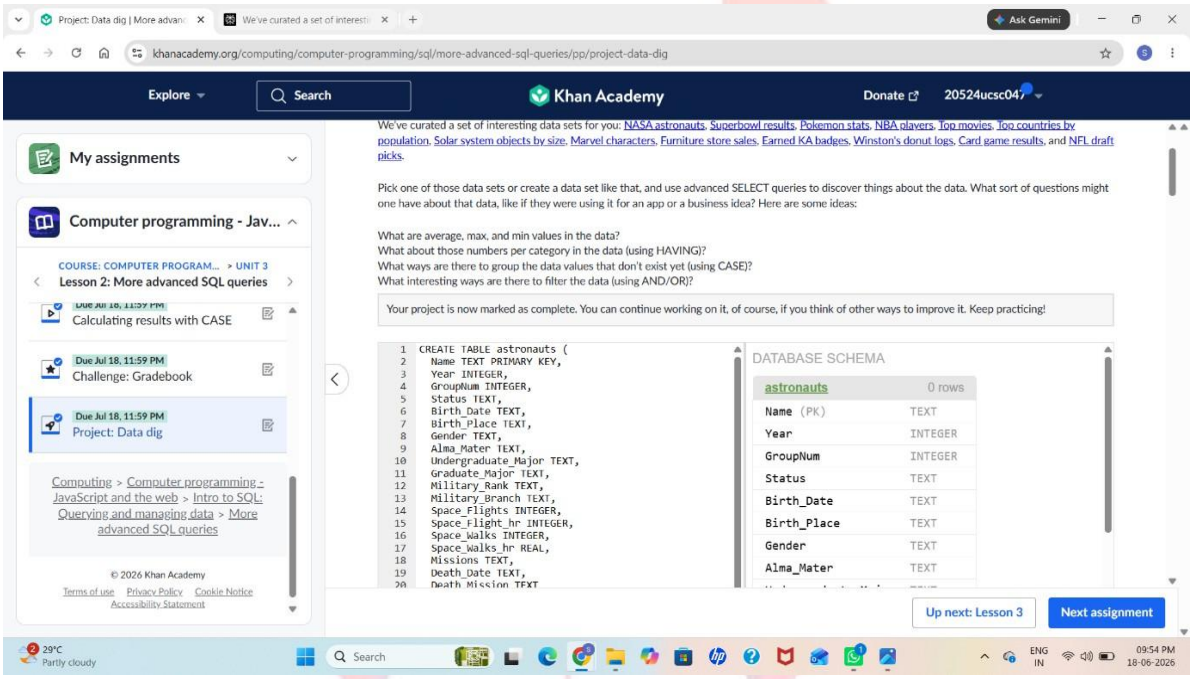
SELECT
  Gender,
  AVG(Space_Flights) AS avg_flights, MAX(Space_Flights)
  AS max_flights,
```

MIN(Space_Flights) AS min_flights FROM

astronauts

GROUP BY Gender;

OUTPUT



17. JOINING RELATED TABLES

PROGRAM

```
CREATE TABLE students (id INTEGER PRIMARY KEY,
   first_name TEXT,
   last_name TEXT, email
   TEXT, phone TEXT,
   birthdate TEXT);

INSERT INTO students (first_name, last_name, email, phone, birthdate) VALUES
   ("Peter", "Rabbit", "peter@rabbit.com", "555-6666", "2002-06-24");
INSERT INTO students (first_name, last_name, email, phone, birthdate)
   VALUES ("Alice", "Wonderland", "alice@wonderland.com", "555-4444", "2002-07-04");

CREATE TABLE student_grades (id INTEGER PRIMARY KEY, student_id
   INTEGER,
   test TEXT,
   grade INTEGER);

INSERT INTO student_grades (student_id, test, grade)
   VALUES (1, "Nutrition", 95);
INSERT INTO student_grades (student_id, test, grade)
   VALUES (2, "Nutrition", 92);
INSERT INTO student_grades (student_id, test, grade)
   VALUES (1, "Chemistry", 85);
```

```
INSERT INTO student_grades (student_id, test, grade)
   VALUES (2, "Chemistry", 95);
SELECT * FROM student_grades;

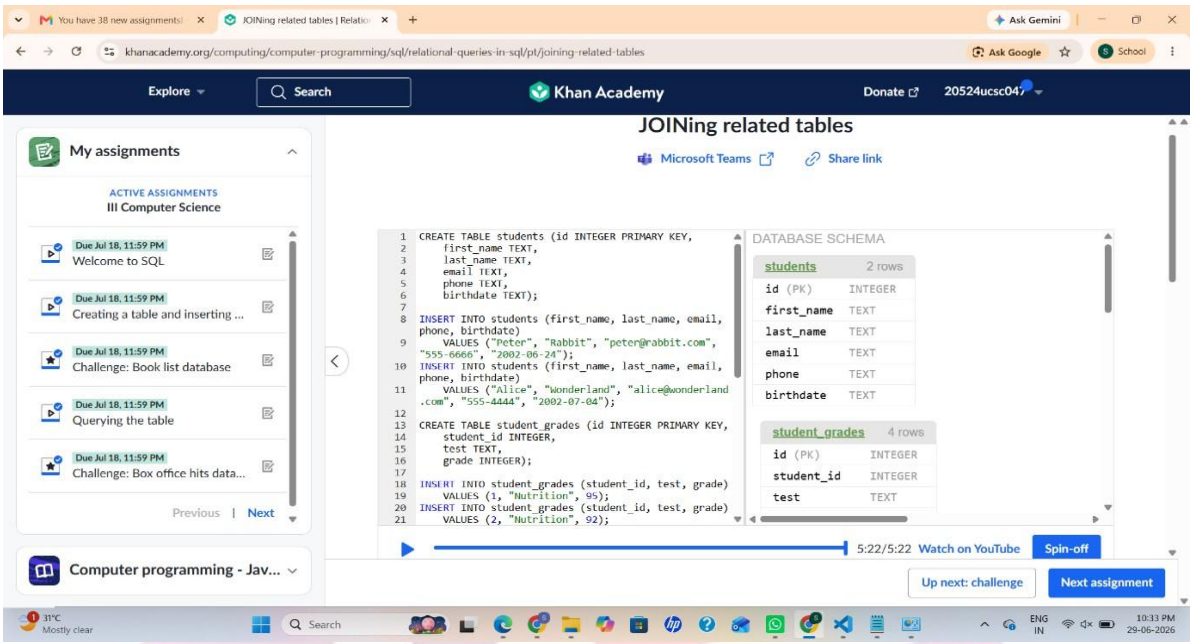
/* cross join */
SELECT * FROM student_grades, students;

/* implicit inner join */
SELECT * FROM student_grades, students WHERE
   student_grades.student_id = students.id;

/* explicit inner join - JOIN */
SELECT students.first_name, students.last_name, students.email, student_grades.test,
student_grades.grade FROM students
   JOIN student_grades
   ON students.id = student_grades.student_id WHERE
   grade > 90;
```

AUTONOMOUS

OUTPUT



TRUTH-TOIL-TRIUMPH
AUTONOMOUS

18.BOBBY'S HOBBIES

PROGRAM

```
CREATE TABLE persons (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT, age
    INTEGER
);


INSERT INTO persons (name, age) VALUES ("Bobby McBobbyFace", 12);
INSERT INTO persons (name, age) VALUES ("Lucy BoBucie", 25);
INSERT INTO persons (name, age) VALUES ("Banana FoFanna", 14);
INSERT INTO persons (name, age) VALUES ("Shish Kabob", 20);
INSERT INTO persons (name, age) VALUES ("Fluffy Sparkles", 8);


CREATE TABLE hobbies (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    person_id INTEGER, name
    TEXT
);


INSERT INTO hobbies (person_id, name) VALUES (1, "drawing");
INSERT INTO hobbies (person_id, name) VALUES (1, "coding");
INSERT INTO hobbies (person_id, name) VALUES (2, "dancing");
INSERT INTO hobbies (person_id, name) VALUES (2, "coding");
INSERT INTO hobbies (person_id, name) VALUES (3, "skating");
INSERT INTO hobbies (person_id, name) VALUES (3, "rowing");
```

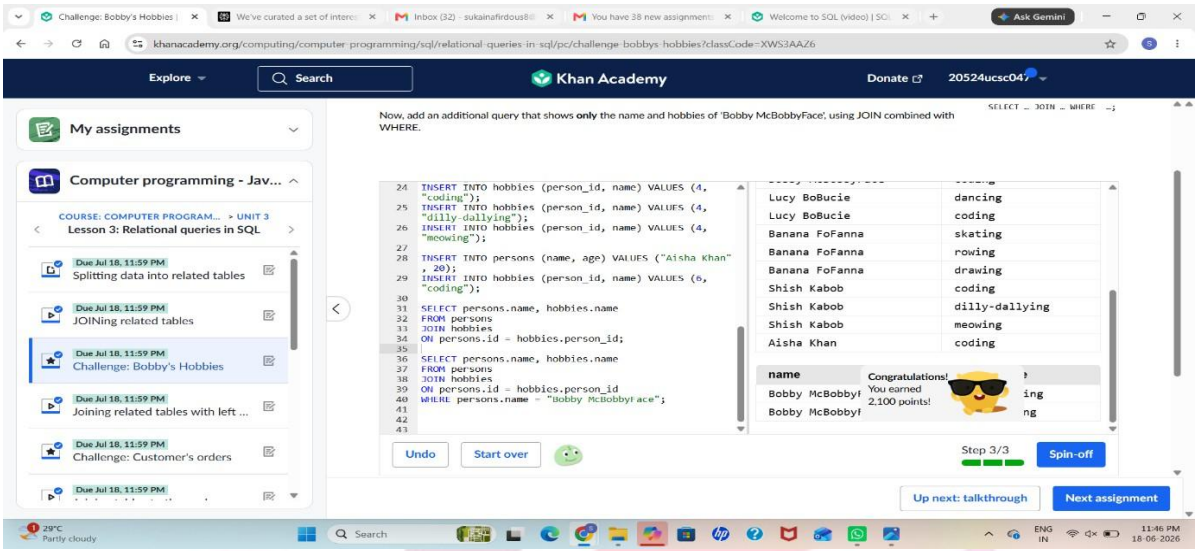
```
INSERT INTO hobbies (person_id, name) VALUES (3, "drawing");
INSERT INTO hobbies (person_id, name) VALUES (4, "coding");
INSERT INTO hobbies (person_id, name) VALUES (4, "dilly-dallying");
INSERT INTO hobbies (person_id, name) VALUES (4, "meowing");
INSERT INTO persons (name, age) VALUES ("Aisha Khan", 20);
INSERT INTO hobbies (person_id, name) VALUES (6, "coding");

SELECT persons.name, hobbies.name FROM
persons
JOIN hobbies
ON persons.id = hobbies.person_id;

SELECT persons.name, hobbies.name FROM
persons
JOIN hobbies
ON persons.id = hobbies.person_id
WHERE persons.name = "Bobby McBobbyFace";
```

TRUTH-TOIL-TRIUMPH
AUTONOMOUS

OUTPUT



TRUTH-TOIL-TRIUMPH
AUTONOMOUS

19. JOINING RELATED TABLES WITH LEFT OUTER JOINS

PROGRAM

```
CREATE TABLE students (id INTEGER PRIMARY KEY,
   first_name TEXT,
   last_name TEXT, email
   TEXT, phone TEXT,
   birthdate TEXT);

INSERT INTO students (first_name, last_name, email, phone, birthdate) VALUES
   ("Peter", "Rabbit", "peter@rabbit.com", "555-6666", "2002-06-24");
INSERT INTO students (first_name, last_name, email, phone, birthdate)
   VALUES ("Alice", "Wonderland", "alice@wonderland.com", "555-4444", "2002-07-04");

CREATE TABLE student_grades (id INTEGER PRIMARY KEY, student_id
   INTEGER,
   test TEXT,
   grade INTEGER);

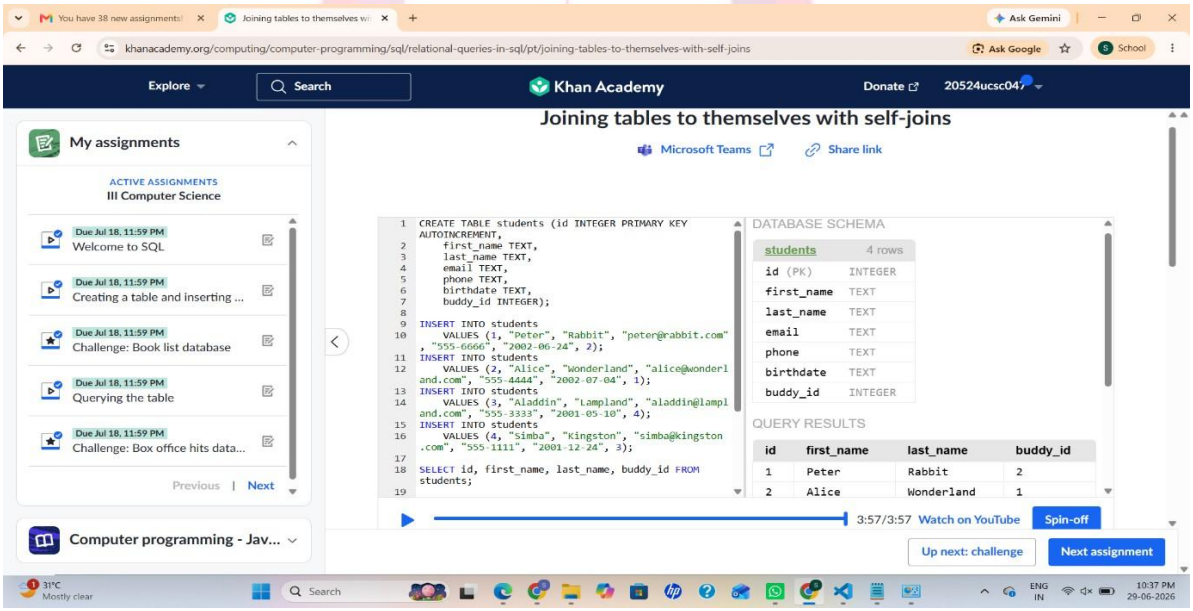
INSERT INTO student_grades (student_id, test, grade)
   VALUES (1, "Nutrition", 95);
INSERT INTO student_grades (student_id, test, grade)
   VALUES (2, "Nutrition", 92);
INSERT INTO student_grades (student_id, test, grade)
   VALUES (1, "Chemistry", 85);
INSERT INTO student_grades (student_id, test, grade)
   VALUES (2, "Chemistry", 95);
```

```
CREATE TABLE student_projects (
    id INTEGER PRIMARY KEY, student_id INTEGER,
    title TEXT);

INSERT INTO student_projects (student_id, title) VALUES
    (1,"Carrotapault");

/* outer join */
SELECT students.first_name, students.last_name, student_projects.title
FROM students
    LEFT OUTER JOIN student_projects
    ON students.id = student_projects.student_id;
```

OUTPUT



20. CUSTOMER'S ORDERS

PROGRAM

```
CREATE TABLE customers (
   id INTEGER PRIMARY KEY AUTOINCREMENT,
   name TEXT, email
   TEXT);


INSERT INTO customers (name, email) VALUES ("Doctor Who", "doctorwho@timelords.com");
INSERT INTO customers (name, email) VALUES ("Harry Potter", "harry@potter.com");
INSERT INTO customers (name, email) VALUES ("Captain Awesome",
"captain@awesome.com");

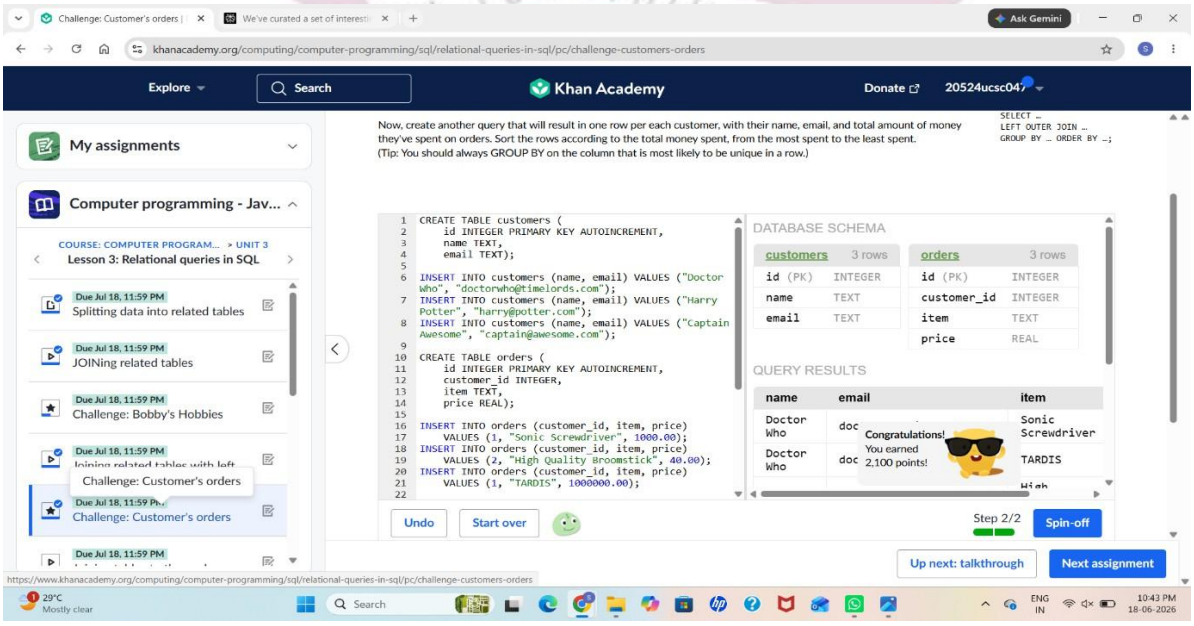

CREATE TABLE orders (
   id INTEGER PRIMARY KEY AUTOINCREMENT,
   customer_id INTEGER, item
   TEXT,
   price REAL);


    INSERT INTO orders (customer_id, item, price)
 VALUES (1, "Sonic Screwdriver", 1000.00); INSERT
         INTO orders (customer_id, item, price)
   VALUES (2, "High Quality Broomstick", 40.00); INSERT
INTO orders (customer_id, item, price)
   VALUES (1, "TARDIS", 1000000.00);
```

```
SELECT customers.name, customers.email, orders.item, orders.price
 FROM customers
LEFT OUTER JOIN orders
ON customers.id = orders.customer_id;
```

```
SELECT customers.name, customers.email, SUM(orders.price) AS total_spent FROM
customers
LEFT OUTER JOIN orders
ON customers.id = orders.customer_id
GROUP BY customers.id, customers.name, customers.email ORDER
BY total_spent DESC;
```

OUTPUT



21. SEQUELS IN SQL

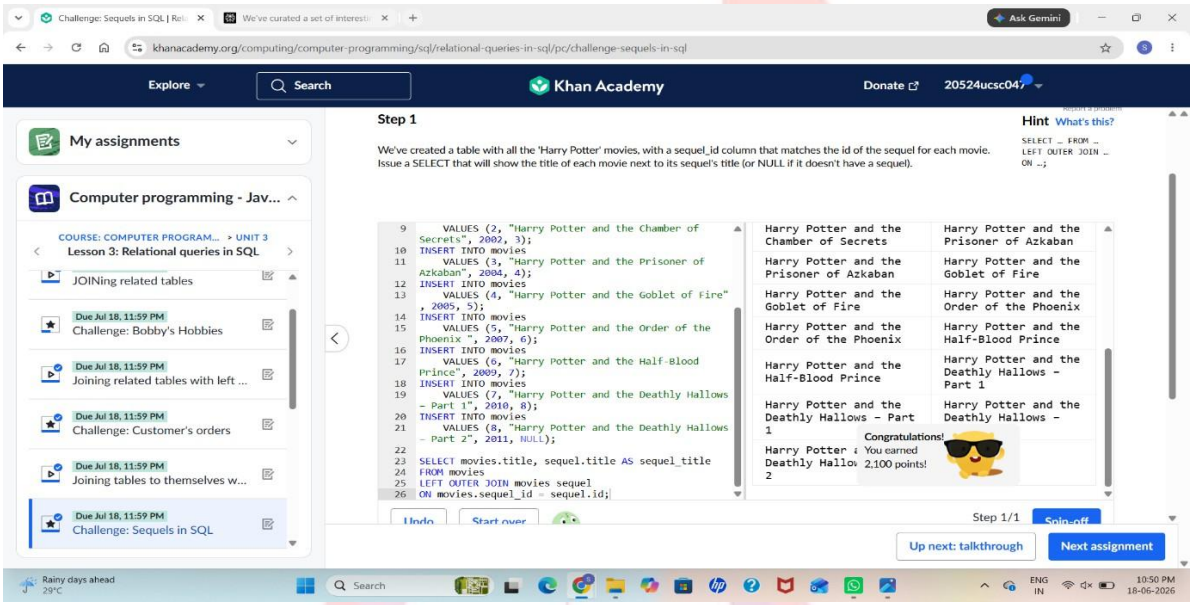
PROGRAM

```
CREATE TABLE movies (id INTEGER PRIMARY KEY AUTOINCREMENT,
   title TEXT,
   released INTEGER,
   sequel_id INTEGER);


INSERT INTO movies
   VALUES (1, "Harry Potter and the Philosopher's Stone", 2001, 2);
INSERT INTO movies
   VALUES (2, "Harry Potter and the Chamber of Secrets", 2002, 3);
INSERT INTO movies
   VALUES (3, "Harry Potter and the Prisoner of Azkaban", 2004, 4);
INSERT INTO movies
   VALUES (4, "Harry Potter and the Goblet of Fire", 2005, 5);
INSERT INTO movies
   VALUES (5, "Harry Potter and the Order of the Phoenix ", 2007, 6); INSERT
INTO movies
   VALUES (6, "Harry Potter and the Half-Blood Prince", 2009, 7); INSERT
INTO movies
   VALUES (7, "Harry Potter and the Deathly Hallows – Part 1", 2010, 8); INSERT INTO
movies
   VALUES (8, "Harry Potter and the Deathly Hallows – Part 2", 2011, NULL);
```

```
SELECT movies.title, sequel.title AS sequel_title
FROM movies
LEFT OUTER JOIN movies sequel ON
movies.sequel_id = sequel.id;
```

OUTPUT



22. COMBINING MULTIPLE JOINS

PROGRAM

```
CREATE TABLE students (id INTEGER PRIMARY KEY,
    first_name TEXT,
    last_name TEXT, email
    TEXT, phone TEXT,
    birthdate TEXT);

INSERT INTO students (first_name, last_name, email, phone, birthdate) VALUES
    ("Peter", "Rabbit", "peter@rabbit.com", "555-6666", "2002-06-24");
INSERT INTO students (first_name, last_name, email, phone, birthdate)
    VALUES ("Alice", "Wonderland", "alice@wonderland.com", "555-4444", "2002-07-04"); INSERT
INTO students (first_name, last_name, email, phone, birthdate)
    VALUES ("Aladdin", "Lampland", "aladdin@lampland.com", "555-3333", "2001-05-10"); INSERT
INTO students (first_name, last_name, email, phone, birthdate)
    VALUES ("Simba", "Kingston", "simba@kingston.com", "555-1111", "2001-12-24");

CREATE TABLE student_projects (id INTEGER PRIMARY KEY,
    student_id INTEGER,
    title TEXT);

INSERT INTO student_projects (student_id, title) VALUES (1,
    "Carrotapault");
INSERT INTO student_projects (student_id, title) VALUES (2,
    "Mad Hattery");
```

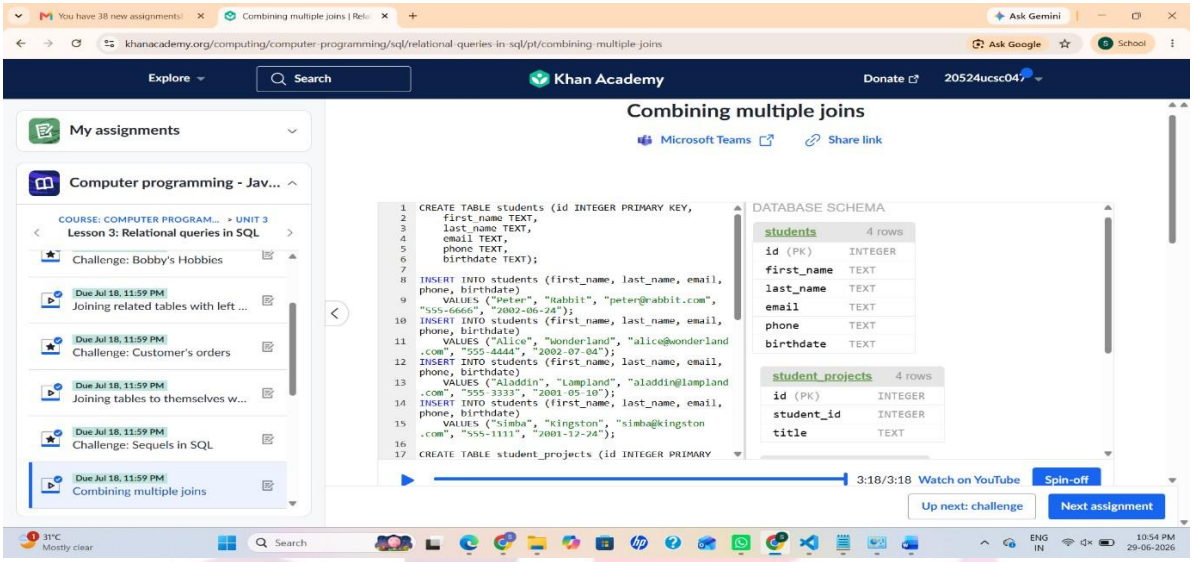
```
INSERT INTO student_projects (student_id, title) VALUES (3,
"Carpet Physics");
INSERT INTO student_projects (student_id, title) VALUES (4,
    "Hyena Habitats");

CREATE TABLE project_pairs (id INTEGER PRIMARY KEY,
    project1_id INTEGER,
    project2_id INTEGER);

INSERT INTO project_pairs (project1_id, project2_id) VALUES(1, 2);
INSERT INTO project_pairs (project1_id, project2_id) VALUES(3, 4);

SELECT a.title, b.title FROM project_pairs JOIN
    student_projects a
    ON project_pairs.project1_id = a.id JOIN
    student_projects b
    ON project_pairs.project2_id = b.id;
```

OUTPUT



TRUTH-TOIL-TRIUMPH
AUTONOMOUS

23.FRIENDBOOK

PROGRAM

```
CREATE TABLE persons (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    fullname TEXT, age
    INTEGER
);

INSERT INTO persons (fullname, age) VALUES ("Bobby McBobbyFace", 12);
INSERT INTO persons (fullname, age) VALUES ("Lucy BoBucie", 25); INSERT
INTO persons (fullname, age) VALUES ("Banana FoFanna", 14); INSERT INTO
persons (fullname, age) VALUES ("Shish Kabob", 20); INSERT INTO persons
(fullname, age) VALUES ("Fluffy Sparkles", 8);

CREATE TABLE hobbies (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    person_id INTEGER, name
    TEXT
);

INSERT INTO hobbies (person_id, name) VALUES (1, "drawing");
INSERT INTO hobbies (person_id, name) VALUES (1, "coding");
INSERT INTO hobbies (person_id, name) VALUES (2, "dancing");
INSERT INTO hobbies (person_id, name) VALUES (2, "coding");
INSERT INTO hobbies (person_id, name) VALUES (3, "skating");
```

```
INSERT INTO hobbies (person_id, name) VALUES (3, "rowing");
INSERT INTO hobbies (person_id, name) VALUES (3, "drawing");
INSERT INTO hobbies (person_id, name) VALUES (4, "coding");
INSERT INTO hobbies (person_id, name) VALUES (4, "dilly-dallying"); INSERT
INTO hobbies (person_id, name) VALUES (4, "meowing");

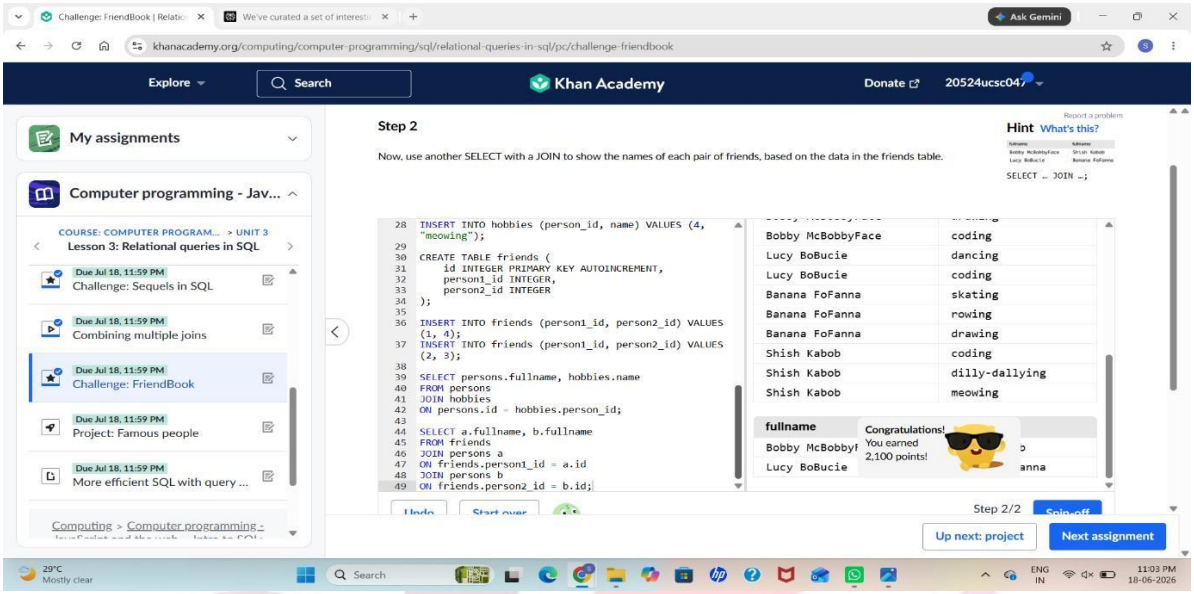
CREATE TABLE friends (
   id INTEGER PRIMARY KEY AUTOINCREMENT,
   person1_id INTEGER,
   person2_id INTEGER
);

INSERT INTO friends (person1_id, person2_id) VALUES (1, 4); INSERT
INTO friends (person1_id, person2_id) VALUES (2, 3);

SELECT persons.fullname, hobbies.name FROM
persons
JOIN hobbies
ON persons.id = hobbies.person_id;

SELECT a.fullname, b.fullname FROM
friends
JOIN persons a
ON friends.person1_id = a.id JOIN
persons b
ON friends.person2_id = b.id;
```

OUTPUT



TRUTH-TOIL-TRIUMPH
AUTONOMOUS

24. PROJECT: FAMOUS PEOPLE

PROGRAM

```
CREATE TABLE people (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT, age
    INTEGER,
    profession TEXT
);


INSERT INTO people (name, age, profession) VALUES ("A. P. J. Abdul Kalam", 83, "Scientist");
INSERT INTO people (name, age, profession) VALUES ("M. S. Subbulakshmi", 88, "Singer");
INSERT INTO people (name, age, profession) VALUES ("C. V. Raman", 82, "Scientist");
INSERT INTO people (name, age, profession) VALUES ("Lata Mangeshkar", 92, "Singer");
INSERT INTO people (name, age, profession) VALUES ("Dr. B. R. Ambedkar", 65, "Lawyer");


CREATE TABLE works (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    person_id INTEGER,
    work_title TEXT, work_type
    TEXT
);


INSERT INTO works (person_id, work_title, work_type) VALUES (1, "Missile Man of India",
"Book");
INSERT INTO works (person_id, work_title, work_type) VALUES (2, "Vande Mataram", "Song");
```

INSERT INTO works (person_id, work_title, work_type) VALUES (3, "Raman Effect Research", "Discovery");

INSERT INTO works (person_id, work_title, work_type) VALUES (4, "Aye Mere Watan Ke Logon", "Song");

INSERT INTO works (person_id, work_title, work_type) VALUES (5, "Annihilation of Caste", "Book");

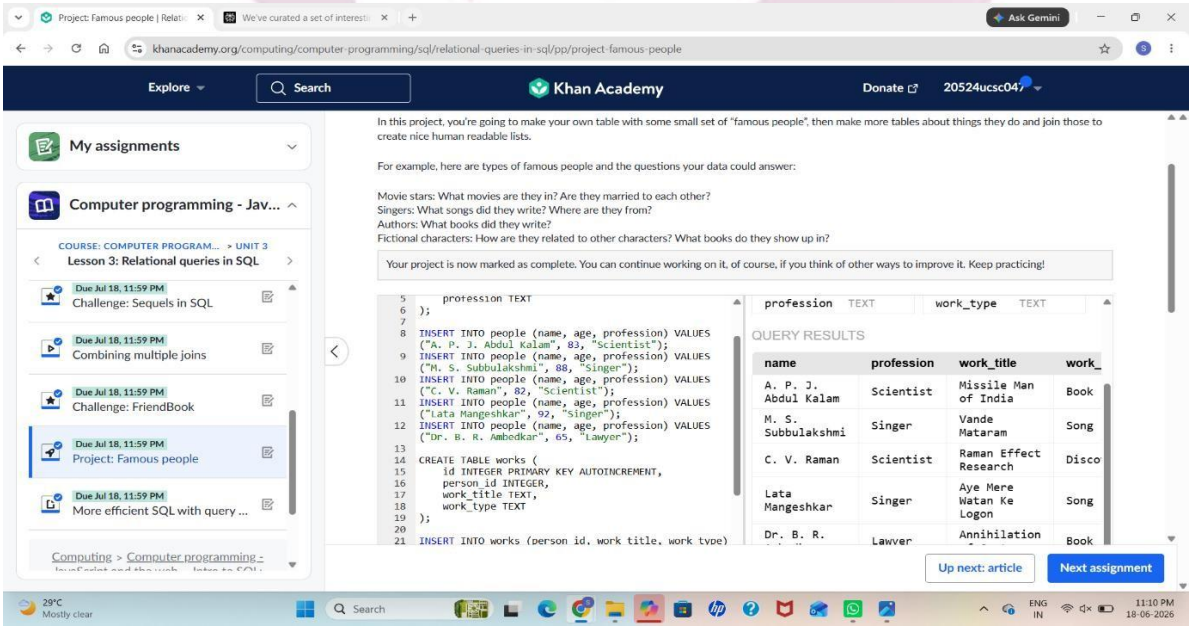
SELECT people.name, people.profession, works.work_title, works.work_type FROM

people

JOIN works

ON people.id = works.person_id;

OUTPUT



25. CHANGING ROWS WITH UPDATE AND DELETE

PROGRAM

```
CREATE TABLE users (
   id INTEGER PRIMARY KEY,
   name TEXT);

CREATE TABLE diary_logs (
   id INTEGER PRIMARY KEY,
   user_id INTEGER, date
   TEXT, content TEXT
   );

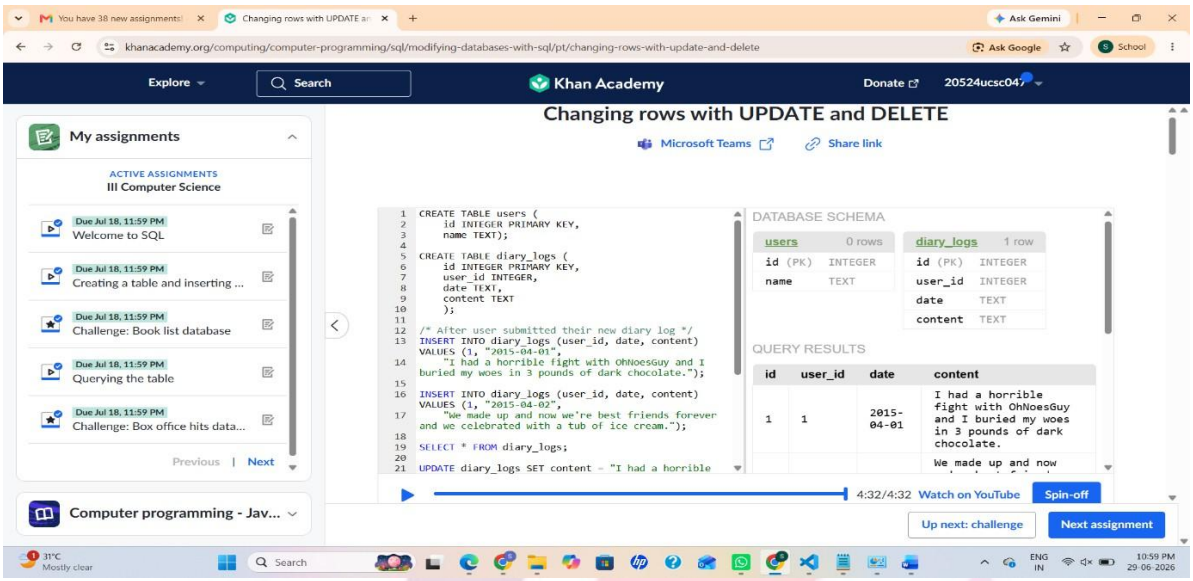
/* After user submitted their new diary log */
INSERT INTO diary_logs (user_id, date, content) VALUES (1, "2015-04-01",
   "I had a horrible fight with OhNoesGuy and I buried my woes in 3 pounds of dark chocolate.");

INSERT INTO diary_logs (user_id, date, content) VALUES (1, "2015-04-02",
   "We made up and now we're best friends forever and we celebrated with a tub of ice cream.");

SELECT * FROM diary_logs;
UPDATE diary_logs SET content = "I had a horrible fight with OhNoesGuy" WHERE id = 1;
SELECT * FROM diary_logs;
DELETE FROM diary_logs WHERE id = 1;

SELECT * FROM diary_logs;
```

OUTPUT


Changing rows with UPDATE and DELETE
Microsoft Teams
Share link
My assignments
ACTIVE ASSIGNMENTS
III Computer Science
Welcome to SQL
Creating a table and inserting ...
Challenge: Book list database
Querying the table
Challenge: Box office hits data...
Previous | Next
Computer programming - Jav...
DATABASE SCHEMA
users 0 rows
id (PK) INTEGER
name TEXT
diary_logs 1 row
id (PK) INTEGER
user_id INTEGER
date TEXT
content TEXT
QUERY RESULTS
id user_id date content
1 1 2015-04-01 I had a horrible fight with OhNoesGuy and I buried my woes in 3 pounds of dark chocolate.
4:32/4:32 Watch on YouTube
Spin-off
Up next: challenge
Next assignment

26. DYNAMIC DOCUMENTS

PROGRAM

```
CREATE table documents (
   id INTEGER PRIMARY KEY AUTOINCREMENT,
   title TEXT, content
   TEXT, author TEXT);


INSERT INTO documents (author, title, content)
   VALUES ("Puff T.M. Dragon", "Fancy Stuff", "Ceiling wax, dragon wings, etc."); INSERT
INTO documents (author, title, content)
   VALUES ("Puff T.M. Dragon", "Living Things", "They're located in the left ear, you know.");
INSERT INTO documents (author, title, content)
   VALUES ("Jackie Paper", "Pirate Recipes", "Cherry pie, apple pie, blueberry pie."); INSERT
INTO documents (author, title, content)
   VALUES ("Jackie Paper", "Boat Supplies", "Rudder - guitar. Main mast - bed post."); INSERT
INTO documents (author, title, content)
   VALUES ("Jackie Paper", "Things I'm Afraid Of", "Talking to my parents, the sea, giant pirates,
heights.");


SELECT * FROM documents;

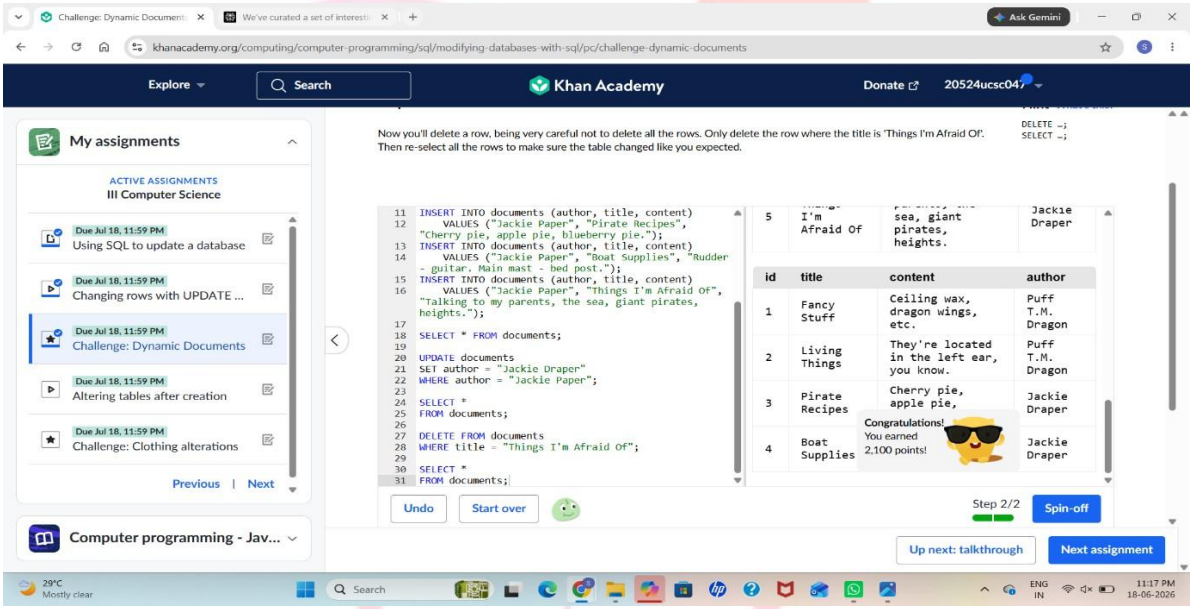

UPDATE documents
SET author = "Jackie Draper" WHERE
author = "Jackie Paper";


SELECT *
FROM documents;
```

```
DELETE FROM documents
WHERE title = "Things I'm Afraid Of";

SELECT *
FROM documents;
```

OUTPUT



27. ALTERING TABLES AFTER CREATION

PROGRAM

```
CREATE TABLE users (
   id INTEGER PRIMARY KEY,
   name TEXT);


CREATE TABLE diary_logs (
   id INTEGER PRIMARY KEY,
   user_id INTEGER, date
   TEXT, content TEXT
   );

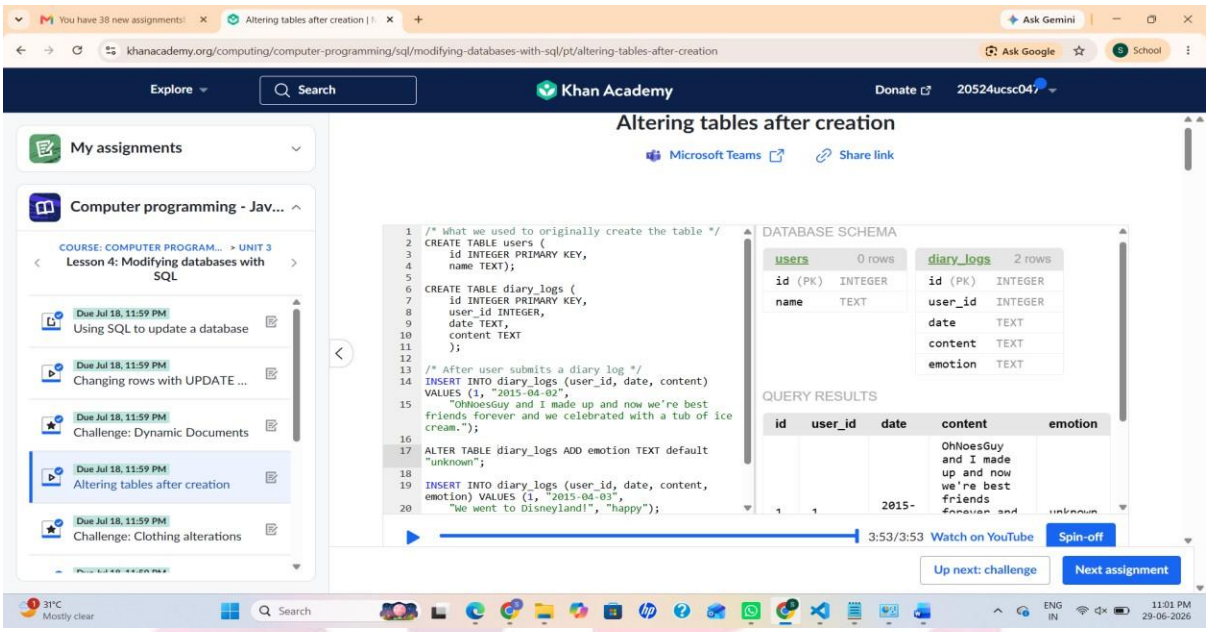

INSERT INTO diary_logs (user_id, date, content) VALUES (1, "2015-04-02", "OhNoesGuy and I
   made up and now we're best friends forever and we celebrated with a
tub of ice cream.");


ALTER TABLE diary_logs ADD emotion TEXT default "unknown";


INSERT INTO diary_logs (user_id, date, content, emotion) VALUES (1, "2015-04-03", "We went
   to Disneyland!", "happy");


SELECT * FROM diary_logs;
```

OUTPUT



TRUTH-TOIL-TRIUMPH
AUTONOMOUS

28. CLOTHING ALTERATIONS

PROGRAM

```
CREATE TABLE clothes (
   id INTEGER PRIMARY KEY AUTOINCREMENT,
   type TEXT, design
   TEXT
);

INSERT INTO clothes (type, design) VALUES
("dress", "pink polka dots");

INSERT INTO clothes (type, design)
VALUES ("pants", "rainbow tie-dye");

INSERT INTO clothes (type, design) VALUES
("blazer", "black sequin");

ALTER TABLE clothes ADD
price INTEGER;

SELECT *
FROM clothes;

UPDATE clothes SET
price = 10 WHERE id =
1;
```

```
UPDATE clothes
SET price = 20
WHERE id = 2;
```

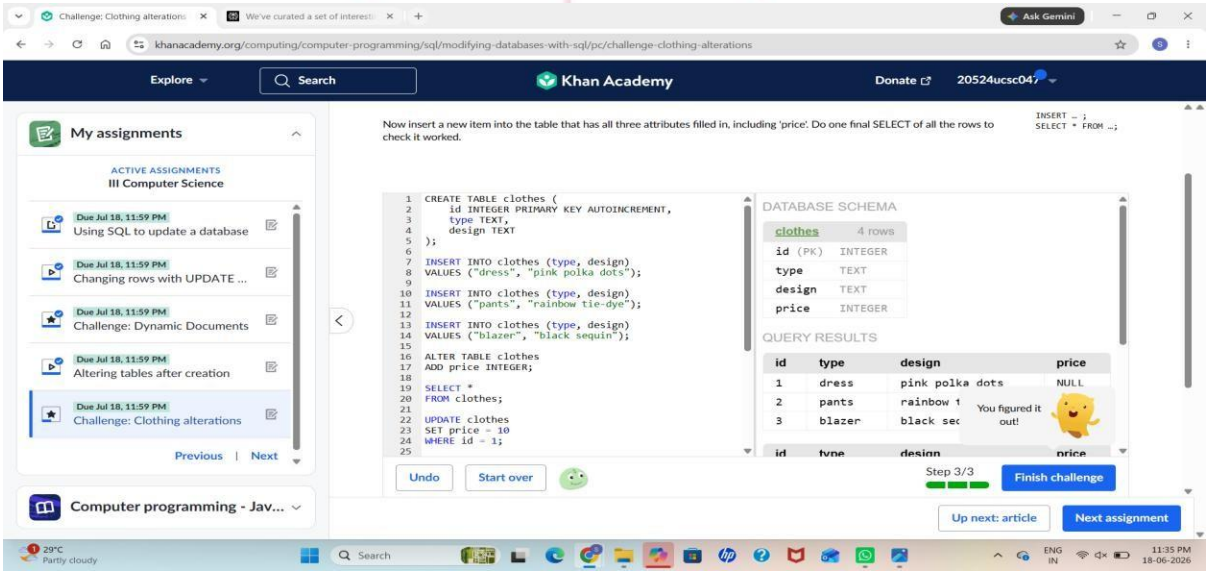
```
UPDATE clothes SET
price = 30 WHERE id =
3;
```

```
SELECT *
FROM clothes;
```

```
INSERT INTO clothes (type, design, price)
VALUES ("Hat", "plain", 40);
```

```
SELECT *
FROM clothes;
```

OUTPUT



29. PROJECT: APP IMPERSONATOR

PROGRAM

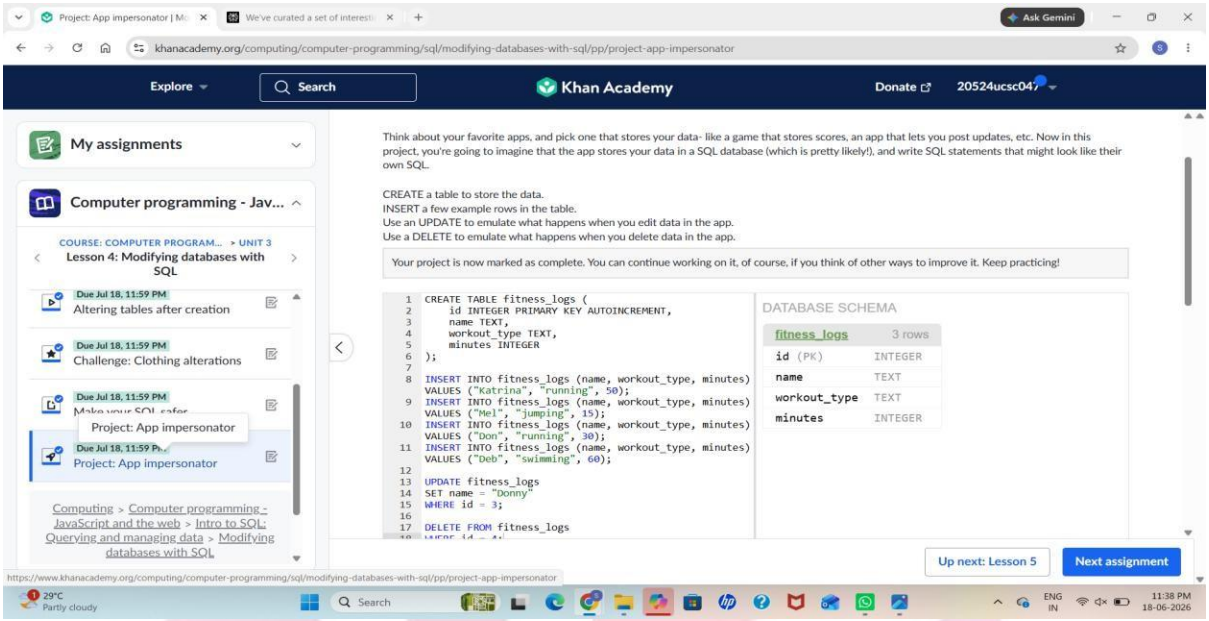
```
CREATE TABLE fitness_logs (
   id INTEGER PRIMARY KEY AUTOINCREMENT,
   name TEXT, workout_type
   TEXT, minutes INTEGER
);

INSERT INTO fitness_logs (name, workout_type, minutes) VALUES ("Katrina", "running", 50);
INSERT INTO fitness_logs (name, workout_type, minutes) VALUES ("Mel", "jumping", 15);
INSERT INTO fitness_logs (name, workout_type, minutes) VALUES ("Don", "running", 30);
INSERT INTO fitness_logs (name, workout_type, minutes) VALUES ("Deb", "swimming", 60);

UPDATE  fitness_logs  SET
name = "Donny"  WHERE id
= 3;

DELETE FROM fitness_logs WHERE id
= 4;
```

OUTPUT



TRUTH-TOIL-TRIUMPH
AUTONOMOUS

HACKER RANK

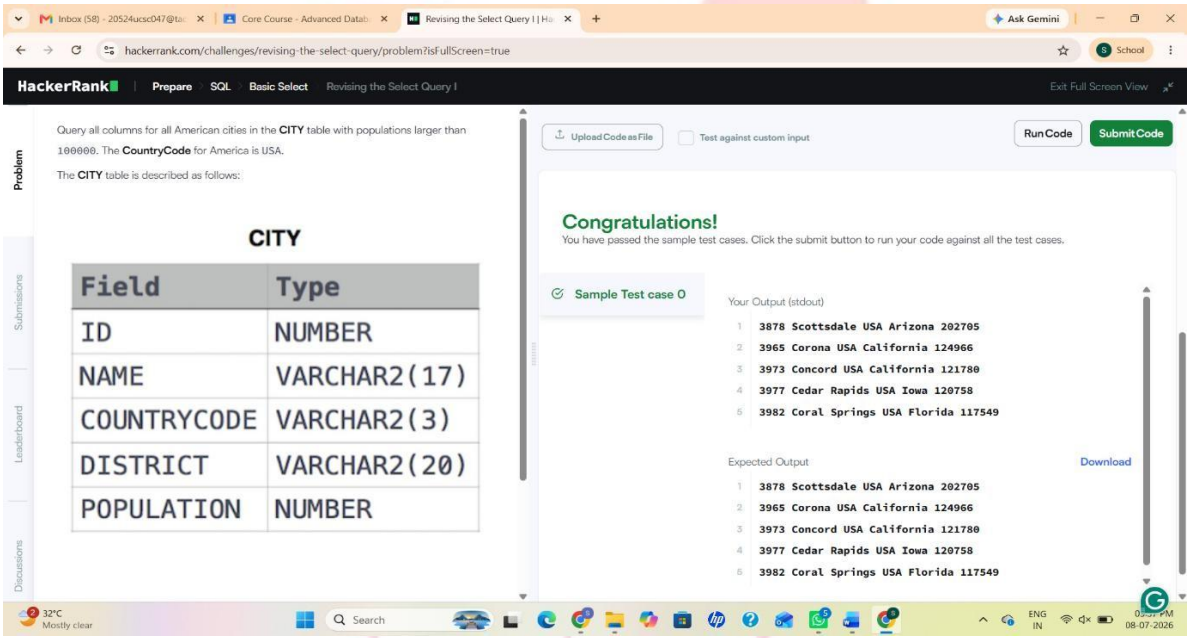
1. REVISING THE SELECT QUERY I

PROGRAM

```
SELECT * FROM CITY

WHERE COUNTRYCODE = 'USA' AND POPULATION > 100000;
```

OUTPUT

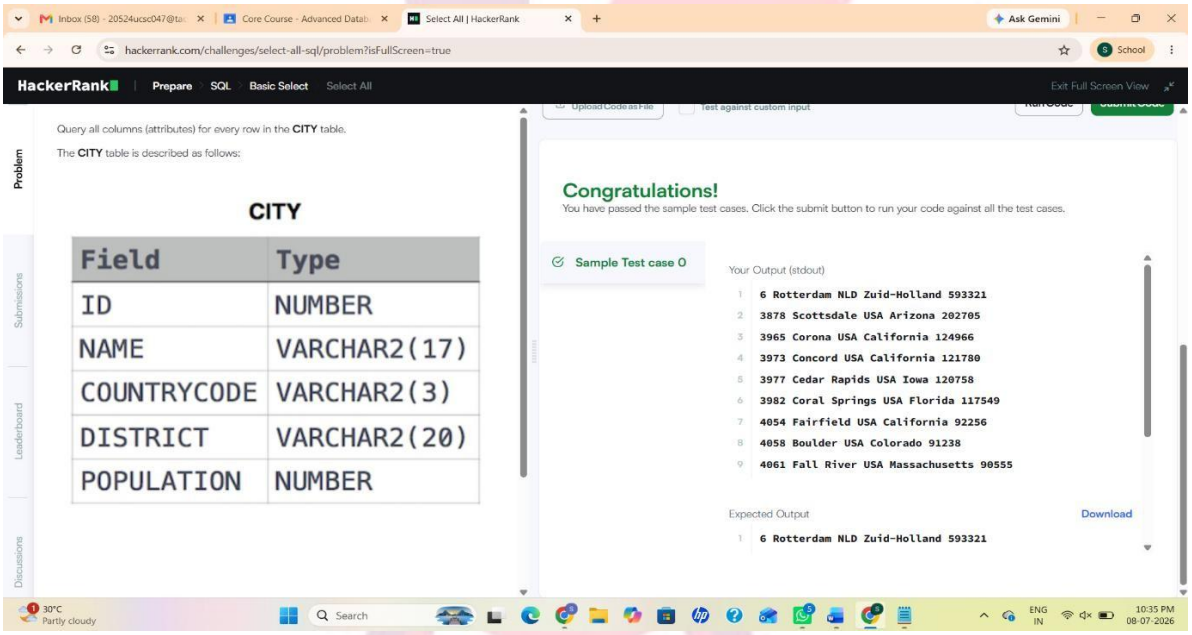


2. SELECT ALL

PROGRAM

```
SELECT t.*
FROM CITY t;
```

OUTPUT

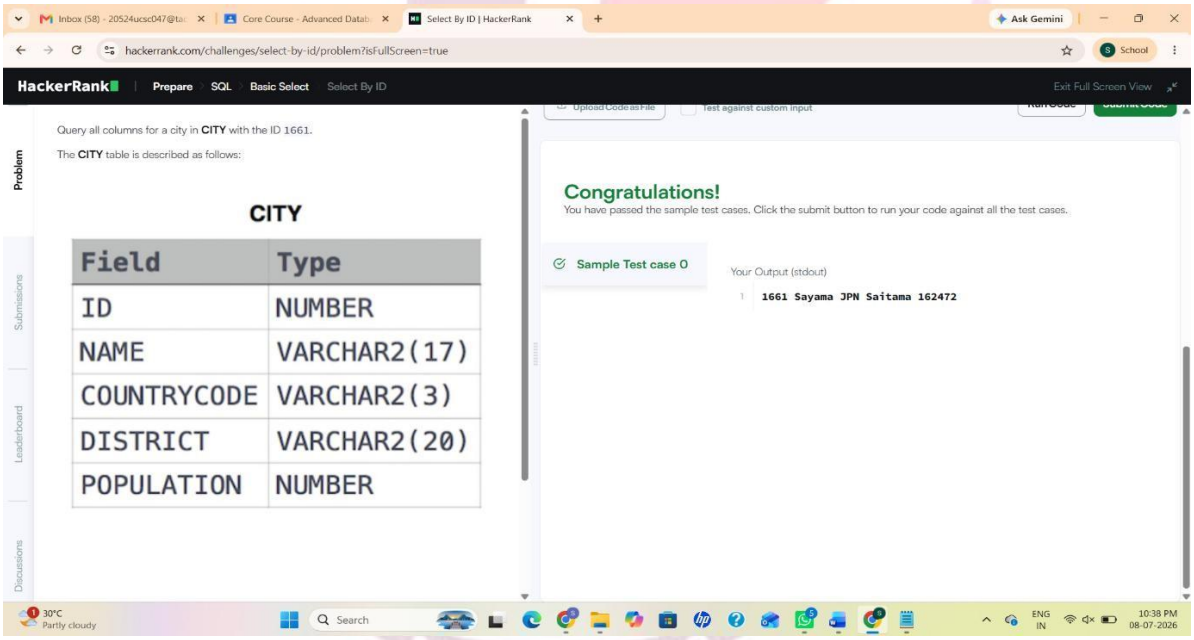


3. SELECT BY ID

PROGRAM

```
SELECT t.*
FROM    CITY t
WHERE t.ID=1661;
```

OUTPUT

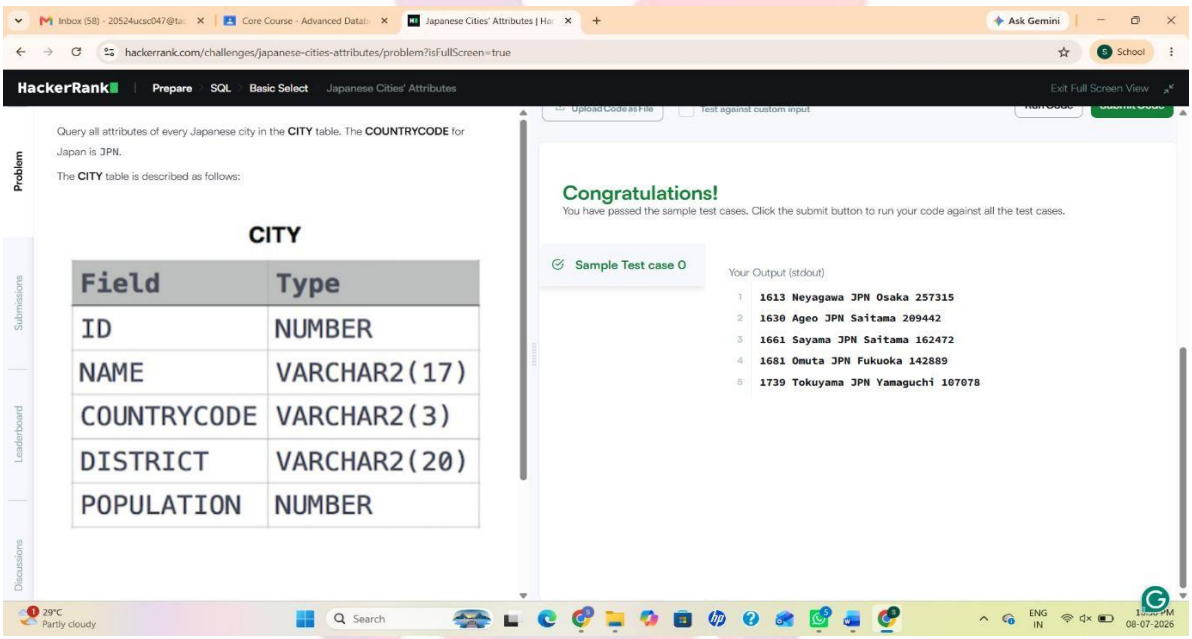


4. JAPANESE CITIES' ATTRIBUTES

PROGRAM

```
SELECT t.* FROM CITY t
WHERE t.COUNTRYCODE = 'JPN';
```

OUTPUT

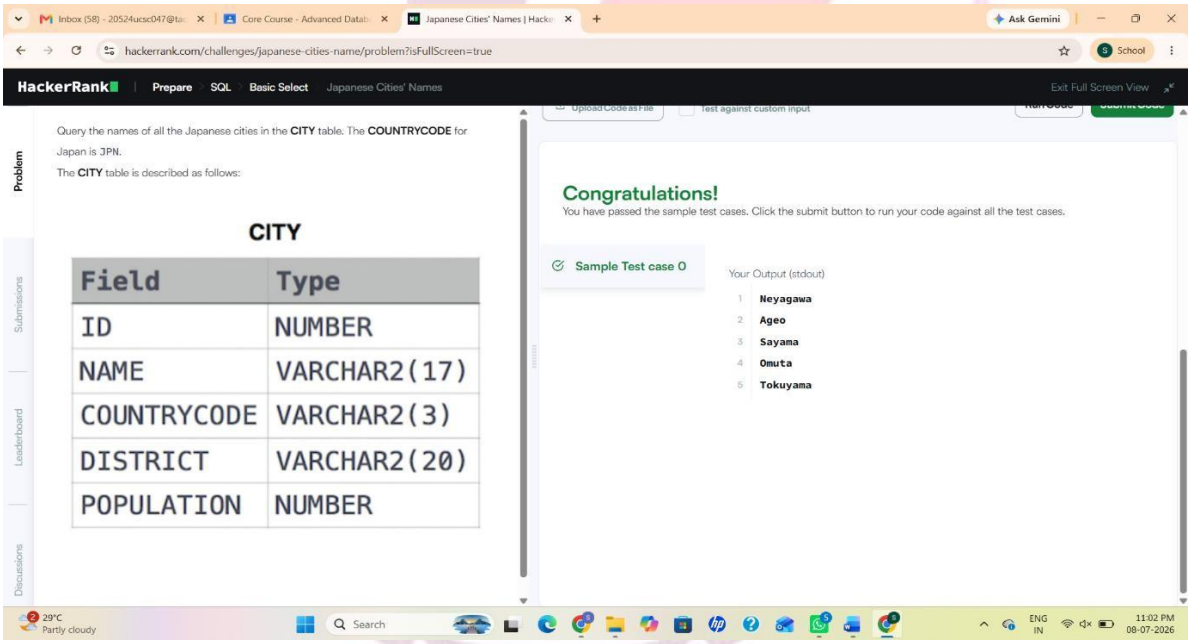


5. JAPANESE CITIES' NAMES

PROGRAM

```
SELECT t.NAME
FROM CITY t
WHERE t.COUNTRYCODE = 'JPN';
```

OUTPUT



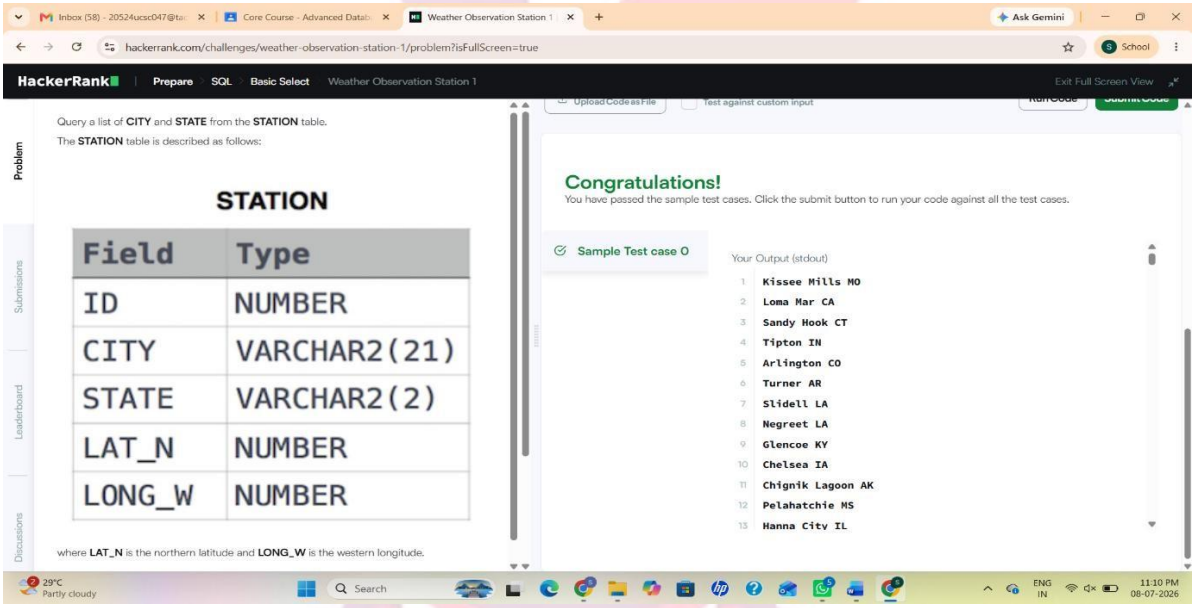
6. WEATHER OBSERVATION STATION 1

PROGRAM

```
SELECT t.CITY,t.STATE

FROM STATION t;
```

OUTPUT

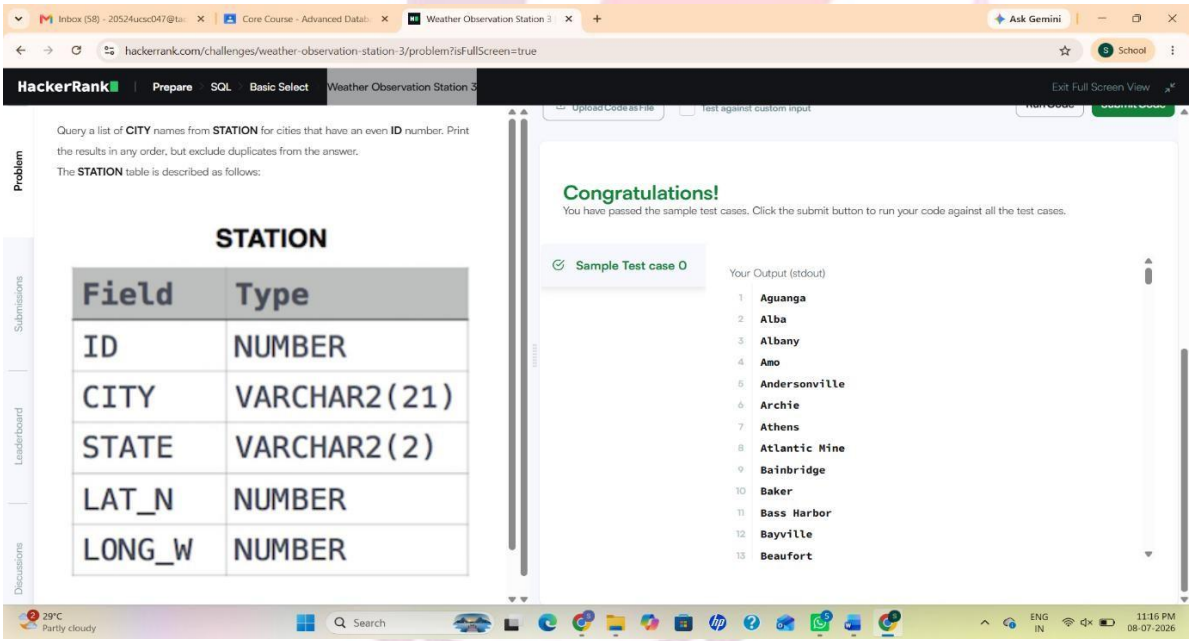


7. WEATHER OBSERVATION STATION 3

PROGRAM

```
SELECT DISTINCT t.CITY FROM
STATION t
WHERE MOD(t.ID,2) = 0;
```

OUTPUT



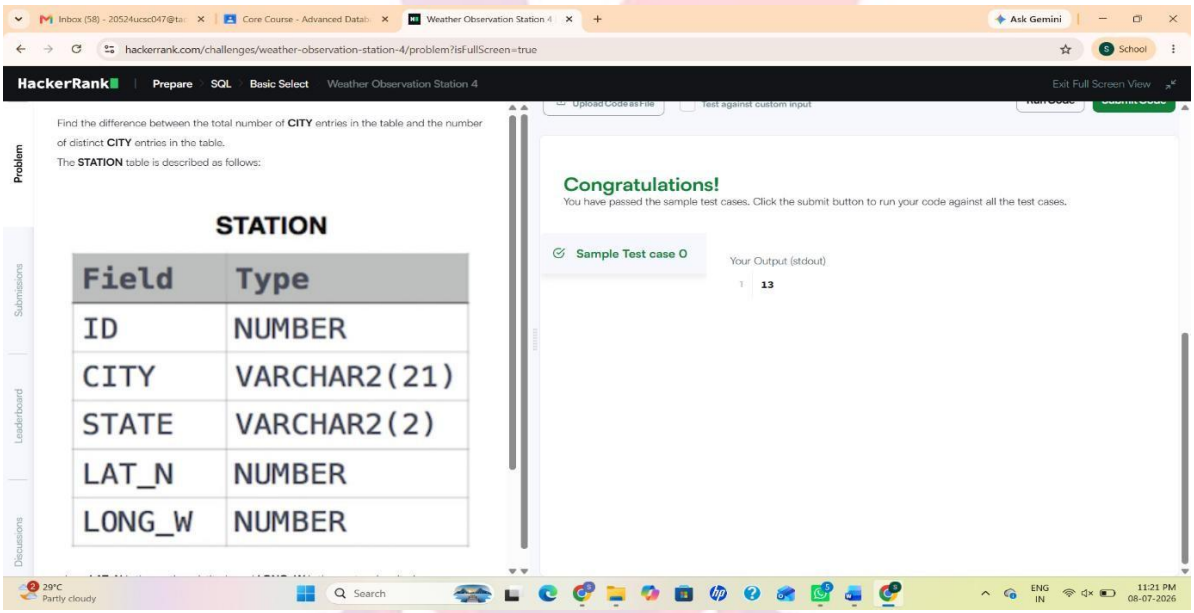
8. WEATHER OBSERVATION STATION 4

PROGRAM

```
SELECT COUNT(t.CITY) - COUNT(DISTINCT t.CITY)

FROM STATION t;
```

OUTPUT



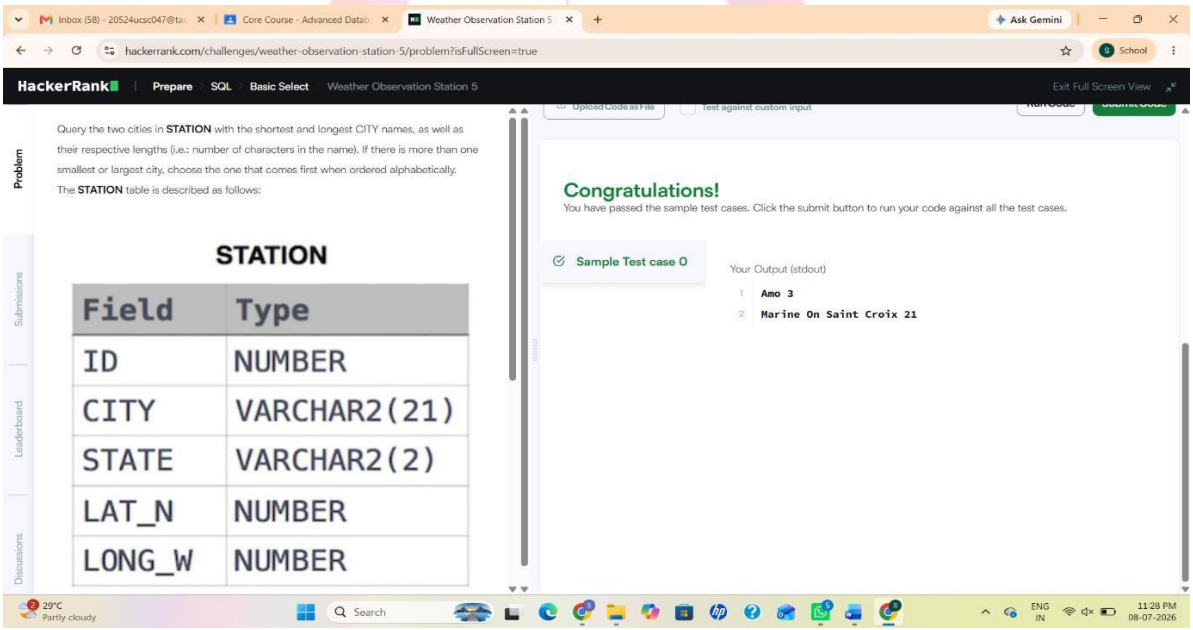
9. WEATHER OBSERVATION STATION 5

PROGRAM

```
SELECT CITY, LENGTH(CITY) FROM
STATION
ORDER BY LENGTH(CITY), CITY FETCH
FIRST 1 ROW ONLY;

SELECT CITY, LENGTH(CITY) FROM
STATION
ORDER BY LENGTH(CITY) DESC, CITY FETCH
FIRST 1 ROW ONLY;
```

OUTPUT



10. WEATHER OBSERVATION STATION 6

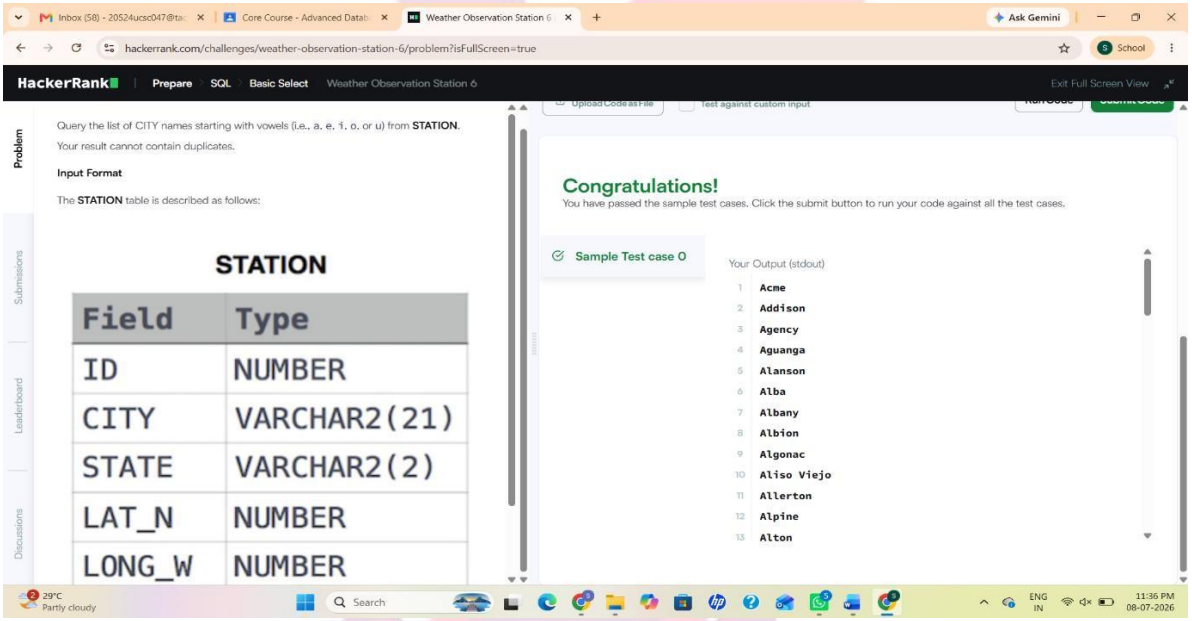
PROGRAM

```
SELECT DISTINCT CITY

FROM STATION

WHERE UPPER(SUBSTR(CITY,1,1)) IN ('A','E','I','O','U');
```

OUTPUT

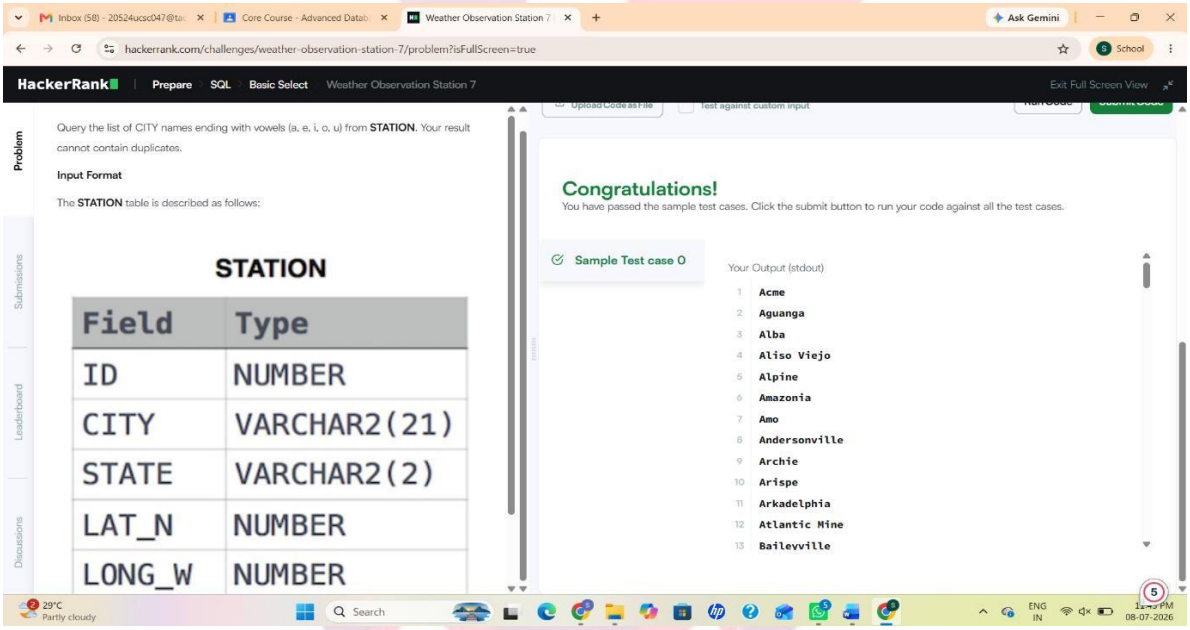


11. WEATHER OBSERVATION STATION 7

PROGRAM

```
SELECT DISTINCT CITY
FROM STATION
WHERE UPPER(SUBSTR(CITY,LENGTH(CITY),1)) IN ('A','E','I','O','U');
```

OUTPUT

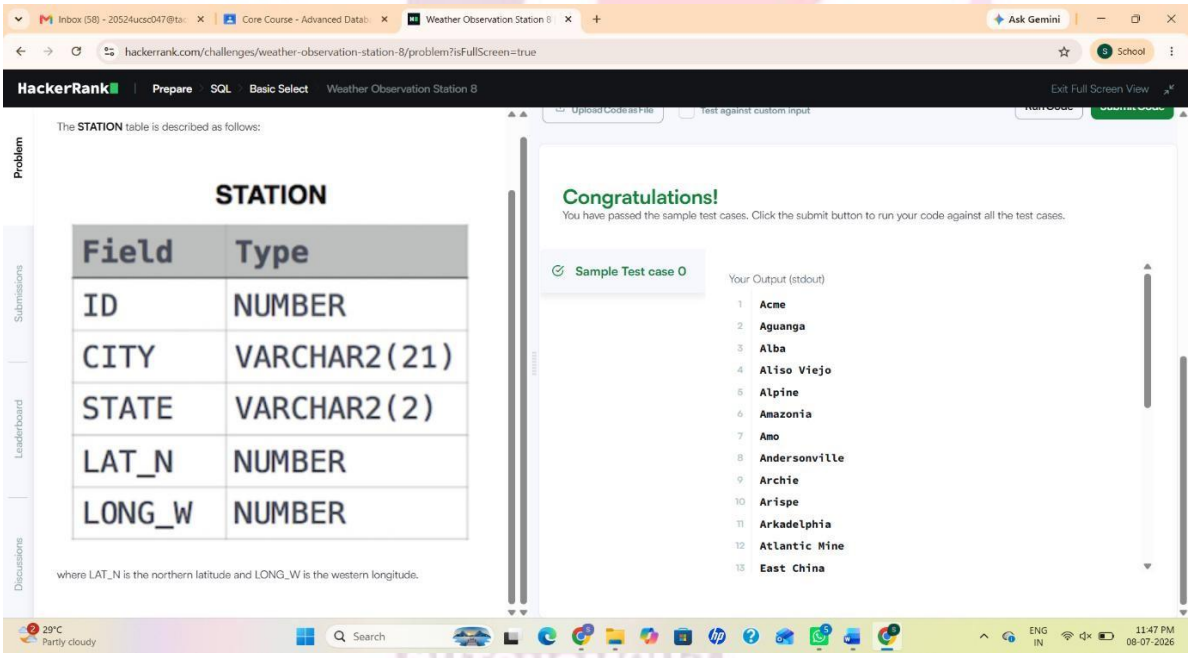


12. WEATHER OBSERVATION STATION 8

PROGRAM

```
SELECT DISTINCT CITY
FROM STATION
WHERE UPPER(SUBSTR(CITY,1,1)) IN ('A','E','I','O','U') AND
UPPER(SUBSTR(CITY,LENGTH(CITY),1)) IN ('A','E','I','O','U');
```

OUTPUT

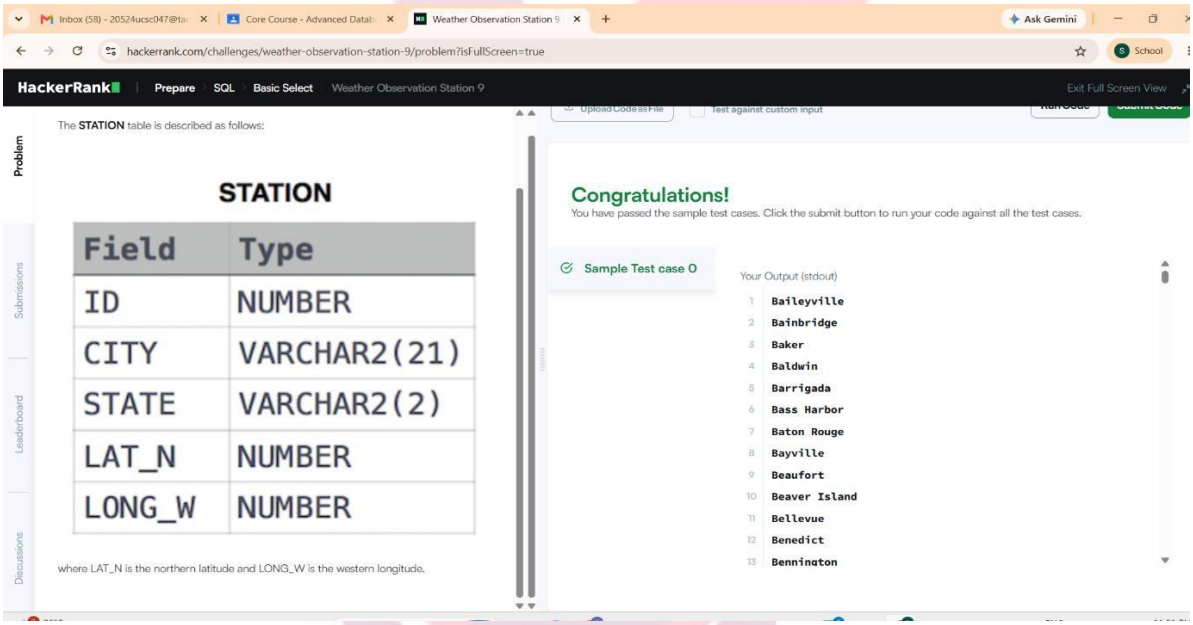


13. WEATHER OBSERVATION STATION 9

PROGRAM

```
SELECT DISTINCT CITY
FROM STATION
WHERE UPPER(SUBSTR(CITY,1,1)) NOT IN ('A','E','I','O','U');
```

OUTPUT

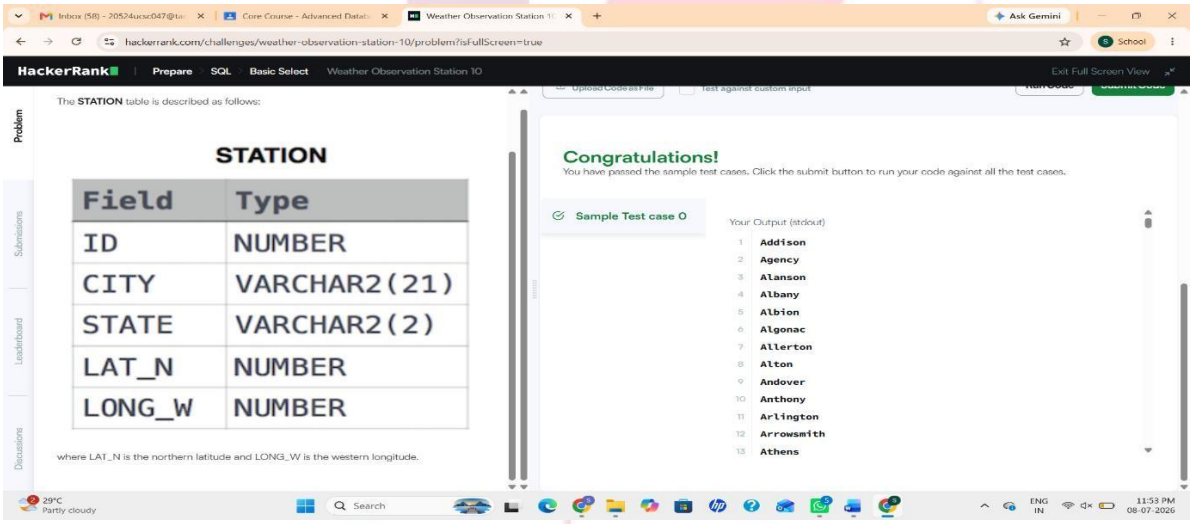


14. WEATHER OBSERVATION STATION 10

PROGRAM

```
SELECT DISTINCT CITY
 FROM STATION
 WHERE UPPER(SUBSTR(CITY,LENGTH(CITY),1)) NOT IN ('A','E','I','O','U');
```

OUTPUT

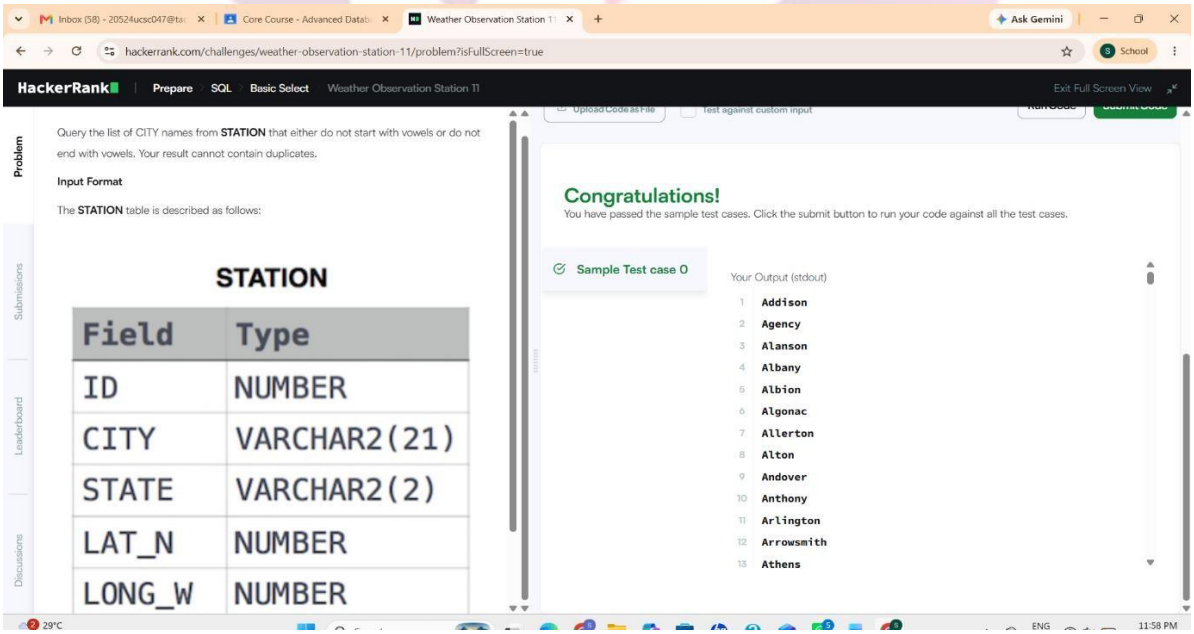


15. WEATHER OBSERVATION STATION 11

PROGRAM

```
SELECT DISTINCT CITY
FROM STATION
WHERE UPPER(SUBSTR(CITY,1,1)) NOT IN ('A','E','I','O','U') OR UPPER
(SUBSTR(CITY,LENGTH(CITY),1)) NOT IN
('A','E','I','O','U');
```

OUTPUT

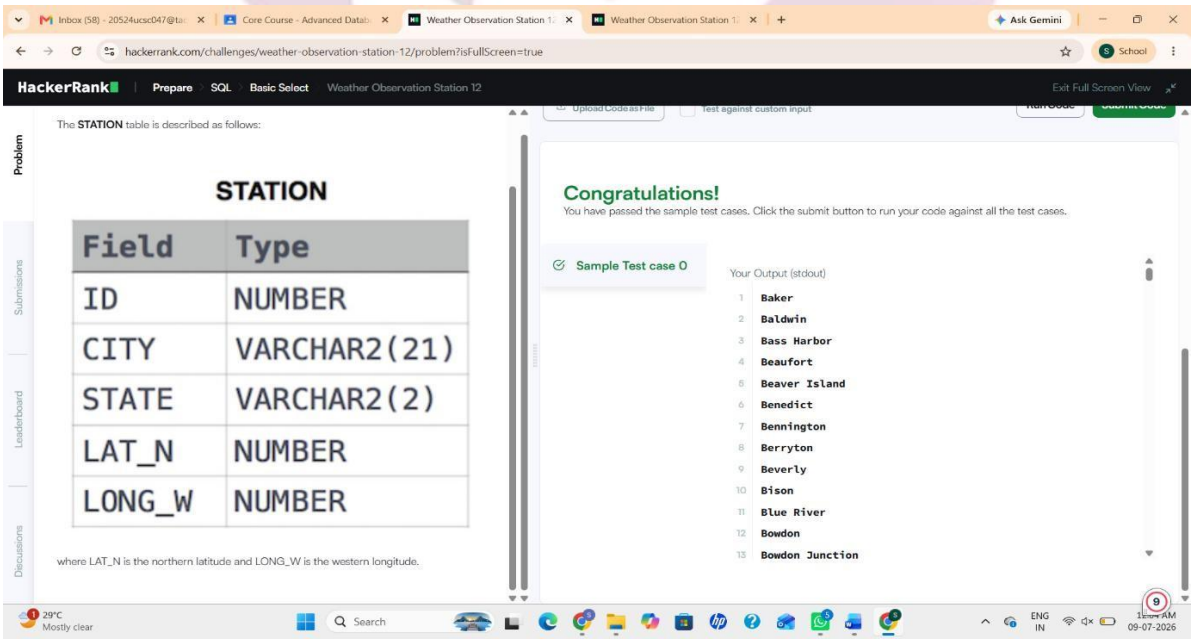


16. WEATHER OBSERVATION STATION 12

PROGRAM

```
SELECT DISTINCT CITY
FROM STATION
WHERE UPPER(SUBSTR(CITY,1,1)) NOT IN ('A','E','I','O','U') AND
UPPER(SUBSTR(CITY,LENGTH(CITY),1)) NOT IN
('A','E','I','O','U');
```

OUTPUT



17. HIGHER THAN 75 MARKS

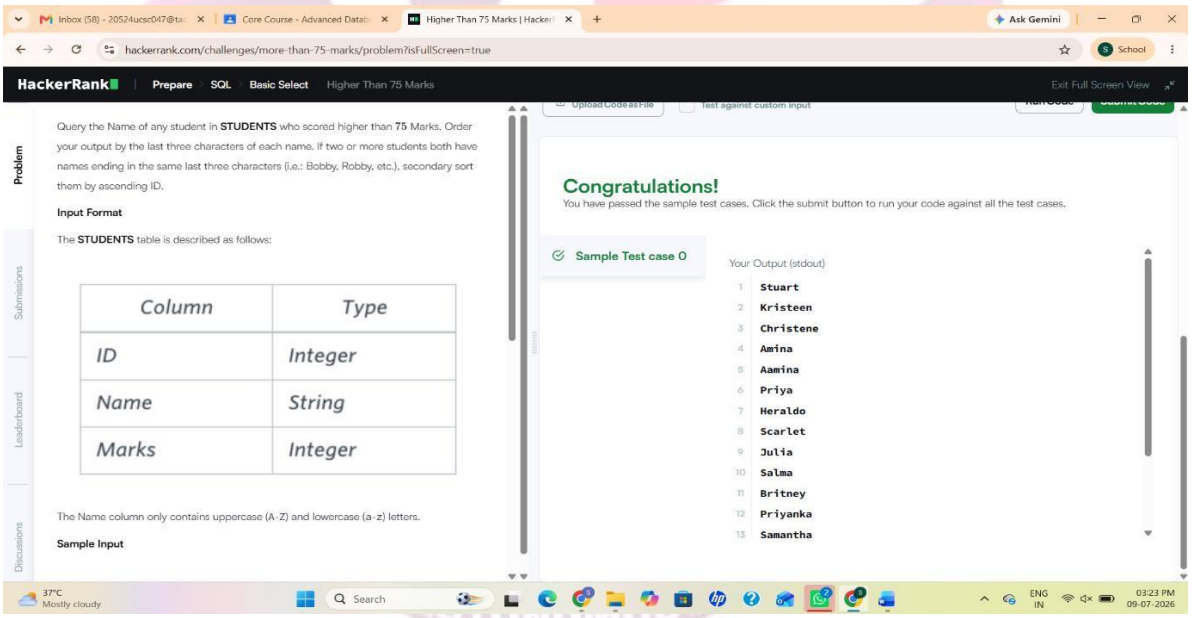
PROGRAM

```
SELECT Name

FROM STUDENTS

WHERE Marks > 75 ORDER BY SUBSTR(Name, LENGTH(Name)-2, 3), ID;
```

OUTPUT

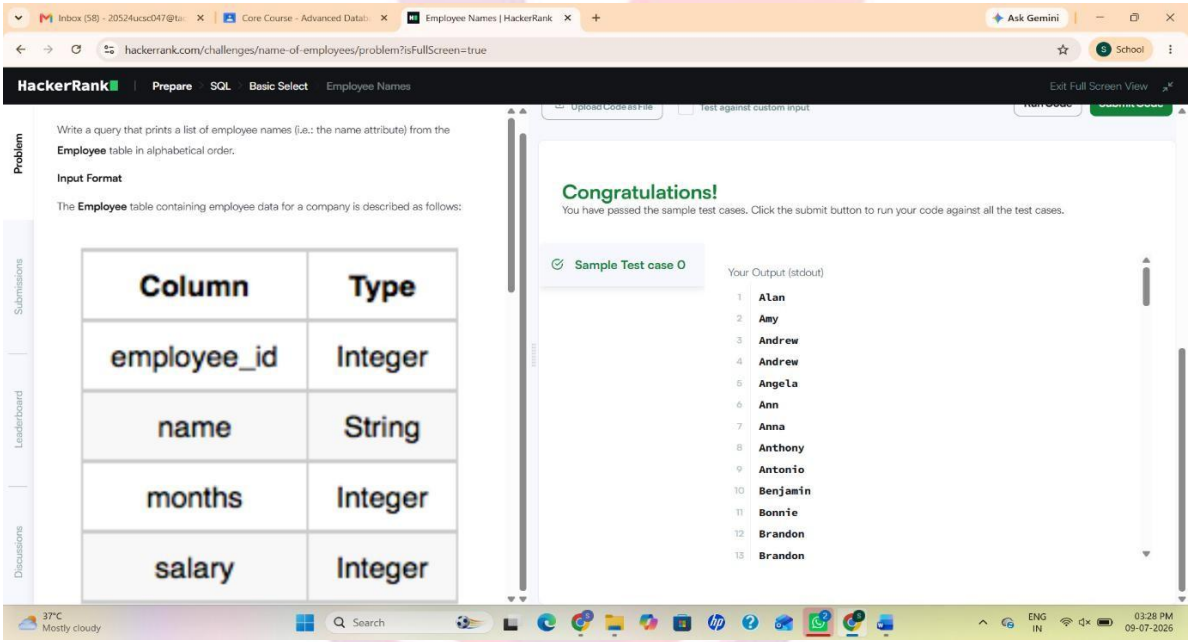


18. EMPLOYEE NAMES

PROGRAM

```
SELECT name
FROM Employee ORDER BY name;
```

OUTPUT

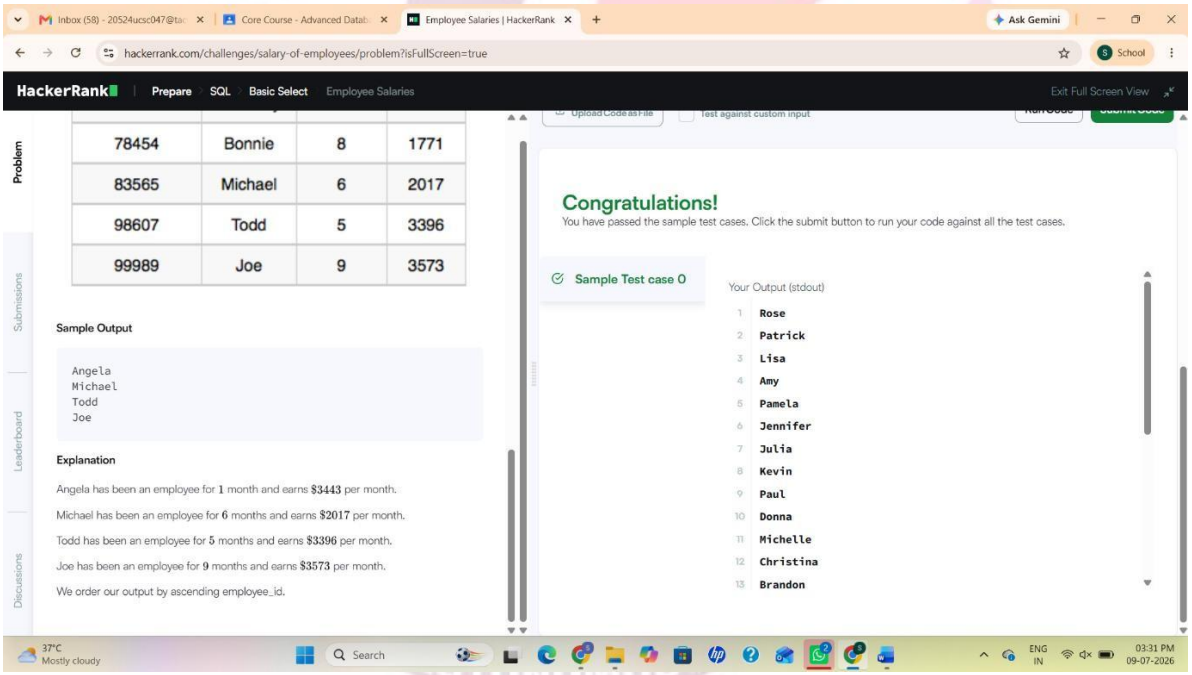


19. EMPLOYEE SALARIES

PROGRAM

```
SELECT name
FROM Employee
WHERE salary > 2000 AND months < 10 ORDER BY employee_id;
```

OUTPUT

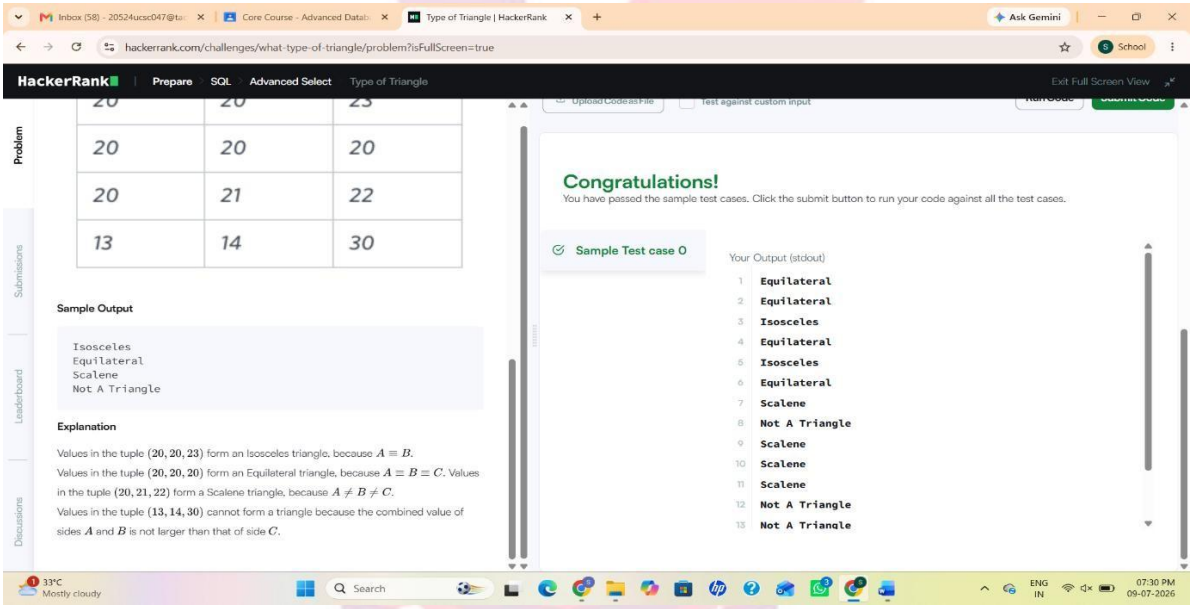


20. TYPES OF TRIANGLE

PROGRAM

```
SELECT CASE WHEN A + B <= C OR A + C <= B OR B + C <= A THEN 'Not A Triangle' WHEN A =
B AND B = C THEN 'Equilateral' WHEN A = B OR B = C OR A = C THEN
'Isosceles' ELSE 'Scalene' END FROM TRIANGLES;
```

OUTPUT

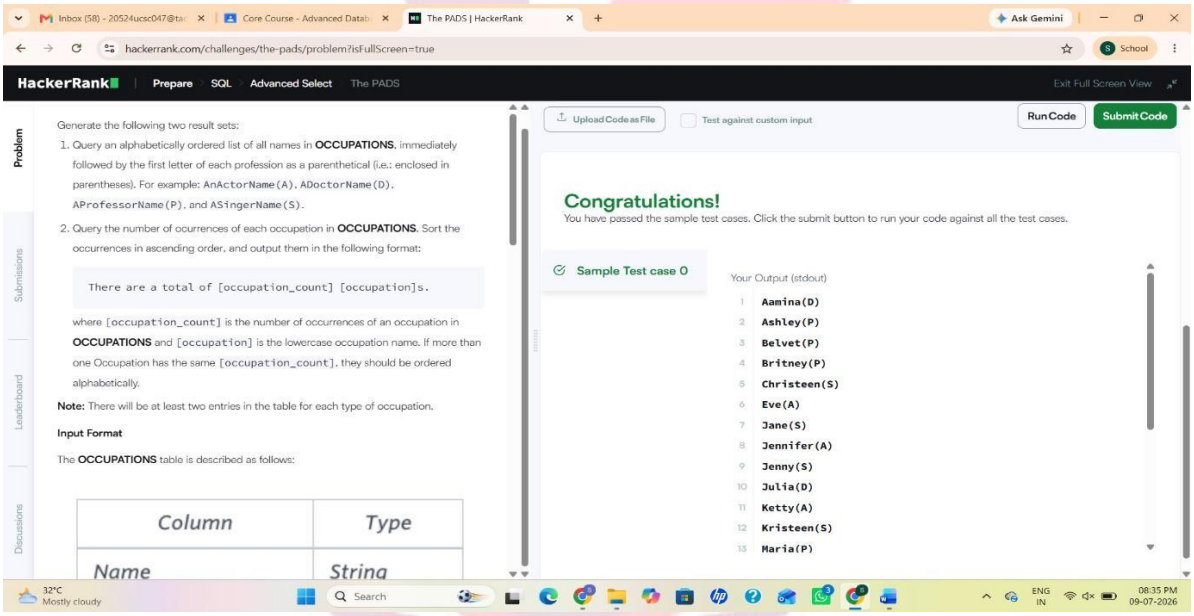


21.THE PADS

PROGRAM

```
SELECT NAME || '(' || SUBSTR(OCCUPATION,1,1) || ')' FROM OCCUPATIONS ORDER BY NAME;
SELECT 'There are a total of ' || COUNT(*) || ' ' || LOWER(OCCUPATION) || 's.' FROM
OCCUPATIONS GROUP BY OCCUPATION ORDER BY COUNT(*), OCCUPATION;
```

OUTPUT

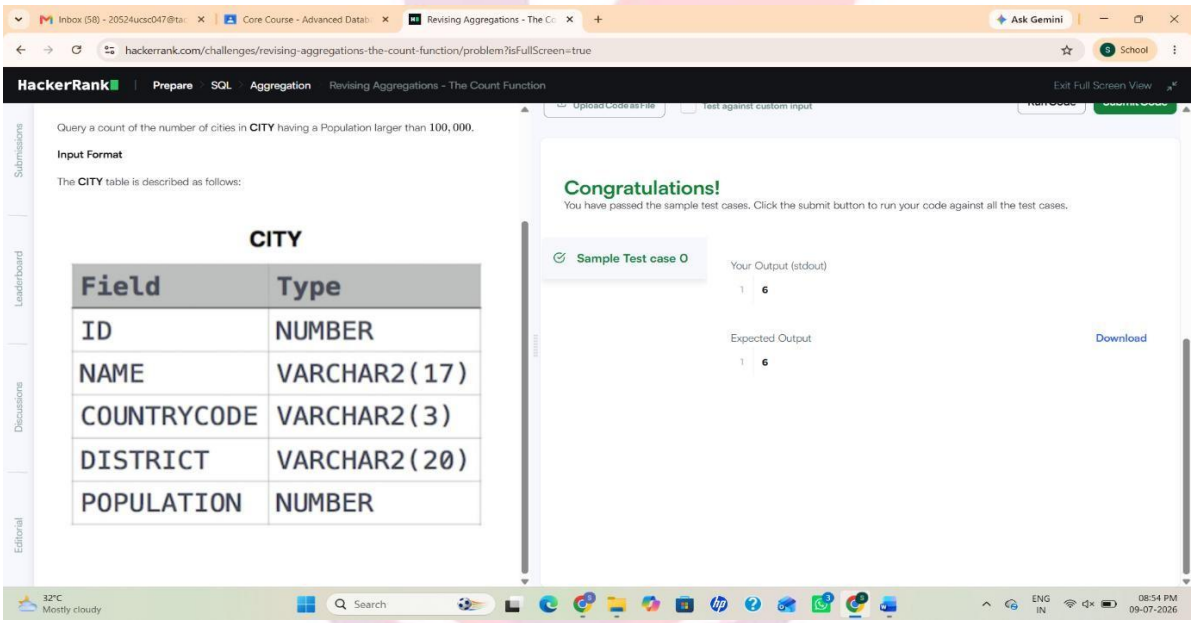


22. REVISING AGGREGATIONS – THE COUNT FUNCTION

PROGRAM

```
SELECT COUNT(*)
FROM CITY
WHERE POPULATION > 100000;
```

OUTPUT



23. REVISING AGGREGATIONS – THE SUM FUNCTION

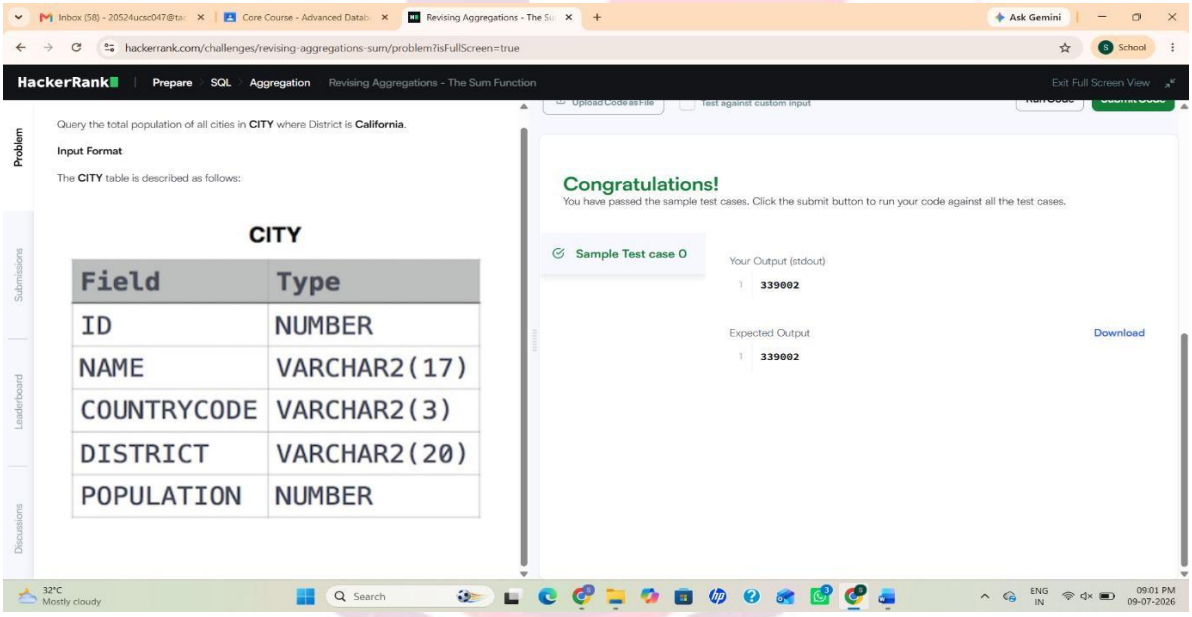
PROGRAM

```
SELECT SUM(POPULATION)

FROM CITY

WHERE DISTRICT = 'California';
```

OUTPUT

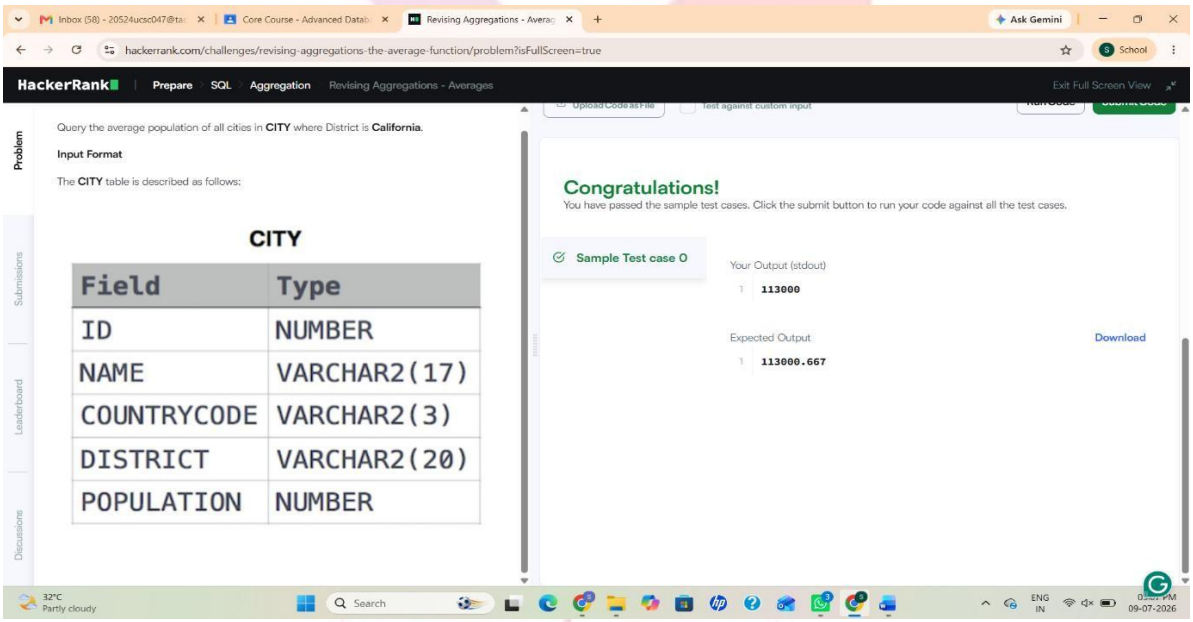


24. REVISING AGGREGATIONS – AVERAGES

PROGRAM

```
SELECT AVG(POPULATION) FROM CITY
WHERE DISTRICT = 'California';
```

OUTPUT



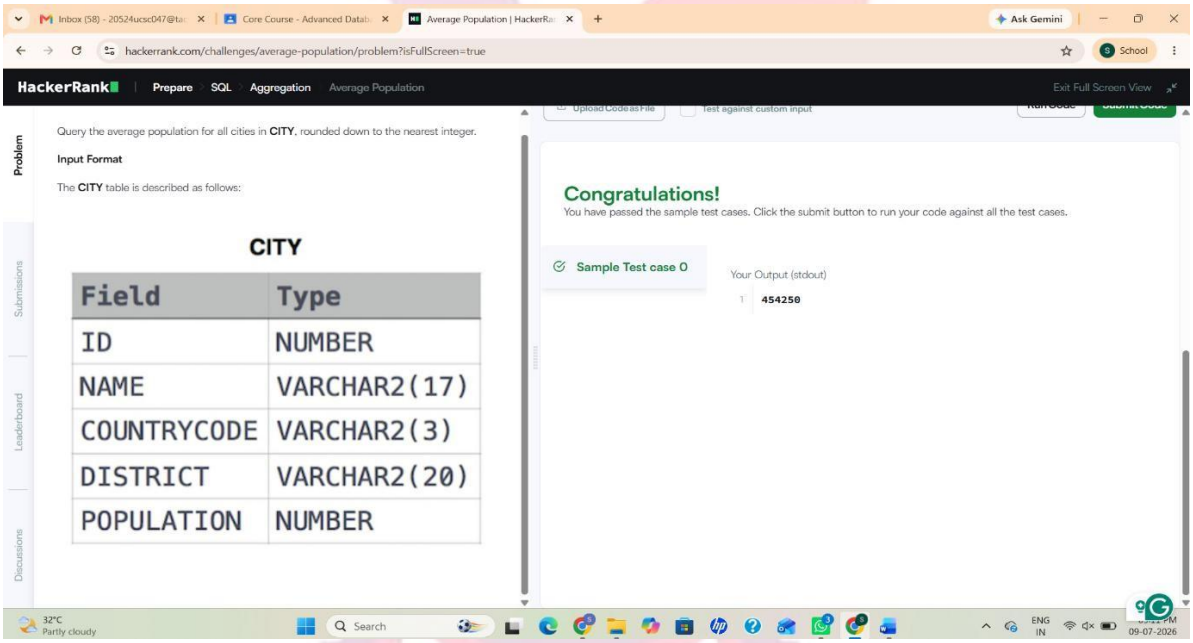
25.AVERGE POPULATION

PROGRAM

```
SELECT FLOOR(AVG(POPULATION))

FROM CITY;
```

OUTPUT



26. JAPAN POPULATION

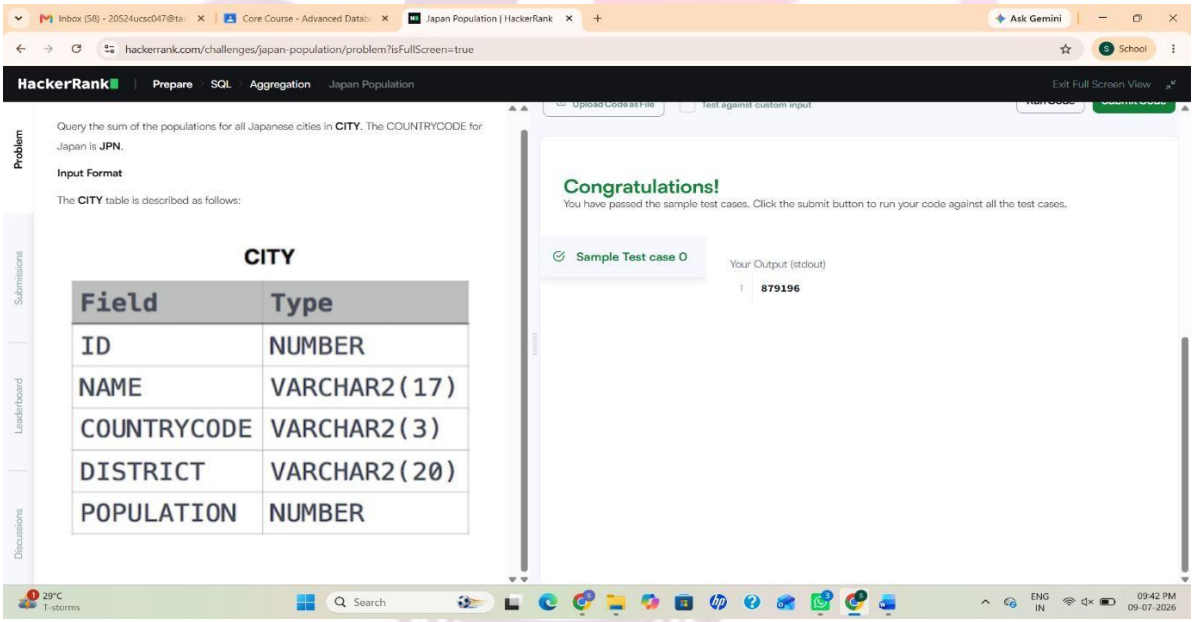
PROGRAM

```
SELECT SUM(POPULATION)

FROM CITY

WHERE COUNTRYCODE = 'JPN';
```

OUTPUT

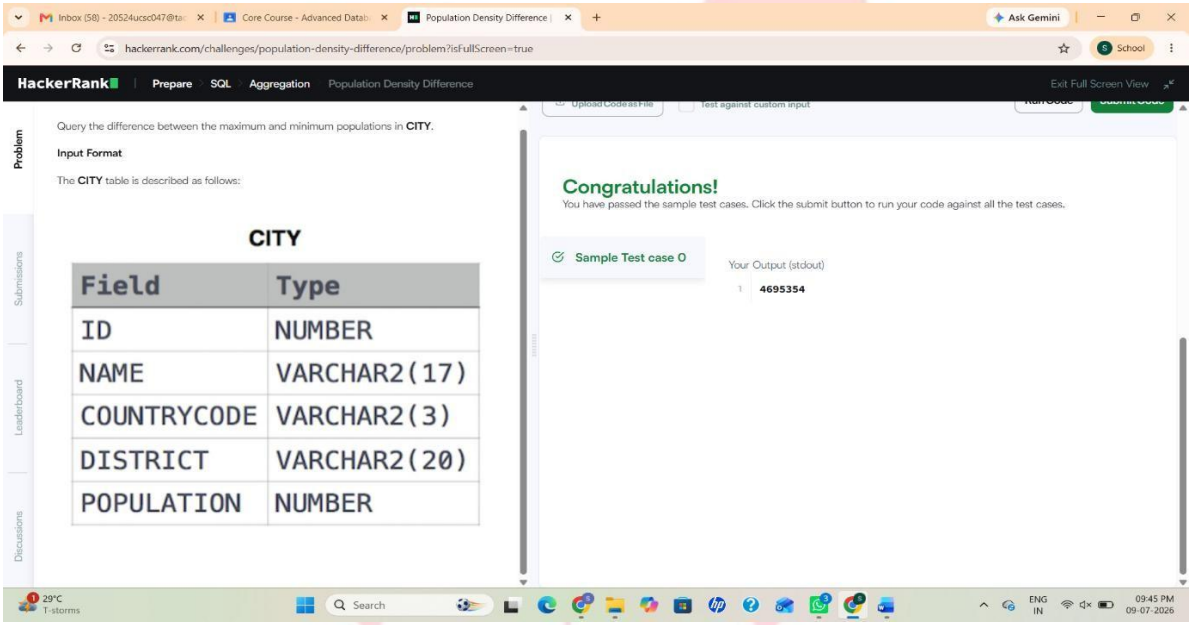


27. POPULATION DENSITY DIFFERENCE

PROGRAM

```
SELECT MAX(POPULATION) - MIN(POPULATION) FROM CITY;
```

OUTPUT



28. TOP EARNERS

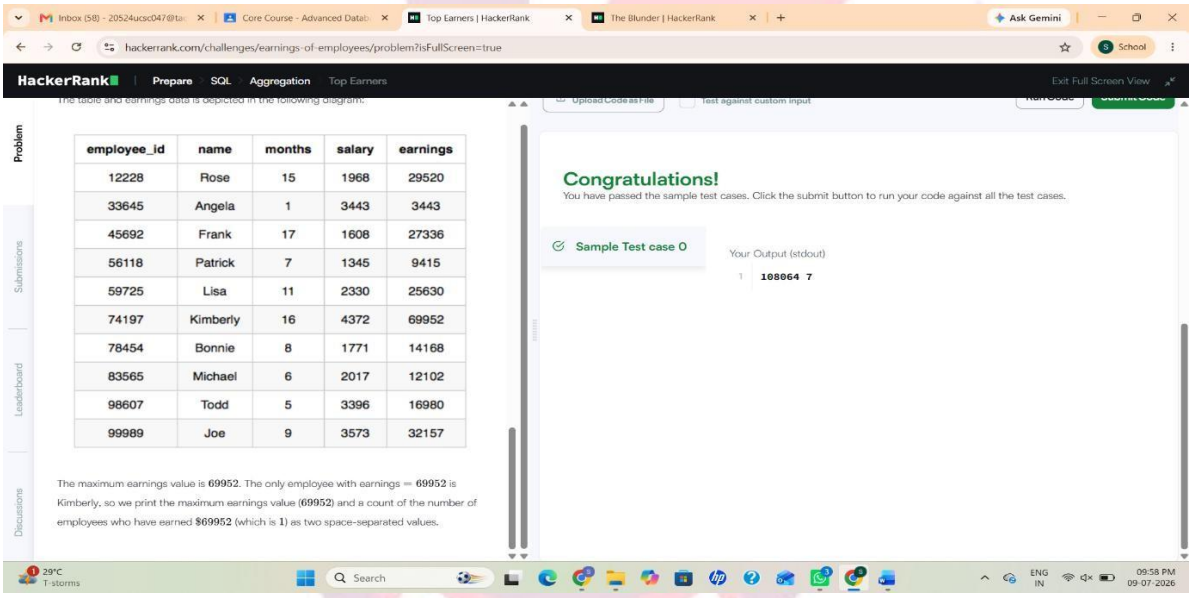
PROGRAM

```
SELECT MAX(salary * months), COUNT(*)

FROM Employee

WHERE salary * months = (SELECT MAX(salary * months) FROM Employee);
```

OUTPUT

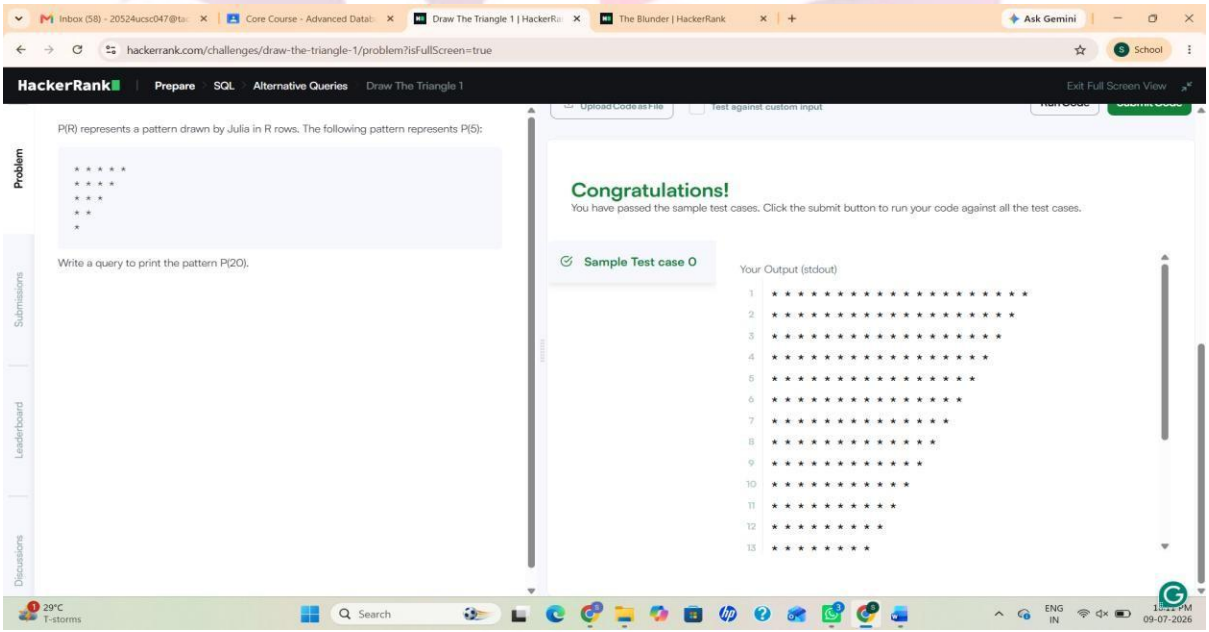


29. DRAW THE TRIANGLE 1

PROGRAM

```
SET @n = 21;
SELECT REPEAT('* ', @n := @n - 1)
FROM information_schema.tables LIMIT
20;
```

OUTPUT

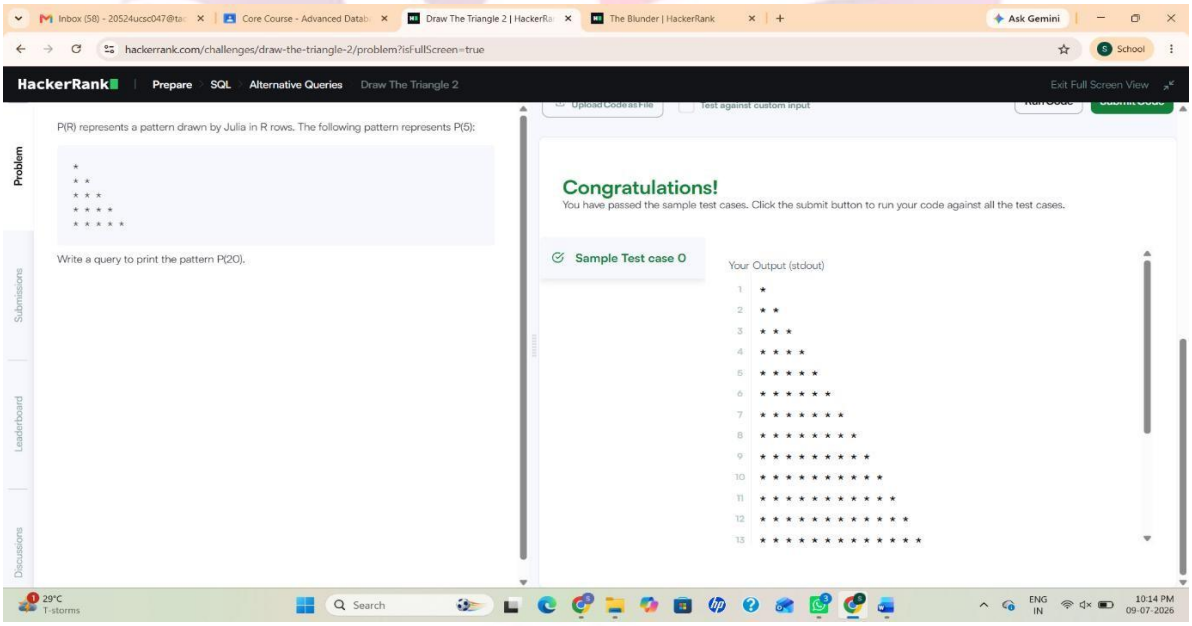


30. DRAW THE TRIANGLE 2

PROGRAM

```
SELECT REPEAT('* ', @n := @n + 1)
FROM information_schema.tables, (SELECT
@n := 0) t
LIMIT 20;
```

OUTPUT

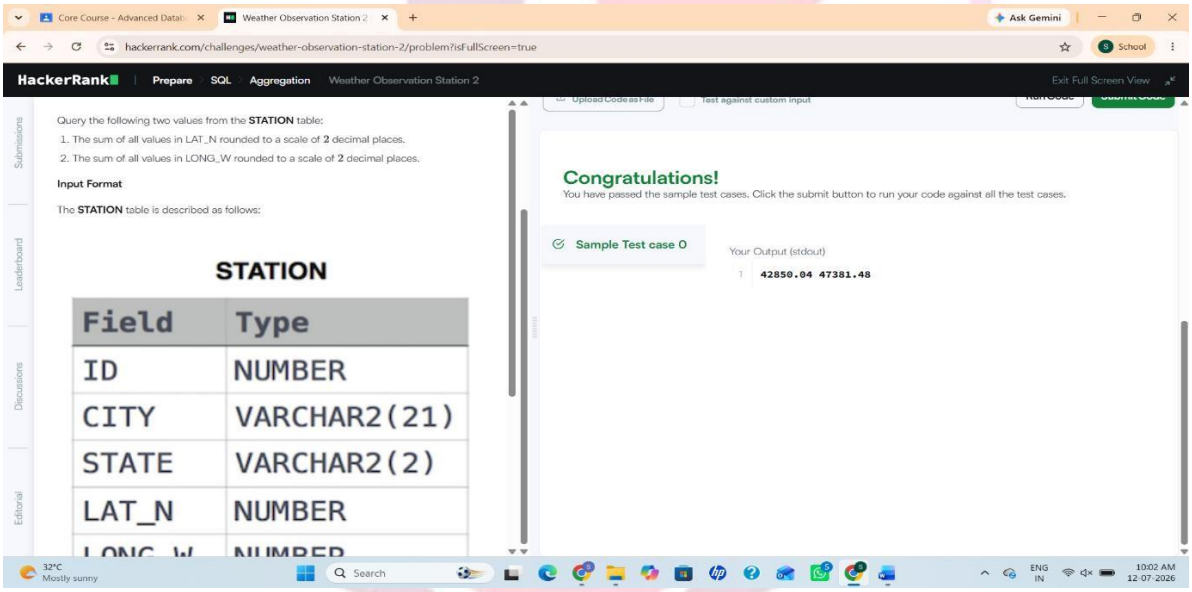


31. WEATHER OBSERVATION STATION 2

PROGRAM

```
SELECT ROUND(SUM(LAT_N), 2),
ROUND(SUM(LONG_W), 2)
FROM STATION;
```

OUTPUT



32. WEATHER OBSERVATION STATION 13

PROGRAM

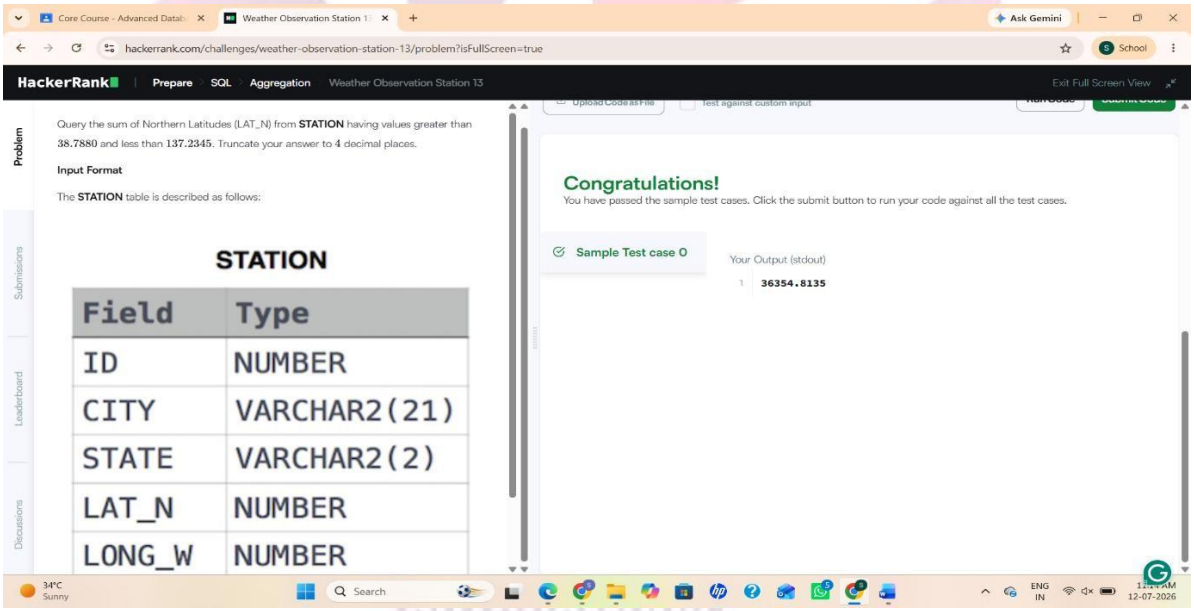
```
SELECT TRUNCATE(SUM(LAT_N), 4)

FROM STATION

WHERE LAT_N > 38.7880 AND

 LAT_N < 137.2345;
```

OUTPUT

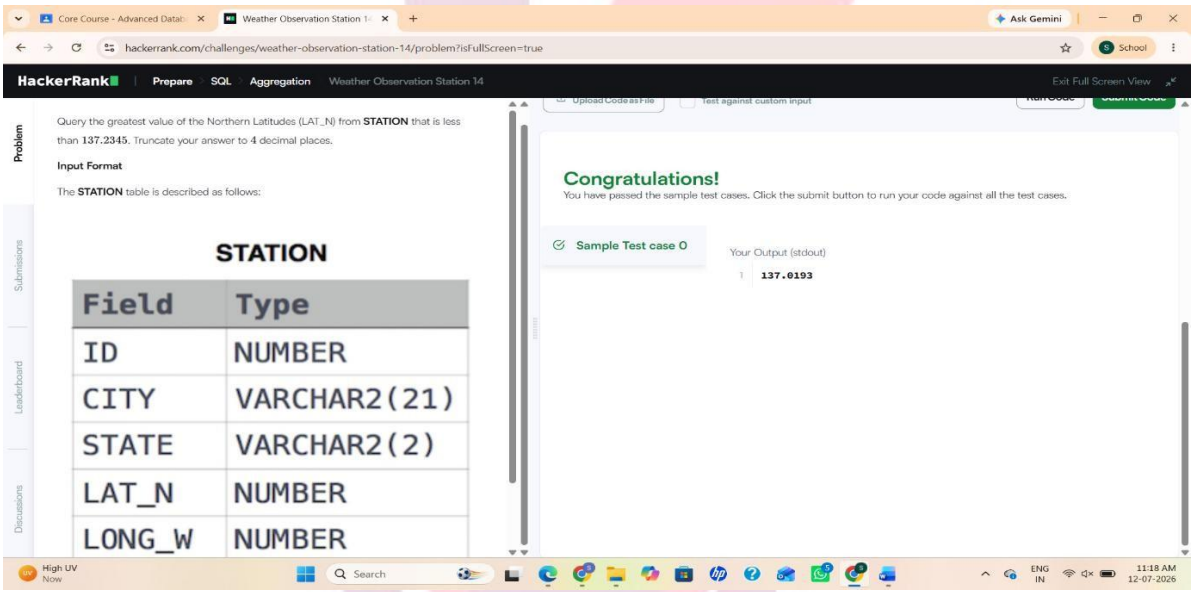


33. WEATHER OBSERVATION STATION 14

PROGRAM

```
SELECT TRUNCATE(MAX(LAT_N), 4) FROM
STATION
WHERE LAT_N < 137.2345;
```

OUTPUT



34. WEATHER OBSERVATION STATION 15

PROGRAM

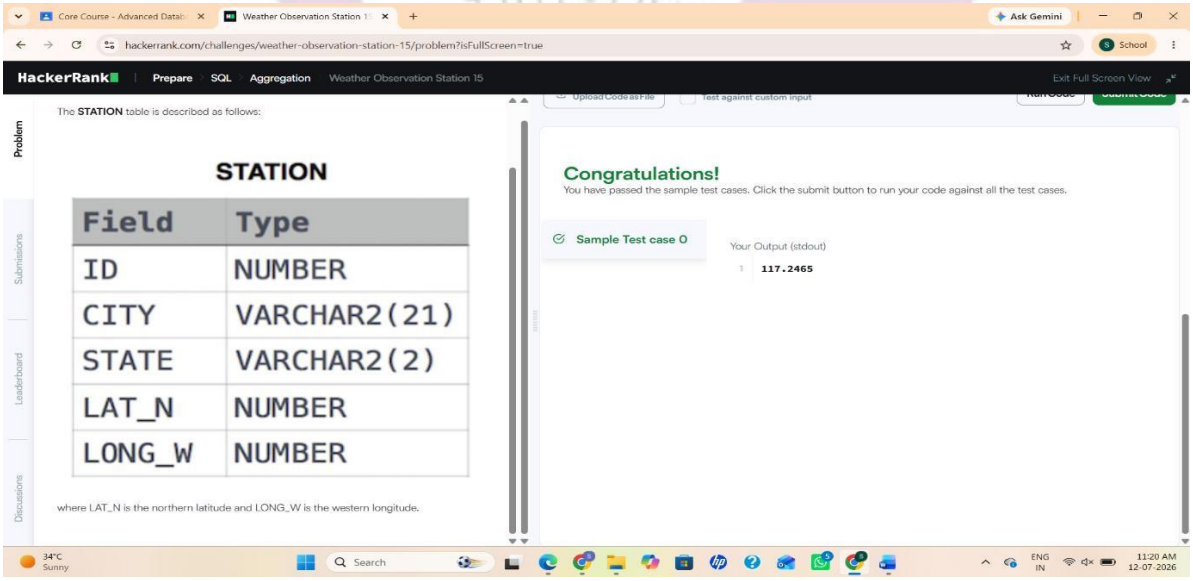
```
SELECT ROUND(LONG_W, 4)

 FROM STATION
WHERE LAT_N = ( SELECT

   MAX(LAT_N) FROM STATION

   WHERE LAT_N < 137.2345
);
```

OUTPUT



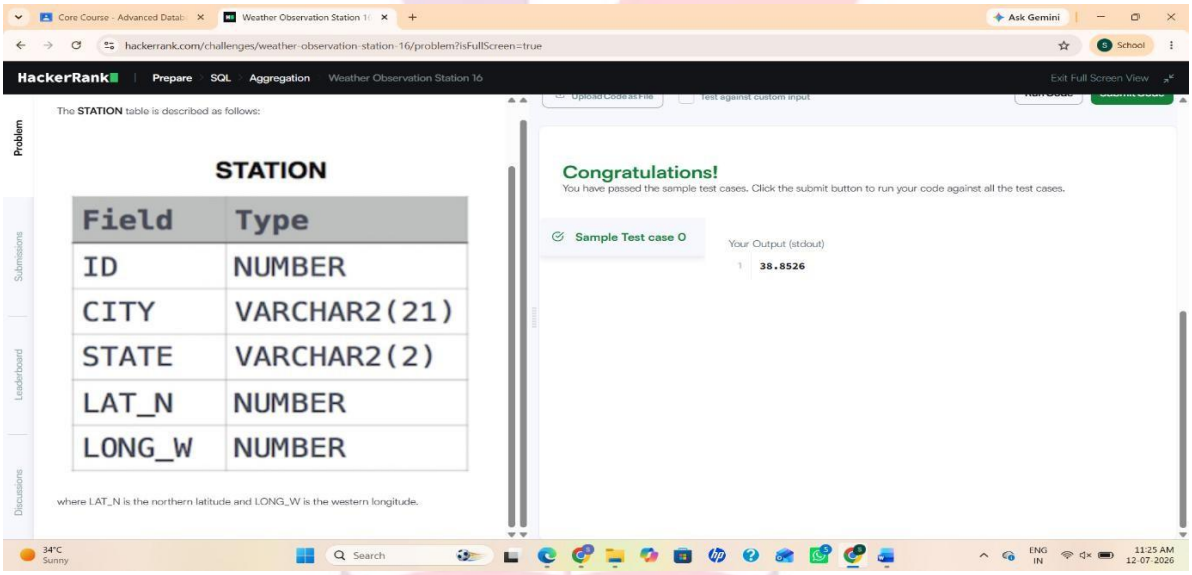
35. WEATHER OBSERVATION STATION 16

PROGRAM

```
SELECT ROUND(MIN(LAT_N), 4)

FROM STATION
WHERE LAT_N > 38.7780;
```

OUTPUT



36. WEATHER OBSERVATION STATION 17

PROGRAM

```
SELECT ROUND(LONG_W, 4)

FROM STATION

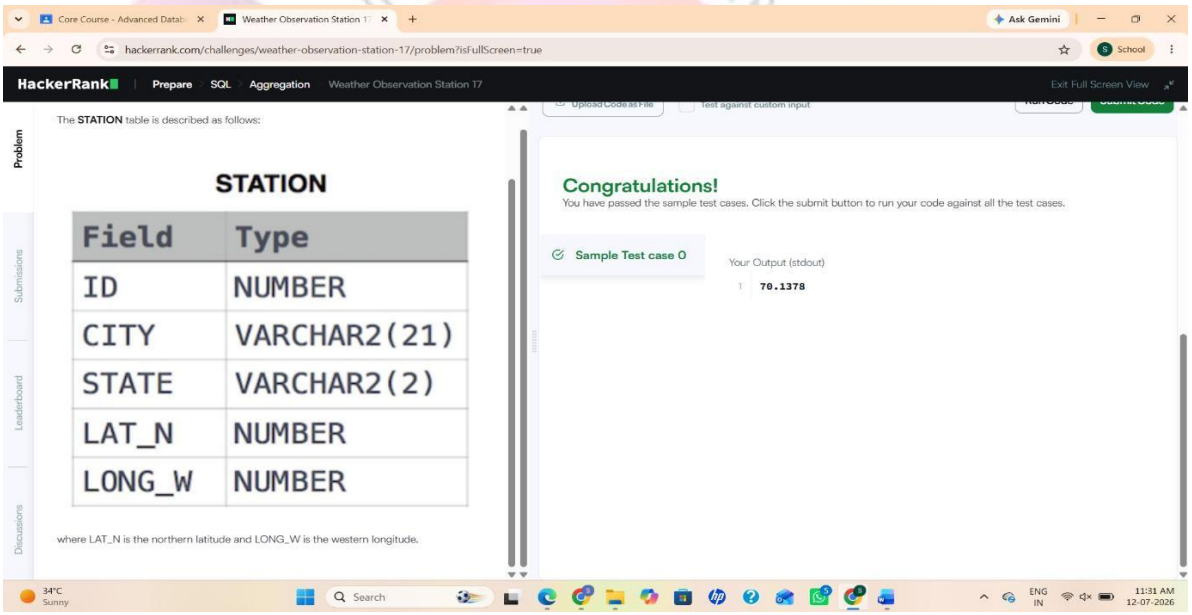
WHERE LAT_N = ( SELECT

   MIN(LAT_N) FROM STATION

   WHERE LAT_N > 38.7780

);
```

OUTPUT

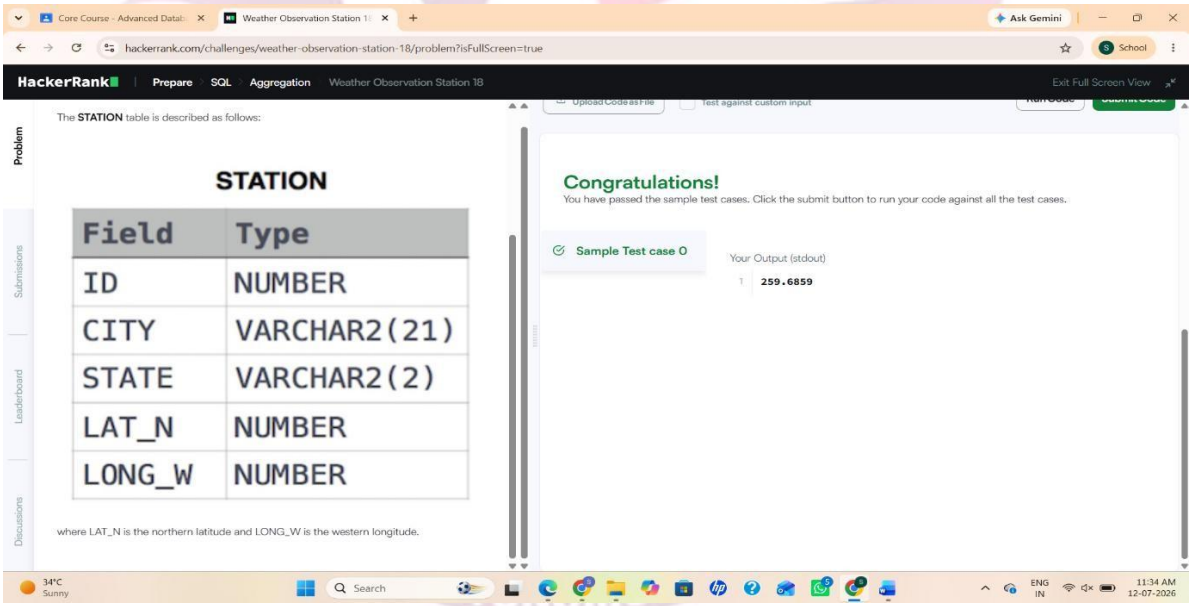


37. WEATHER OBSERVATION STATION 18

PROGRAM

```
SELECT ROUND(
   ABS(MAX(LAT_N) - MIN(LAT_N)) +
   ABS(MAX(LONG_W) - MIN(LONG_W)), 4
)
FROM STATION;
```

OUTPUT

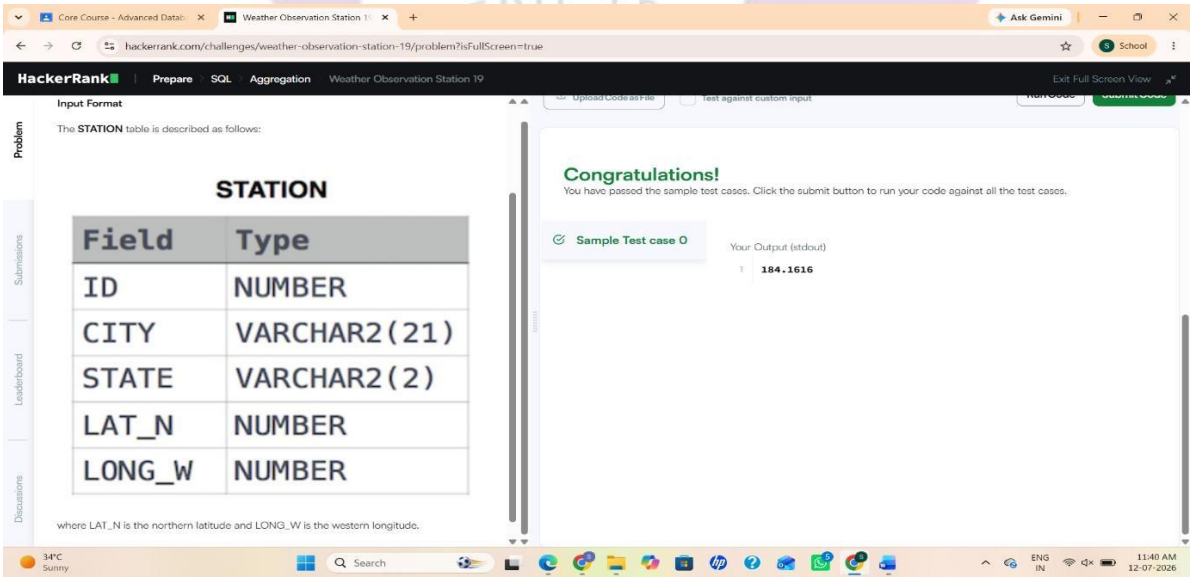


38. WEATHER OBSERVATION STATION 19

PROGRAM

```
SELECT ROUND( SQRT(
      POWER(MAX(LAT_N) - MIN(LAT_N), 2) +
      POWER(MAX(LONG_W) - MIN(LONG_W), 2)
   ), 4
)
FROM STATION;
```

OUTPUT



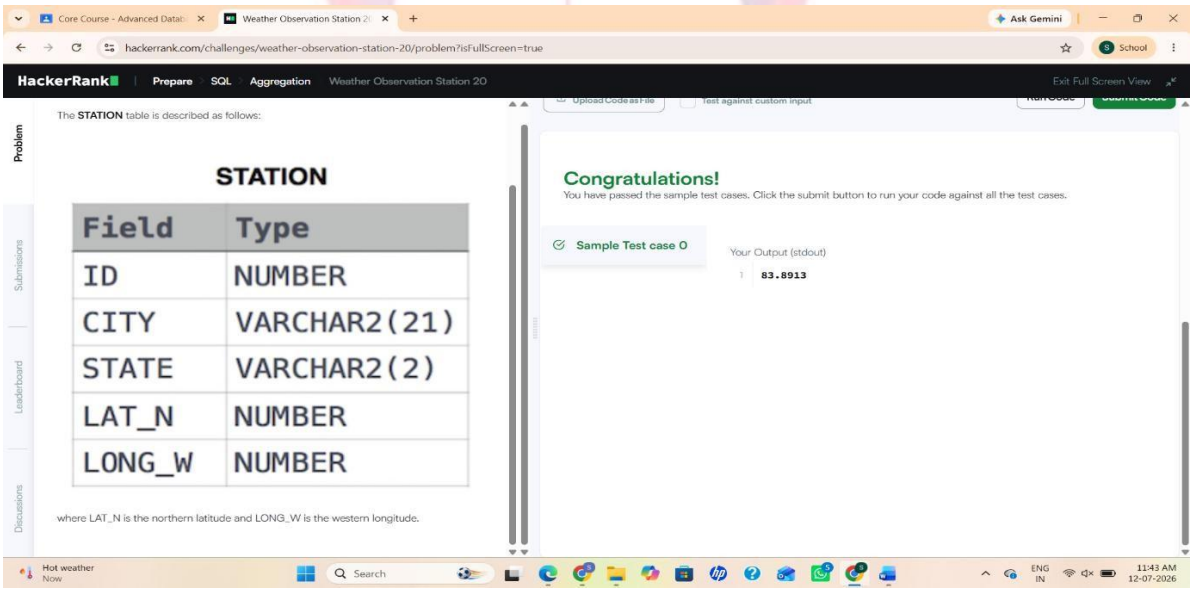
39. WEATHER OBSERVATION STATION 20

PROGRAM

```
SET @rownum := 0;


SELECT ROUND(AVG(LAT_N), 4) FROM (
    SELECT LAT_N,
        @rownum := @rownum + 1 AS row_num,
        (SELECT COUNT(*) FROM STATION) AS total_rows FROM
    STATION
    ORDER BY LAT_N
) AS t
WHERE row_num IN (FLOOR((total_rows + 1) / 2), FLOOR((total_rows + 2) / 2));
```

OUTPUT



40. POPULATION CENSUS

PROGRAM

```
SELECT SUM(CITY.Population)

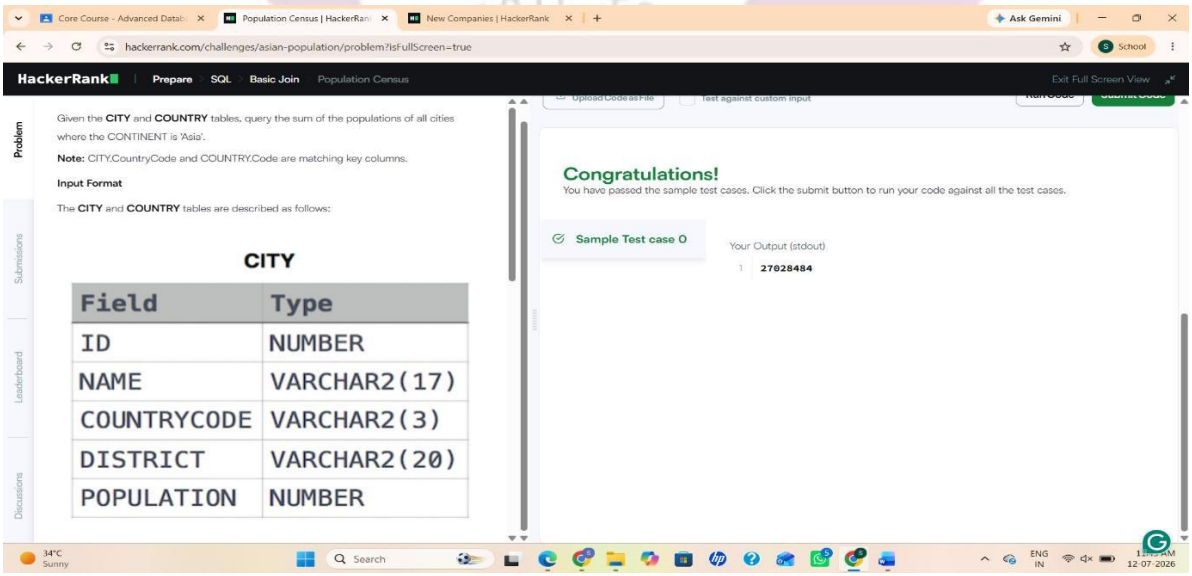
FROM CITY

JOIN COUNTRY

ON CITY.CountryCode = COUNTRY.Code WHERE

COUNTRY.Continent = 'Asia';
```

OUTPUT



41. AFRICAN CITIES

PROGRAM

```
SELECT CITY.NAME

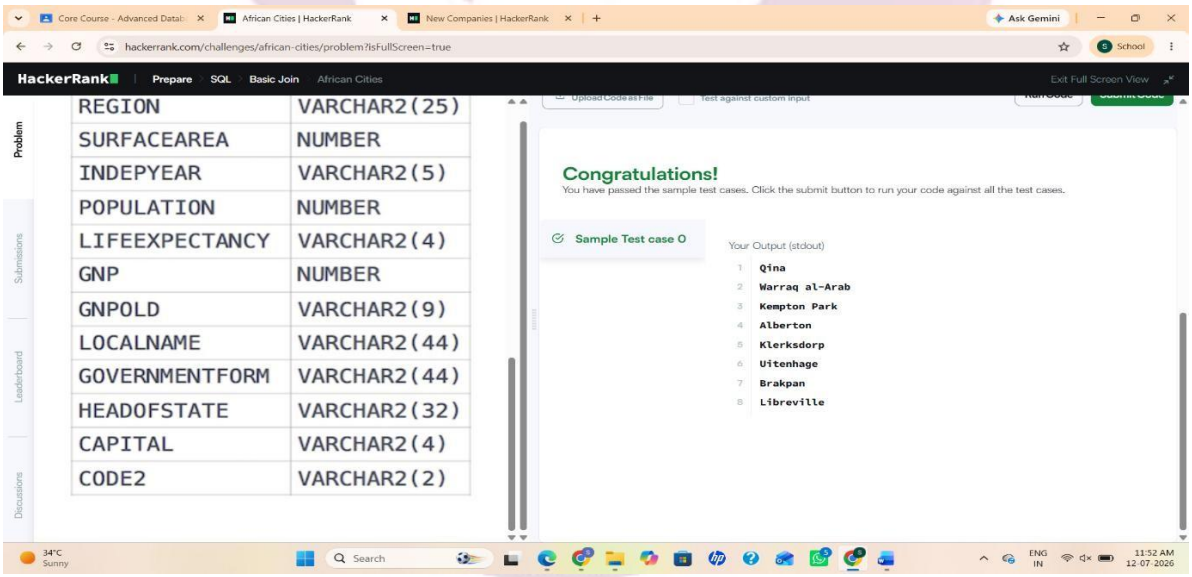
FROM CITY

JOIN COUNTRY

ON CITY.CountryCode = COUNTRY.Code WHERE

COUNTRY.Continent = 'Africa';
```

OUTPUT

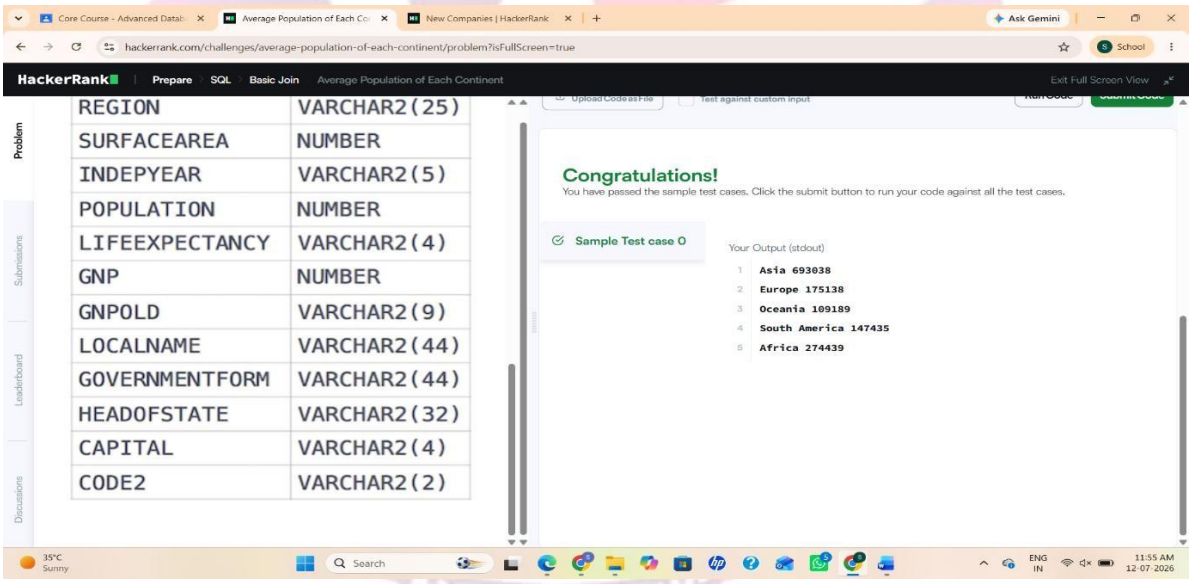


42. AVERAGE POPULATION OF EACH CONTINENT

PROGRAM

```
SELECT COUNTRY.Continent, FLOOR(AVG(CITY.Population)) FROM CITY
JOIN COUNTRY
ON CITY.CountryCode = COUNTRY.Code GROUP BY
COUNTRY.Continent;
```

OUTPUT

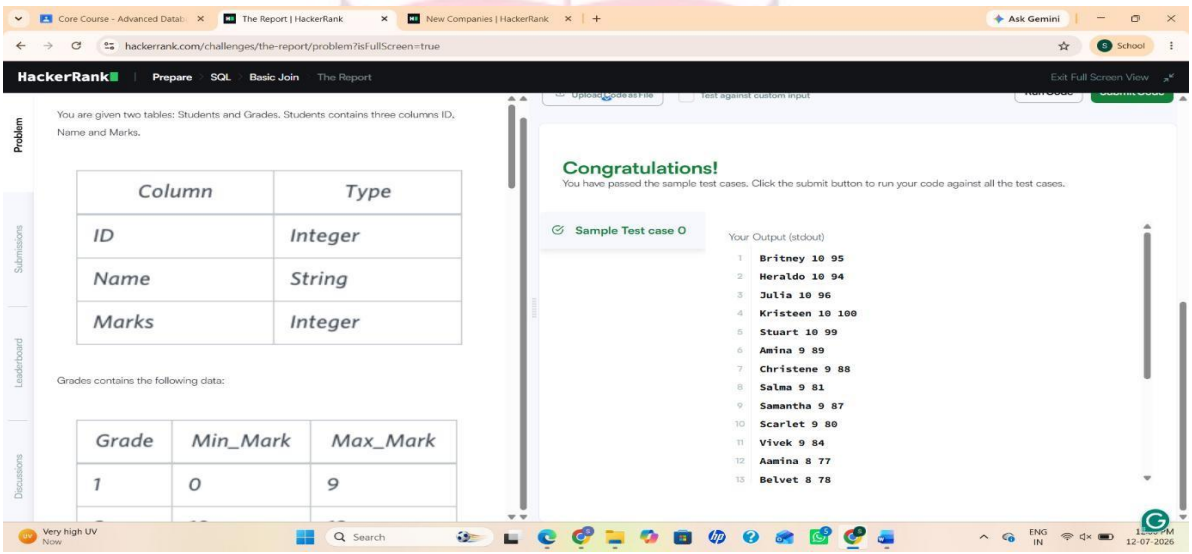


43. THE REPORT

PROGRAM

```
SELECT CASE
    WHEN G.Grade < 8 THEN 'NULL' ELSE S.Name
  END AS Name, G.Grade,
  S.Marks
FROM Students S JOIN
Grades G
ON S.Marks BETWEEN G.Min_Mark AND G.Max_Mark ORDER BY
  G.Grade DESC, CASE
    WHEN G.Grade >= 8 THEN S.Name END ASC,
  CASE
    WHEN G.Grade < 8 THEN S.Marks END
  ASC;
```

OUTPUT



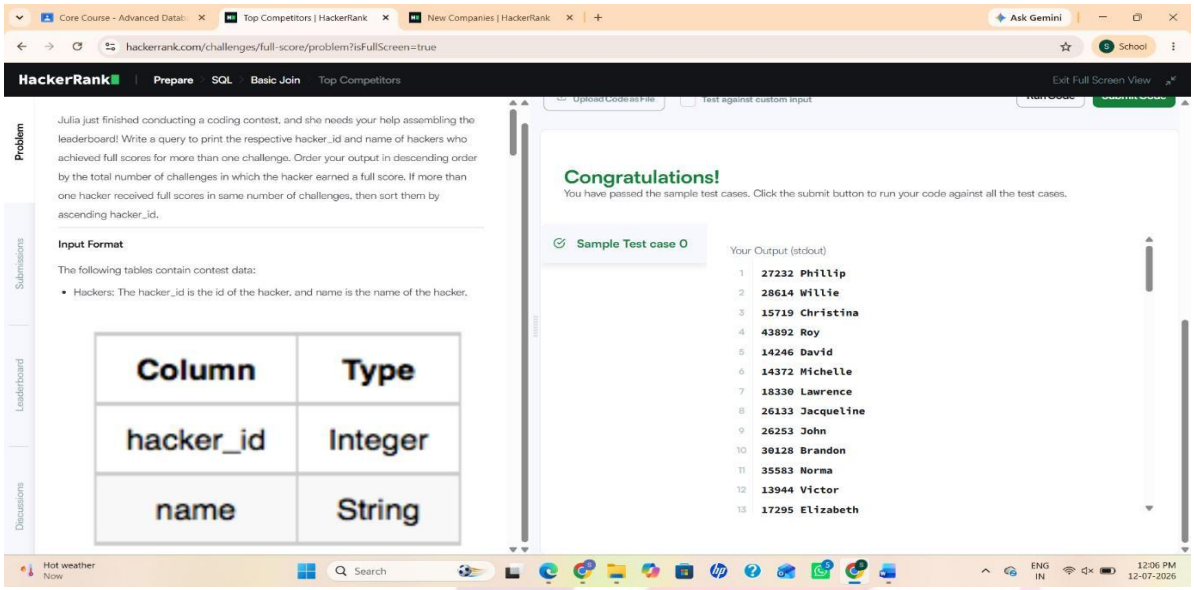
44. TOP COMPETITORS

PROGRAM

```
SELECT
   h.hacker_id, h.name
FROM Hackers h JOIN
Submissions s
   ON h.hacker_id = s.hacker_id JOIN
Challenges c
   ON s.challenge_id = c.challenge_id JOIN
Difficulty d
   ON c.difficulty_level = d.difficulty_level WHERE
s.score = d.score
GROUP BY h.hacker_id, h.name
HAVING COUNT(DISTINCT s.challenge_id) > 1 ORDER BY
COUNT(DISTINCT s.challenge_id) DESC,
      h.hacker_id ASC;
```

TRUTH-TOIL-TRIUMPH
AUTONOMOUS

OUTPUT



TRUTH-TOIL-TRIUMPH
AUTONOMOUS

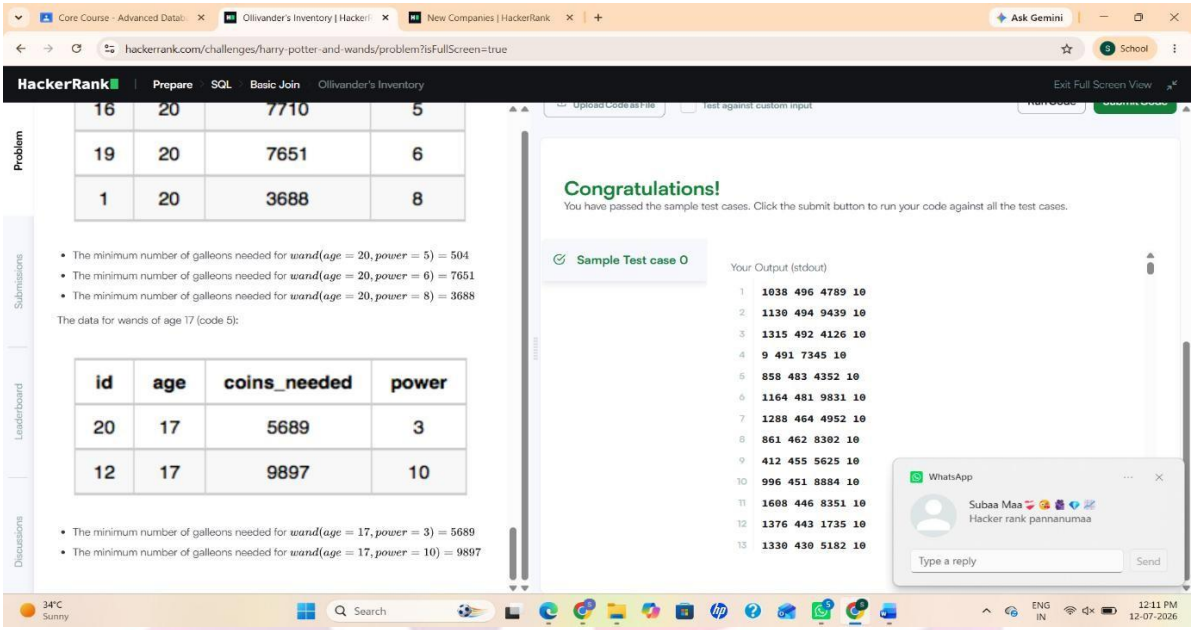
45. OLLIVANDER'S INVENTORY

PROGRAM

```
SELECT
   w.id, wp.age,
   w.coins_needed, w.power
FROM Wands w
JOIN Wands_Property wp ON
   w.code = wp.code
WHERE wp.is_evil = 0 AND
 w.coins_needed = (
    SELECT MIN(w2.coins_needed) FROM
    Wands w2
    JOIN Wands_Property wp2 ON
       w2.code = wp2.code
    WHERE wp2.is_evil = 0 AND
     w2.power = w.power AND wp2.age
     = wp.age
 )
ORDER BY
   w.power DESC, wp.age
   DESC;
```

TRUTH-TOIL-TRIUMPH
AUTONOMOUS

OUTPUT



TRUTH-TOIL-TRIUMPH
AUTONOMOUS

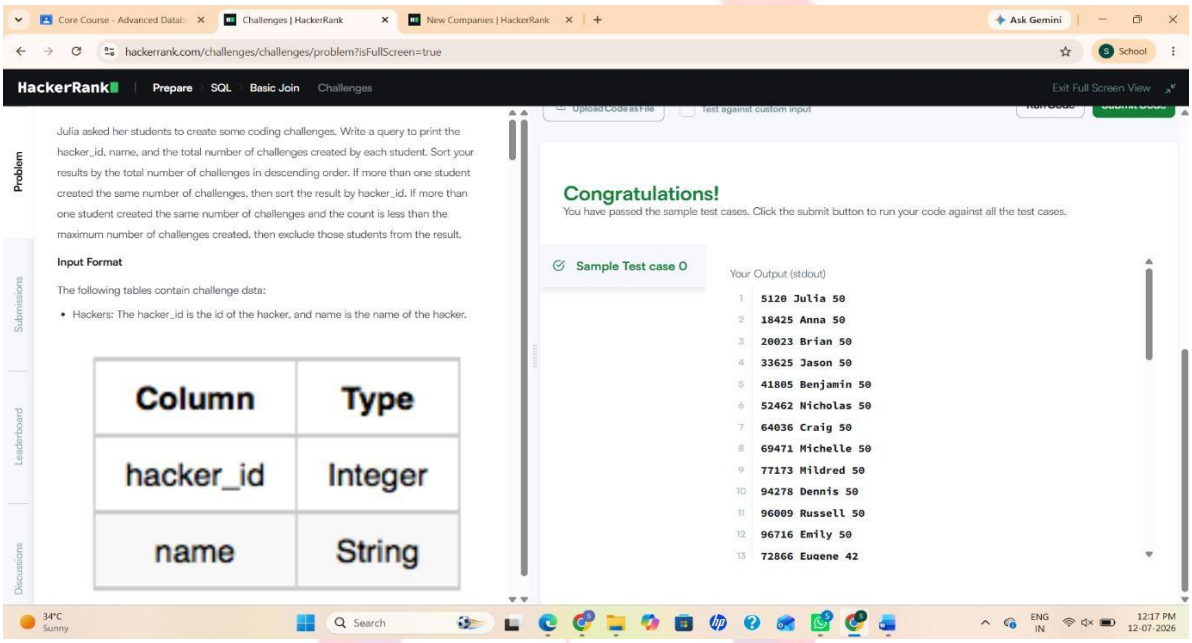
46.CHALLENGES

PROGRAM

```
SELECT h.hacker_id, h.name,
    COUNT(c.challenge_id) AS total_challenges FROM
Hackers h
JOIN Challenges c
ON h.hacker_id = c.hacker_id GROUP BY
h.hacker_id, h.name HAVING
COUNT(c.challenge_id) = (
        SELECT MAX(cnt) FROM (
            SELECT COUNT(*) AS cnt
            FROM Challenges GROUP BY
            hacker_id
        ) t
    )
    OR COUNT(c.challenge_id) IN ( SELECT cnt
        FROM (
            SELECT COUNT(*) AS cnt
            FROM Challenges GROUP BY
            hacker_id
        ) t
        GROUP BY cnt HAVING
        COUNT(*) = 1
    )
```

```
ORDER BY total_challenges DESC, h.hacker_id
    ASC;
```

OUTPUT

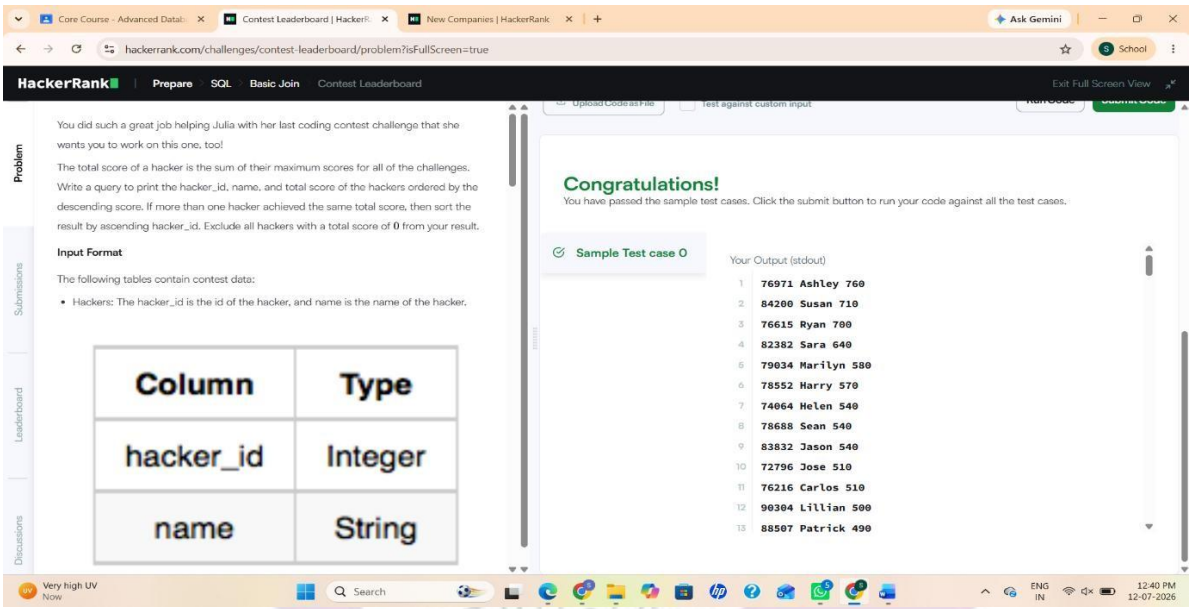


47. CONTEST LEADERBOARD

PROGRAM

```
SELECT
   h.hacker_id, h.name,
   SUM(ms.max_score) AS total_score FROM
Hackers h
JOIN (
   SELECT
      hacker_id,
      challenge_id,
      MAX(score) AS max_score FROM
   Submissions
   GROUP BY hacker_id, challenge_id
) ms
ON h.hacker_id = ms.hacker_id GROUP BY
h.hacker_id, h.name HAVING
SUM(ms.max_score) > 0 ORDER BY
total_score DESC,
        h.hacker_id ASC;
```

OUTPUT



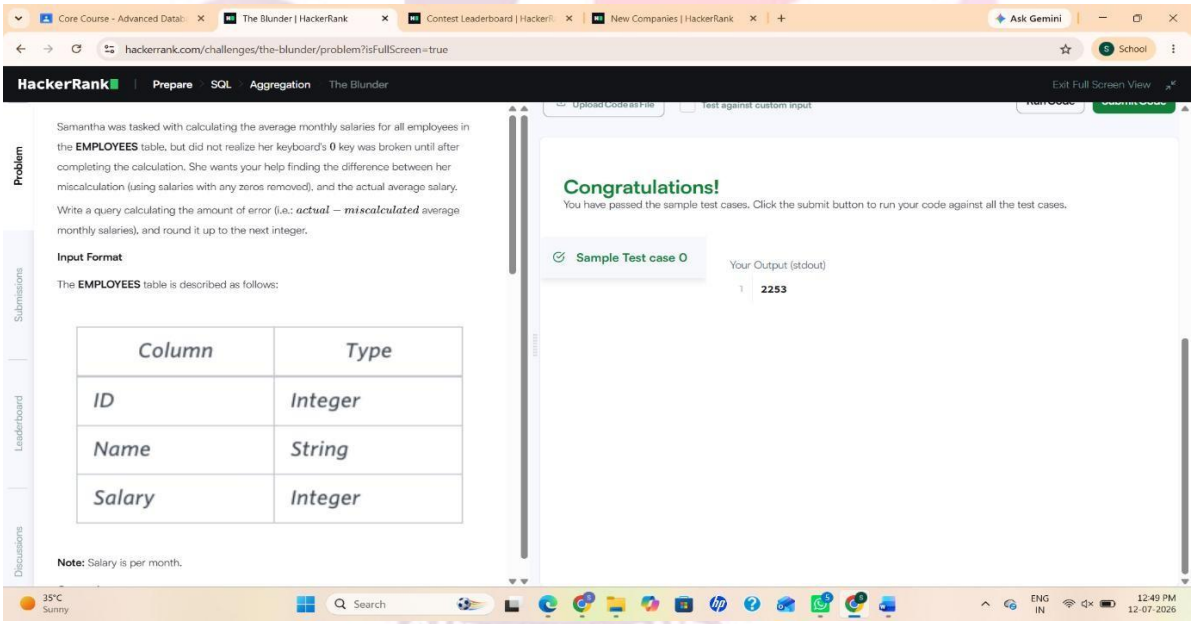
TRUTH · TOIL · TRIUMPH
AUTONOMOUS

48. THE BLUNDER

PROGRAM

```
SELECT CEIL(
   AVG(Salary) - AVG(REPLACE(Salary, '0', ''))
)
FROM EMPLOYEES;
```

OUTPUT

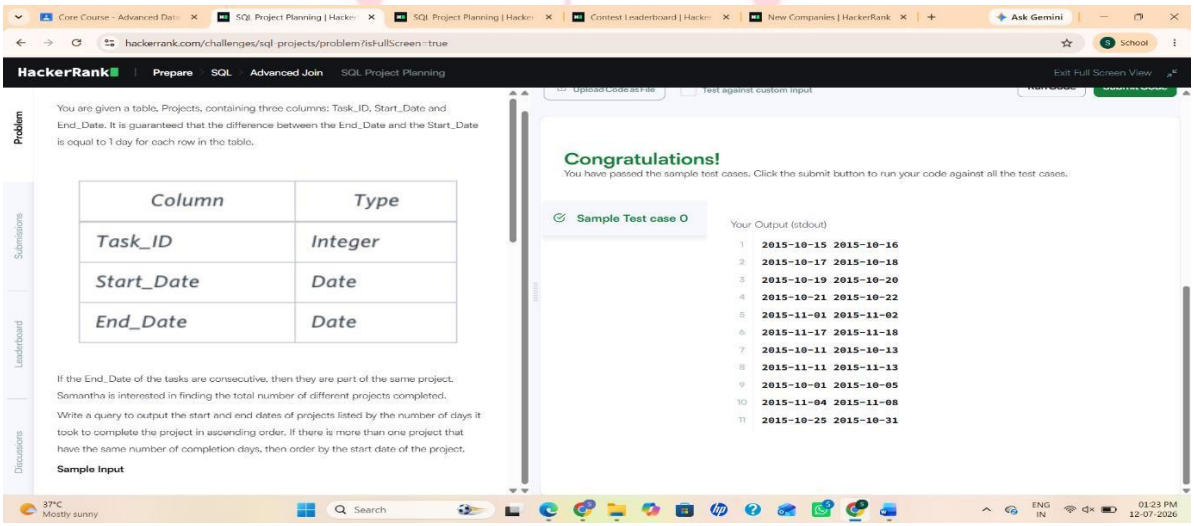


49.SQL PROJECT PLANNING

PROGRAM

```
SELECT
   MIN(Start_Date) AS Start_Date,
   MAX(End_Date) AS End_Date
FROM ( SELECT
       Start_Date, End_Date,
       DATE_SUB(Start_Date, INTERVAL ROW_NUMBER() OVER (ORDER BY
Start_Date) DAY) AS grp FROM
   Projects
) p
GROUP BY grp
ORDER BY DATEDIFF(MAX(End_Date), MIN(Start_Date)),
        MIN(Start_Date);
```

OUTPUT

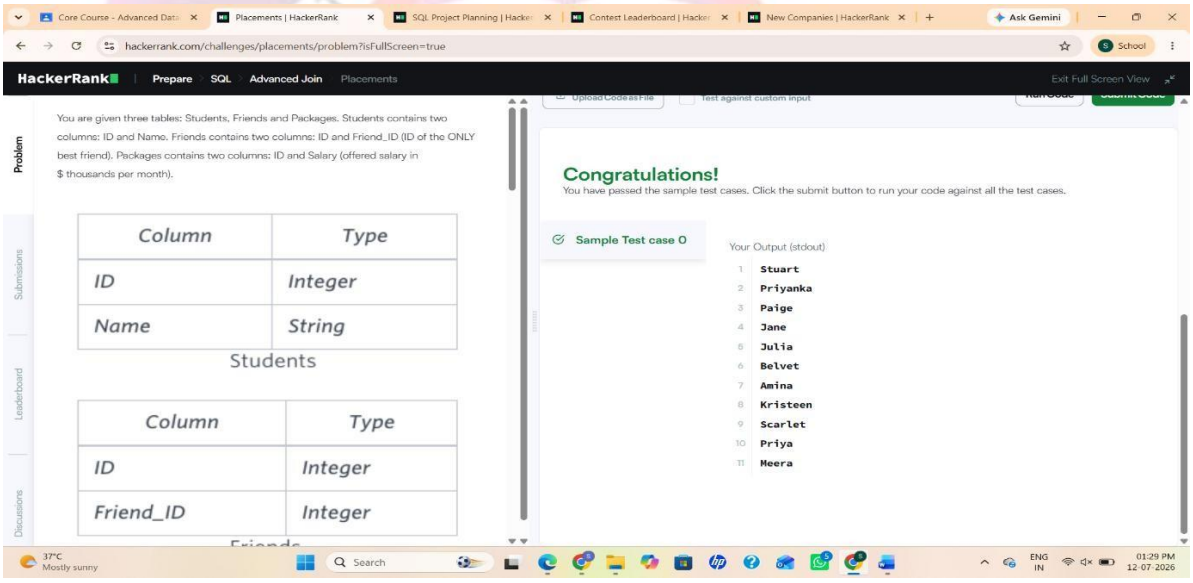


50. PLACEMENTS

PROGRAM

```
SELECT  s.Name  FROM
Students s JOIN Friends f
   ON s.ID = f.ID
JOIN Packages p1 ON s.ID =
   p1.ID
JOIN Packages p2
   ON f.Friend_ID = p2.ID WHERE
p2.Salary > p1.Salary ORDER BY p2.Salary;
```

OUTPUT

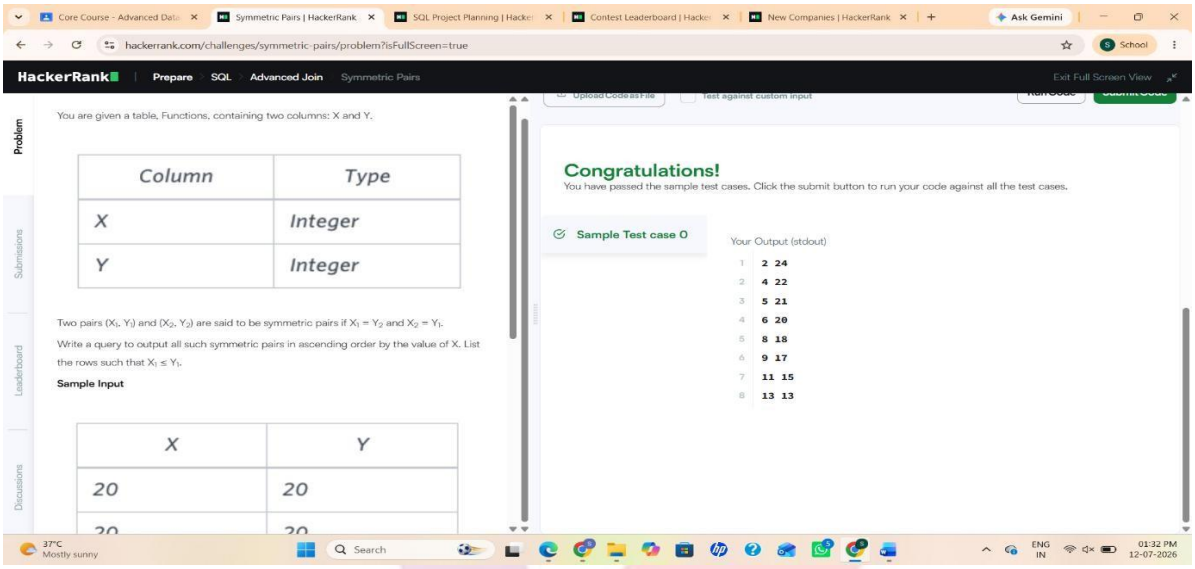


51. SYMMETRIC PAIRS

PROGRAM

```
SELECT DISTINCT f1.X, f1.Y
FROM Functions f1 JOIN
Functions f2 ON f1.X = f2.Y
AND f1.Y = f2.X WHERE
f1.X < f1.Y
  OR (
    f1.X = f1.Y AND (
          SELECT COUNT(*)
       FROM Functions f3 WHERE
        f3.X = f1.X AND f3.Y = f1.Y
    ) > 1
  )
ORDER BY f1.X, f1.Y;
```

OUTPUT



TRUTH-TOIL-TRIUMPH
AUTONOMOUS

52. INTERVIEWS

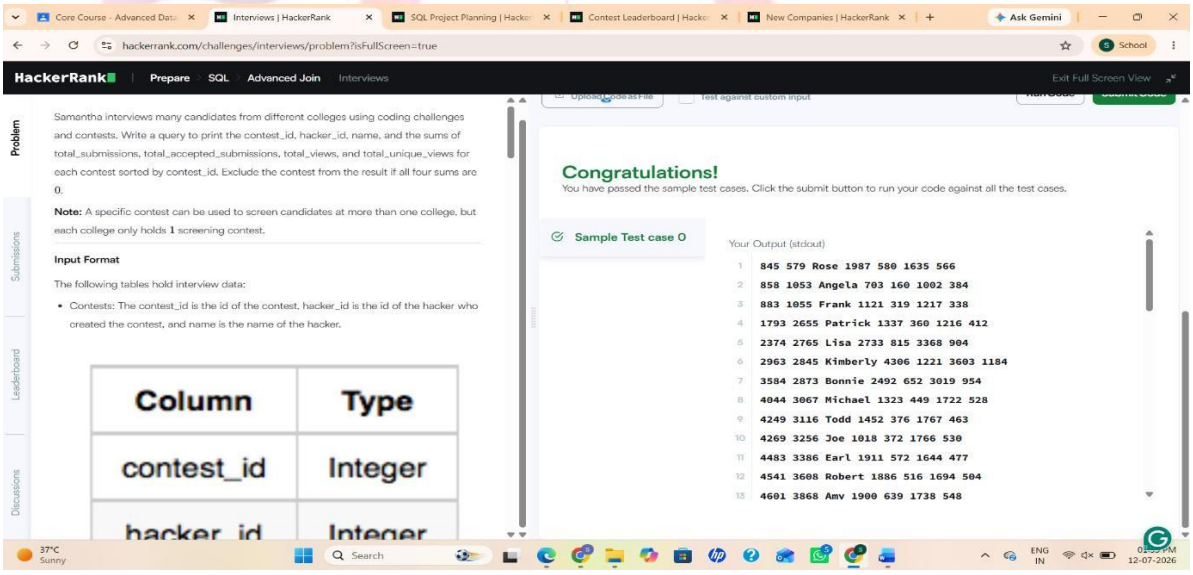
PROGRAM

```
SELECT
   c.contest_id,
   c.hacker_id, c.name,
   SUM(IFNULL(s.total_submissions, 0)) AS total_submissions,
   SUM(IFNULL(s.total_accepted_submissions, 0)) AS total_accepted_submissions,
   SUM(IFNULL(v.total_views, 0)) AS total_views, SUM(IFNULL(v.total_unique_views, 0)) AS
   total_unique_views
FROM Contests c JOIN
Colleges co
   ON c.contest_id = co.contest_id JOIN
Challenges ch
   ON co.college_id = ch.college_id LEFT
JOIN (
   SELECT
      challenge_id,
      SUM(total_submissions) AS total_submissions,
      SUM(total_accepted_submissions) AS total_accepted_submissions
   FROM Submission_Stats GROUP BY
   challenge_id
) s
ON ch.challenge_id = s.challenge_id LEFT JOIN (
   SELECT
      challenge_id,

      SUM(total_views) AS total_views, SUM(total_unique_views) AS
```

```
    total_unique_views
  FROM View_Stats GROUP BY
  challenge_id
) v
ON ch.challenge_id = v.challenge_id GROUP BY
  c.contest_id,
  c.hacker_id, c.name
HAVING
  total_submissions +
  total_accepted_submissions + total_views
  + total_unique_views > 0
ORDER BY
  c.contest_id;
```

OUTPUT

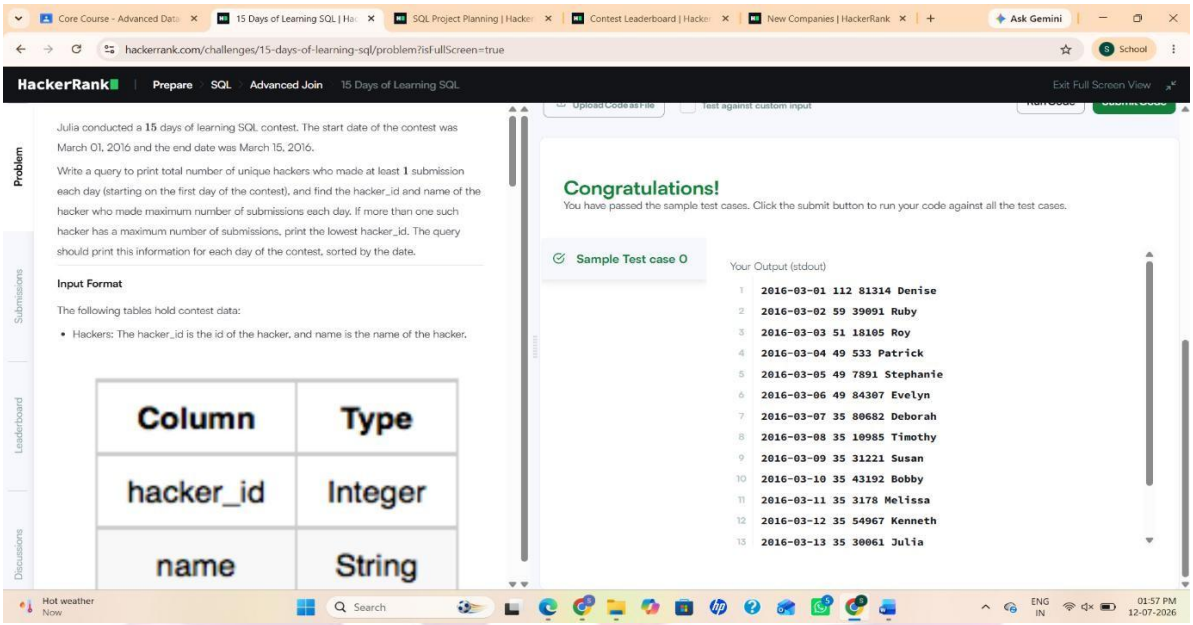


53. 15 DAYS OF LEARNING SQL

PROGRAM

```
SELECT
   d.submission_date, (
      SELECT COUNT(*) FROM (
         SELECT hacker_id FROM
         Submissions s
         WHERE s.submission_date <= d.submission_date GROUP BY
         hacker_id
         HAVING COUNT(DISTINCT s.submission_date)
               = DATEDIFF(d.submission_date,'2016-03-01') + 1
      ) x
   ) AS total_hackers,
   h.hacker_id, h.name
FROM (
   SELECT submission_date, hacker_id,
         ROW_NUMBER() OVER (
            PARTITION BY submission_date ORDER BY
            COUNT(*) DESC, hacker_id
         ) rn
   FROM Submissions
   GROUP BY submission_date,hacker_id
) d
JOIN Hackers h
ON d.hacker_id = h.hacker_id WHERE rn =
1
ORDER BY submission_date;
```

OUTPUT



TRUTH-TOIL-TRIUMPH
AUTONOMOUS

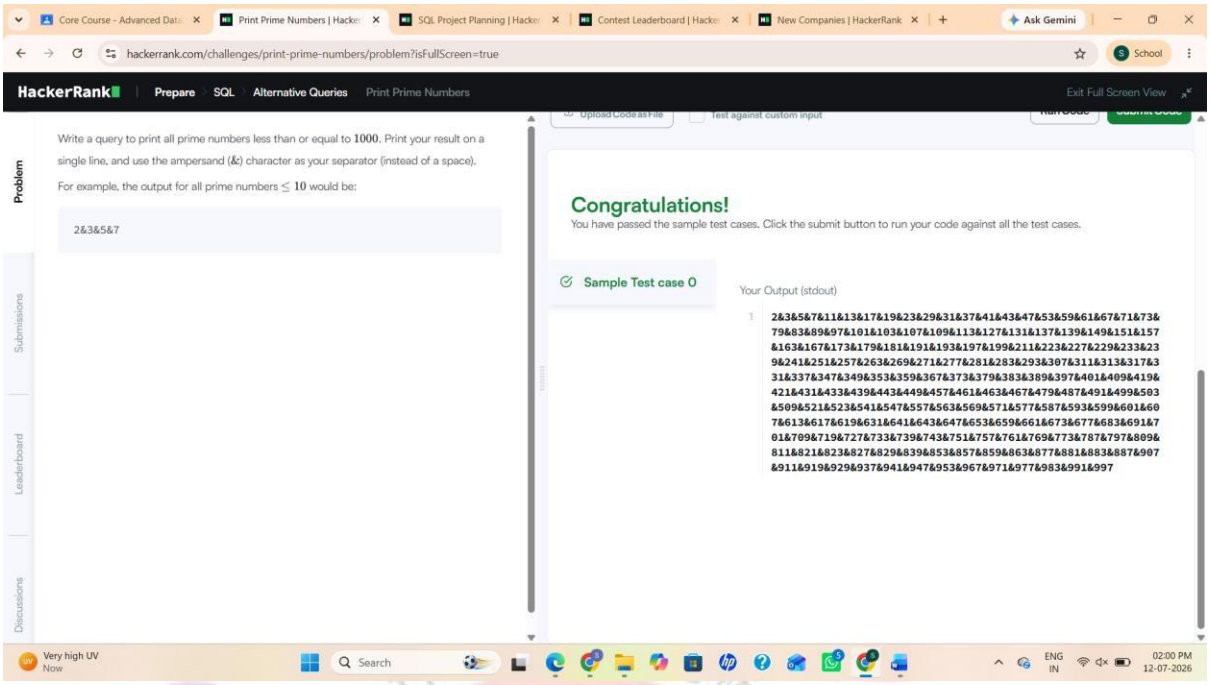
54. PRINT PRIME NUMBERS

PROGRAM

```
SET @num = 1;


SELECT GROUP_CONCAT(n SEPARATOR '&') FROM (
   SELECT @num := @num + 1 AS n FROM
   information_schema.tables t1,
       information_schema.tables t2 LIMIT
   998
) numbers
WHERE NOT EXISTS ( SELECT
   1
   FROM (
      SELECT @div := @div + 1 AS d FROM
      information_schema.tables,
          (SELECT @div := 1) init LIMIT 998
   ) divisors WHERE d < n
   AND d > 1
    AND MOD(n, d) = 0
);
```

OUTPUT


Print Prime Numbers | Hacker
hackerrank.com/challenges/print-prime-numbers/problem?isFullScreen=true
HackerRank
Prepare
SQL
Alternative Queries
Print Prime Numbers
Exit Full Screen View
Problem
Write a query to print all prime numbers less than or equal to 1000. Print your result on a single line, and use the ampersand (&) character as your separator (instead of a space).
For example, the output for all prime numbers ≤ 10 would be:
2&3&5&7
Congratulations!
You have passed the sample test cases. Click the submit button to run your code against all the test cases.
Sample Test case 0
Your Output (stdout)
2&3&5&7&11&13&17&19&23&29&31&37&41&43&47&53&59&61&67&71&73&79&83&89&97&101&103&107&109&113&127&131&137&139&149&151&157&163&167&173&179&181&191&193&197&199&211&223&227&229&233&239&241&251&257&263&269&271&277&281&283&293&307&311&313&317&331&337&347&349&353&359&367&373&379&383&389&397&401&409&419&421&431&433&439&443&449&457&461&463&467&479&487&491&499&503&509&521&523&541&547&557&563&569&571&577&587&593&599&601&607&613&617&619&631&641&643&647&653&659&661&673&677&683&691&701&709&719&727&733&739&743&751&757&761&769&773&787&797&809&811&821&823&827&829&839&853&857&859&863&877&881&883&887&907&911&919&929&937&941&947&953&967&971&977&983&991&997
Submissions
Leaderboard
Discussions
Very high UV
Now
Search
ENG
IN
02:00 PM
12-07-2026

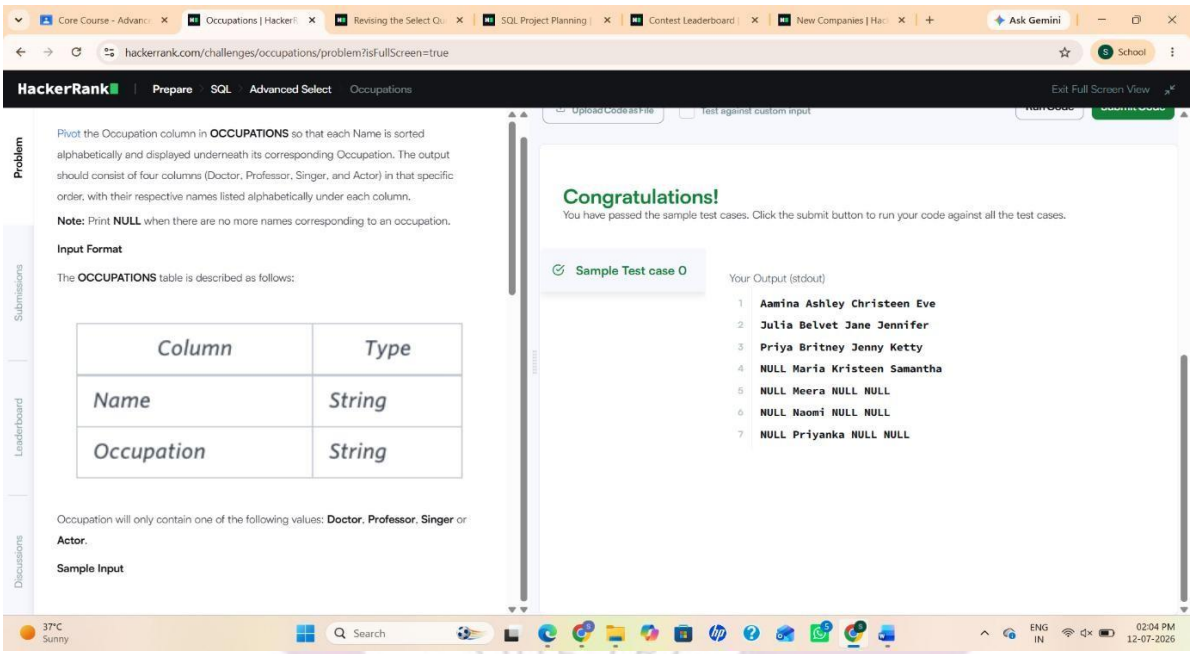
55. OCCUPATIONS

PROGRAM

```
SET @d = 0, @p = 0, @s = 0, @a = 0;


SELECT
   MAX(Doctor), MAX(Professor),
   MAX(Singer), MAX(Actor)
FROM (
   SELECT CASE
         WHEN Occupation = 'Doctor' THEN (@d := @d + 1) WHEN
         Occupation = 'Professor' THEN (@p := @p + 1) WHEN Occupation =
         'Singer' THEN (@s := @s + 1) WHEN Occupation = 'Actor' THEN
         (@a := @a + 1)
      END AS rn,
      CASE WHEN Occupation = 'Doctor' THEN Name END AS Doctor, CASE WHEN
      Occupation = 'Professor' THEN Name END AS Professor, CASE WHEN
      Occupation = 'Singer' THEN Name END AS Singer, CASE WHEN Occupation =
      'Actor' THEN Name END AS Actor
   FROM OCCUPATIONS
   ORDER BY Occupation, Name
) t
GROUP BY rn ORDER BY
rn;
```

OUTPUT



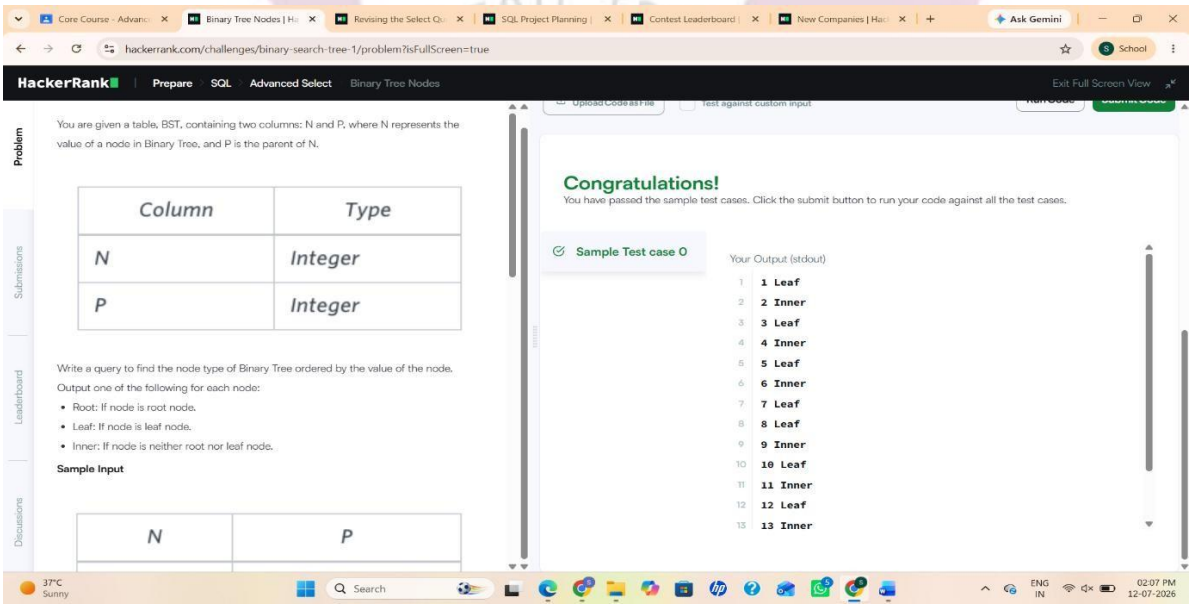
TRUTH-TUTE-TRIUMPH
AUTONOMOUS

56. BINARY TREE NODES

PROGRAM

```
SELECT N, CASE
    WHEN P IS NULL THEN 'Root'
    WHEN N NOT IN (SELECT DISTINCT P FROM BST WHERE P IS NOT NULL) THEN 'Leaf'
    ELSE 'Inner' END AS
  NodeType
FROM BST ORDER BY N;
```

OUTPUT

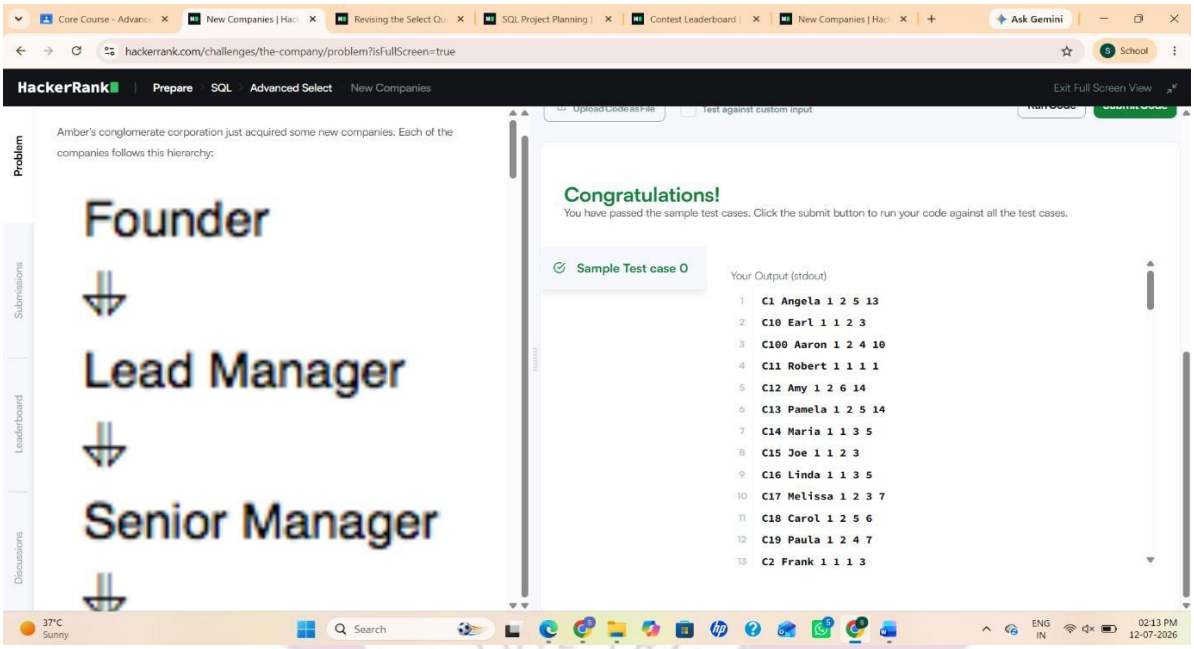


57. NEW COMPANIES

PROGRAM

```
SELECT
    c.company_code, c.founder,
    COUNT(DISTINCT lm.lead_manager_code),
    COUNT(DISTINCT sm.senior_manager_code),
    COUNT(DISTINCT m.manager_code),
    COUNT(DISTINCT e.employee_code)
FROM Company c
LEFT JOIN Lead_Manager lm
    ON c.company_code = lm.company_code LEFT JOIN
Senior_Manager sm
    ON c.company_code = sm.company_code LEFT JOIN
Manager m
    ON c.company_code = m.company_code LEFT JOIN
Employee e
    ON c.company_code = e.company_code GROUP BY
    c.company_code, c.founder
ORDER BY
    c.company_code;
```

OUTPUT

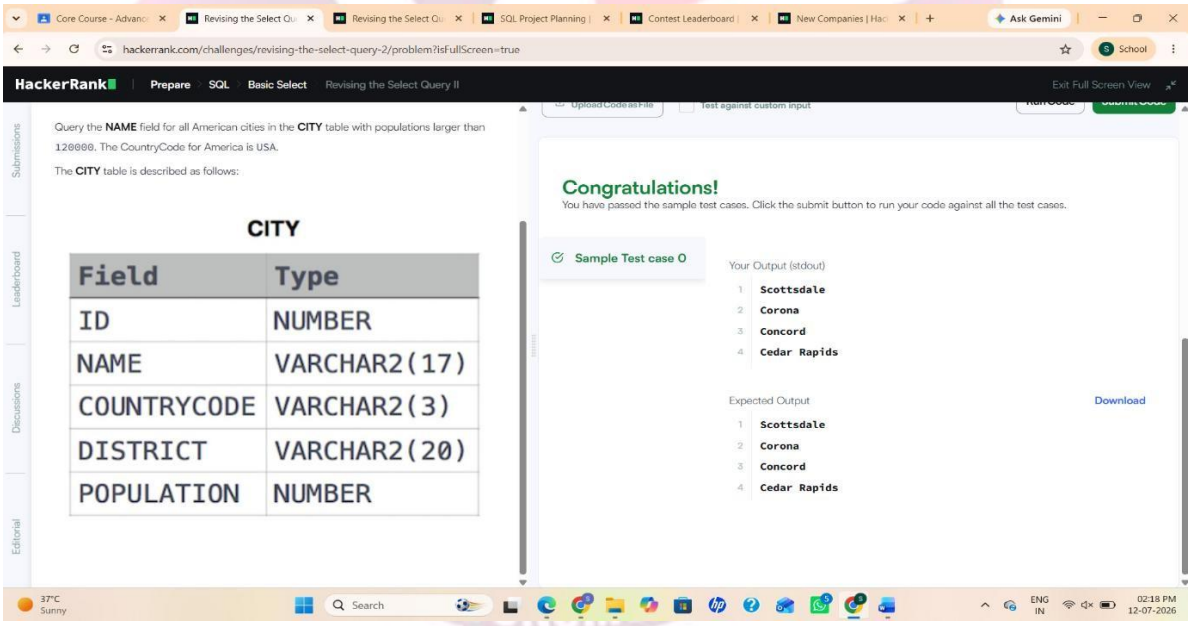

HackerRank
Prepare
SQL
Advanced Select
New Companies
Exit Full Screen View
Problem
Amber's conglomerate corporation just acquired some new companies. Each of the companies follows this hierarchy:
Founder
Lead Manager
Senior Manager
Congratulations!
You have passed the sample test cases. Click the submit button to run your code against all the test cases.
Sample Test case 0
Your Output (stdout)
C1 Angela 1 2 5 13
C10 Earl 1 1 2 3
C100 Aaron 1 2 4 10
C11 Robert 1 1 1 1
C12 Amy 1 2 6 14
C13 Pamela 1 2 5 14
C14 Maria 1 1 3 5
C15 Joe 1 1 2 3
C16 Linda 1 1 3 5
C17 Melissa 1 2 3 7
C18 Carol 1 2 5 6
C19 Paula 1 2 4 7
C2 Frank 1 1 1 3

58. REVISING THE SELECT QUERY II

PROGRAM

```
SELECT NAME
FROM CITY
WHERE COUNTRYCODE = 'USA' AND
POPULATION > 120000;
```

OUTPUT



SQL(BASIC)



Certificate of Accomplishment

SQL (Basic)

PRESENTED TO

SUKAINA FIRDOUS J

The bearer of this certificate has passed the HackerRank skill certification test

Earned on: **12 Jul, 2026**

ID: 14A205F3C3DC

Harishankaran K

CTO, HackerRank

SQL(INTERMEDIATE)



Certificate of Accomplishment

SQL (Intermediate)

PRESENTED TO

SUKAINA FIRDOUS J

The bearer of this certificate has passed the HackerRank skill certification test

Earned on: **12 Jul, 2026**

ID: C163839881AC

Harishankaran K

CTO, HackerRank

AUTONOMOUS

SQL(ADVANCED)



Certificate of Accomplishment

SQL (Advanced)

PRESENTED TO

SUKAINA FIRDOUS J

The bearer of this certificate has passed the HackerRank skill certification test

Earned on: **12 Jul, 2026**

ID: 8C3ADE9E45BF

Harishankaran K

CTO, HackerRank

AUTONOMOUS

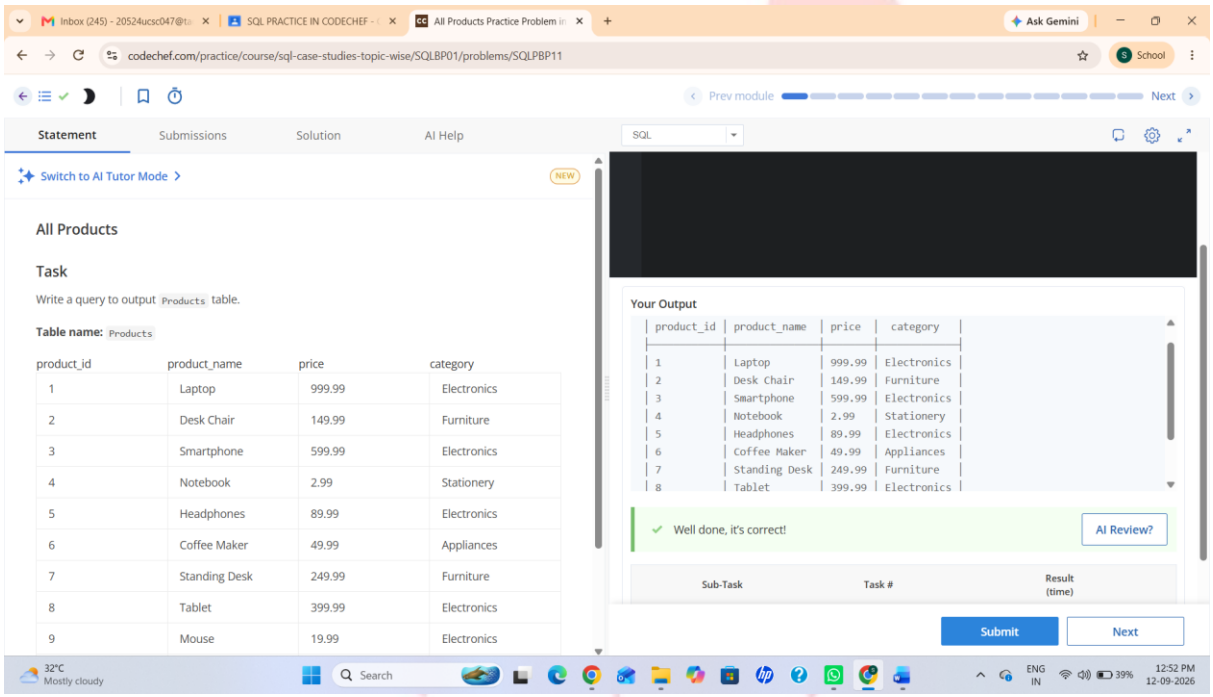
CODECHEF

1.ALL PRODUCTS

QUERY

```
SELECT * FROM Products;
```

OUTPUT

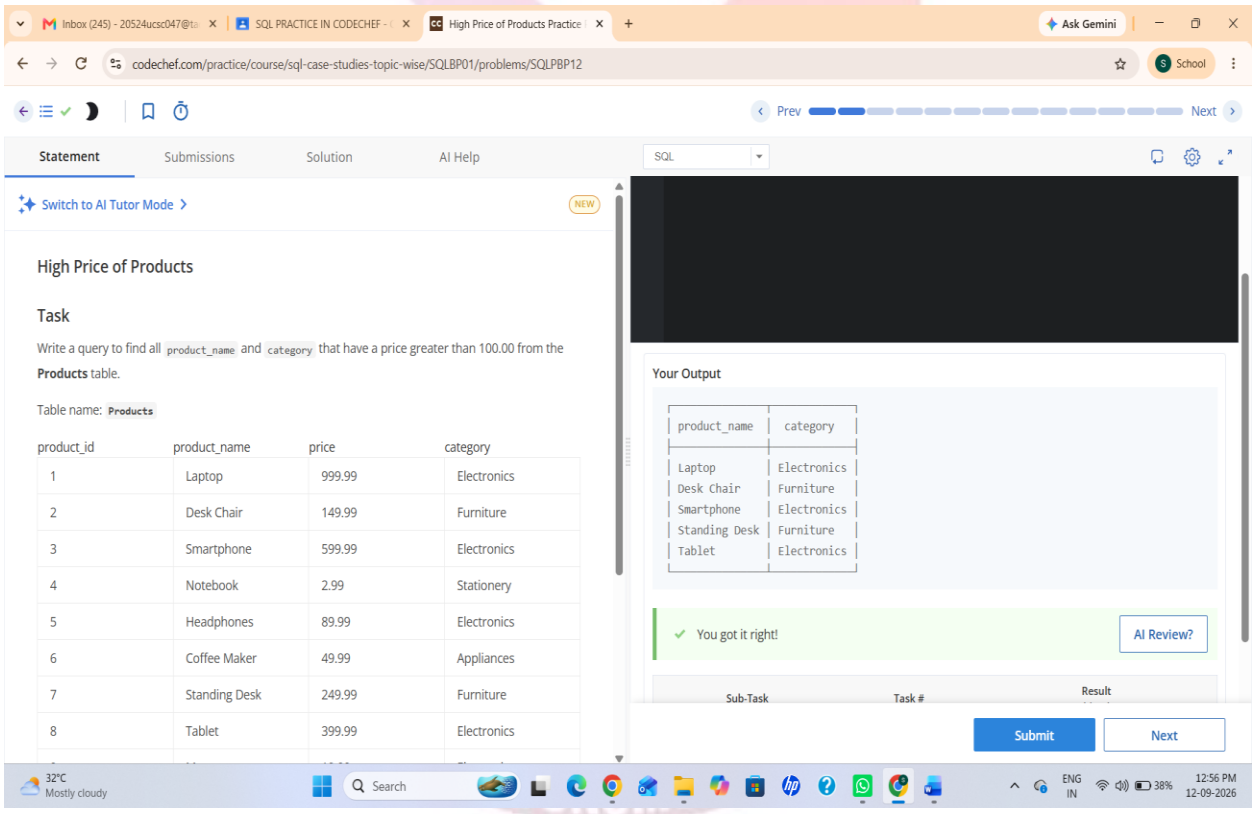


2.HIGH PRICE OF PRODUCTS

QUERY

```
SELECT product_name, category
FROM Products
WHERE price > 100.00;
```

OUTPUT

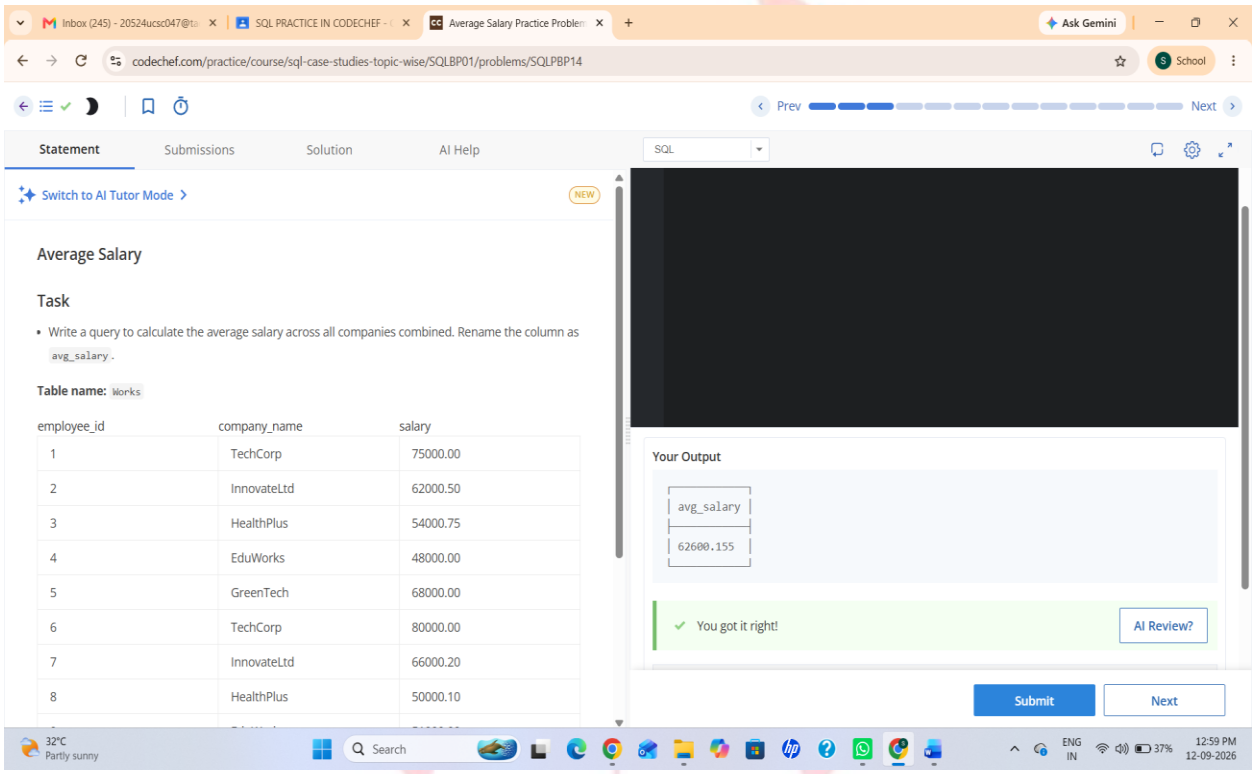


3. AVERAGE SALARY

QUERY

```
SELECT AVG(salary) AS avg_salary
FROM Works;
```

OUTPUT

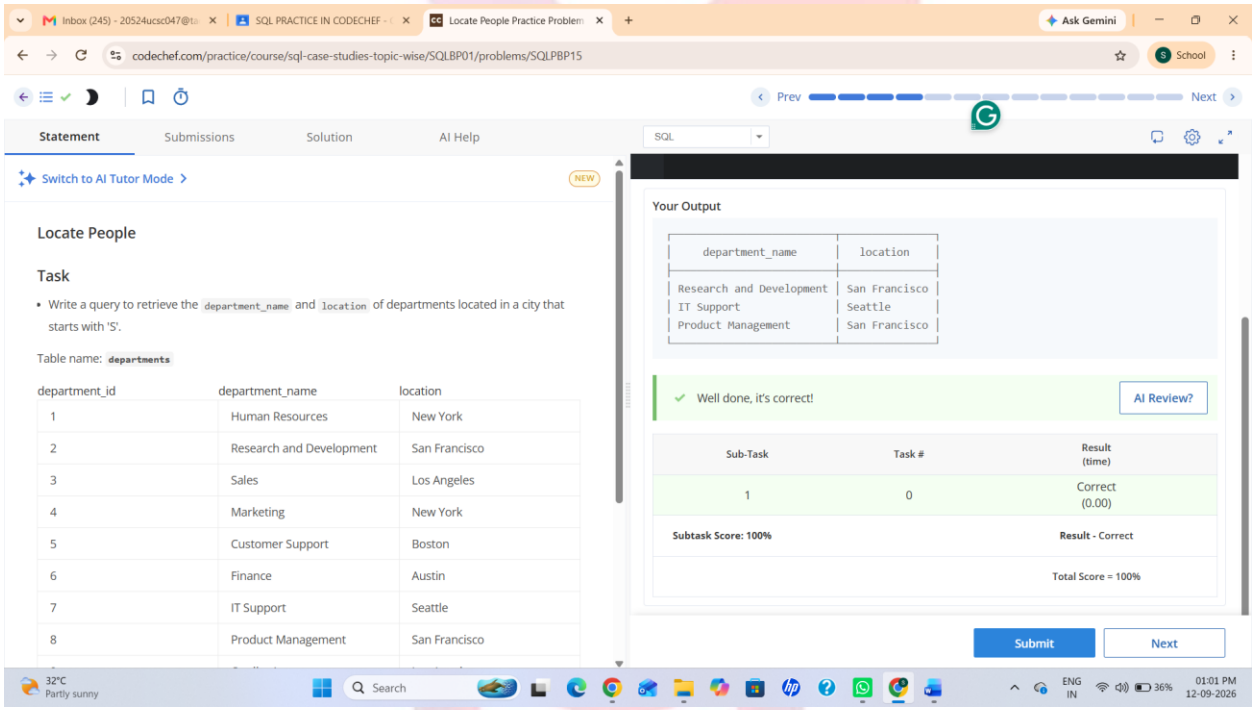


4.LOCATE PEOPLE

QUERY

```
SELECT department_name, location
FROM departments
WHERE location LIKE 'S%';
```

OUTPUT

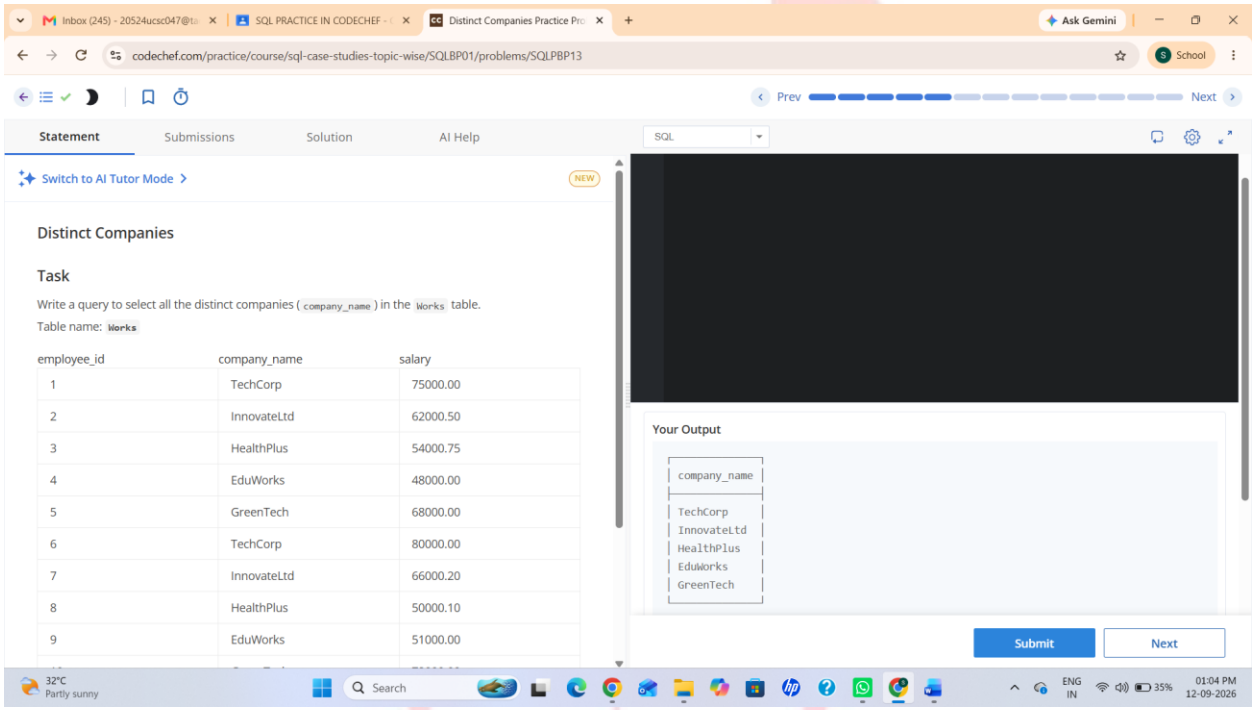


5. DISTINCT COMPANIES

QUERY

```
SELECT DISTINCT company_name
FROM Works;
```

OUTPUT



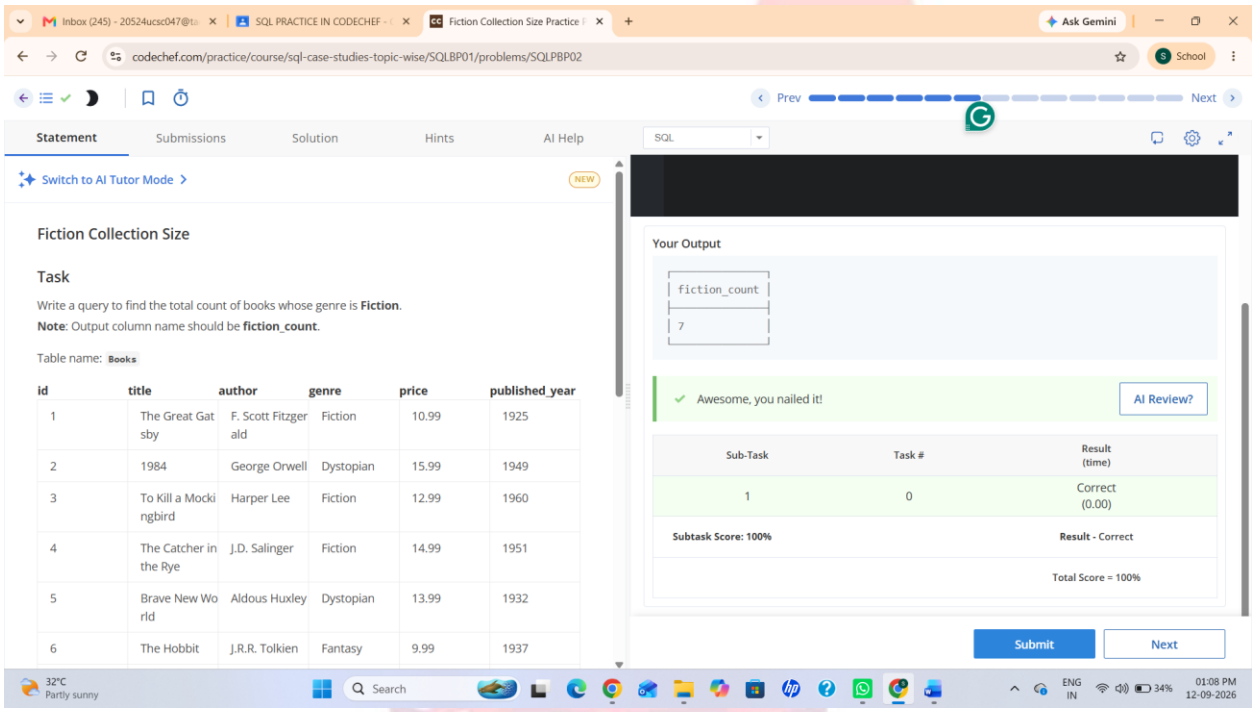
AUTONOMOUS

6.FICTION COLLECTION SIZE

QUERY

```
SELECT COUNT(*) AS fiction_count
FROM Books
WHERE genre = 'Fiction';
```

OUTPUT

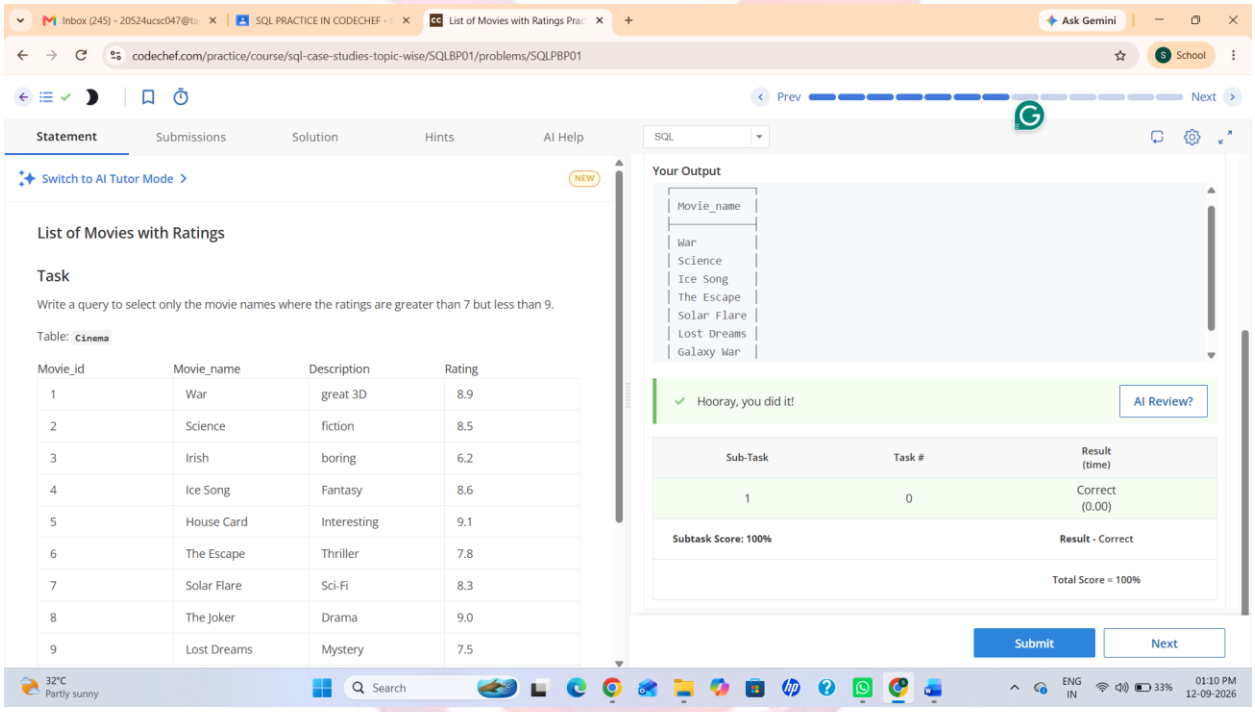


7.LIST OF MOVIES WITH RATINGS

QUERY

```
SELECT Movie_name
FROM Cinema
WHERE Rating > 7 AND Rating < 9;
```

OUTPUT

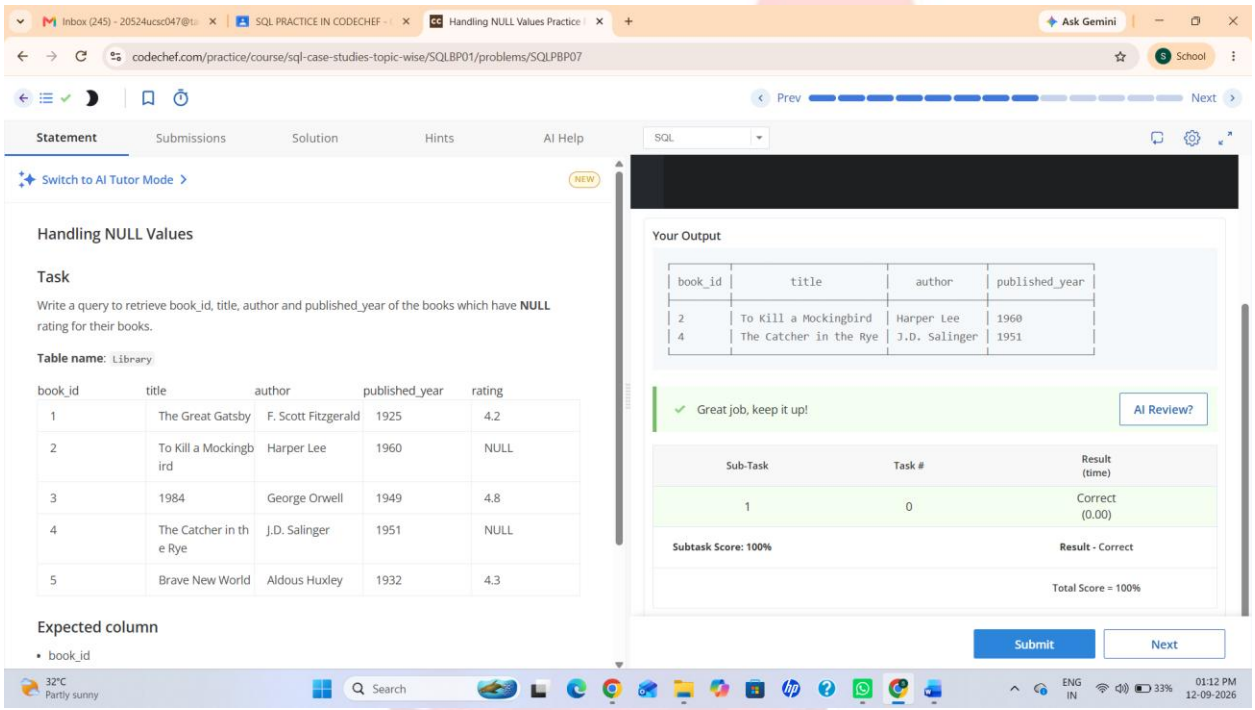


8.HANDLING NULL VALUES

QUERY

```
SELECT book_id, title, author, published_year
FROM Library
WHERE rating IS NULL;
```

OUTPUT

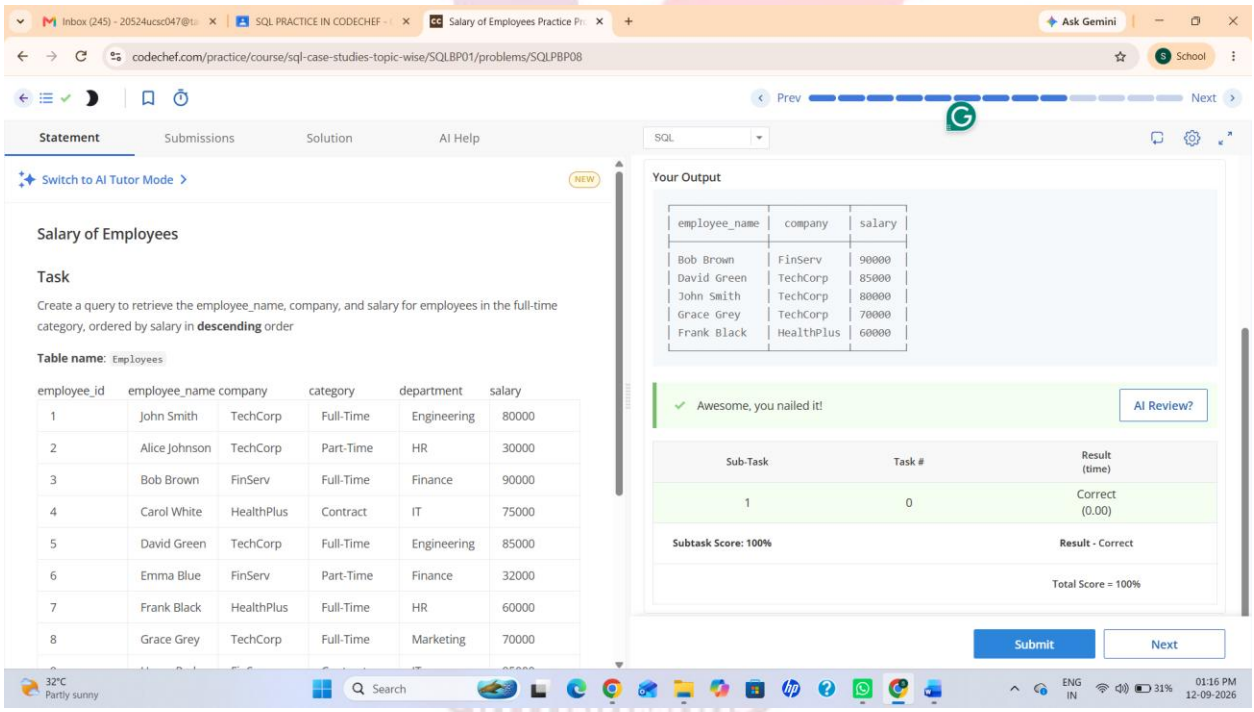


9.SALARY OF EMPLOYEES

QUERY

```
SELECT employee_name, company, salary
FROM Employees
WHERE category = 'Full-Time'
ORDER BY salary DESC;
```

OUTPUT

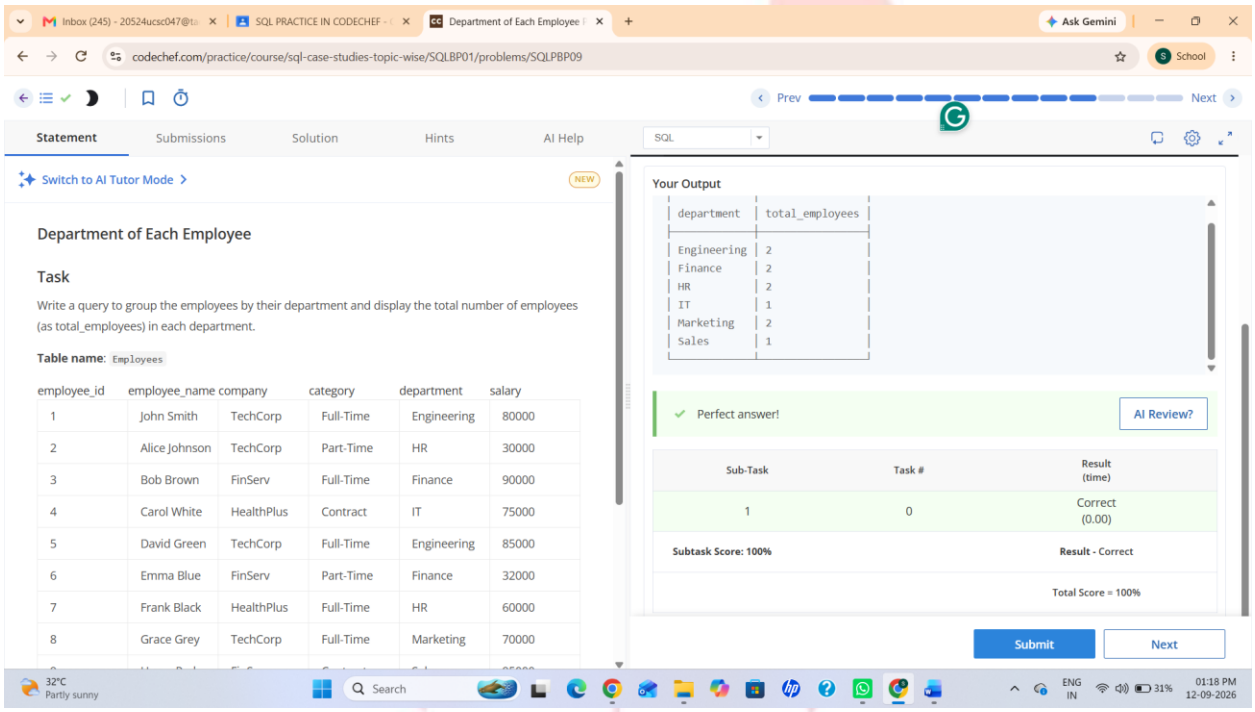


10. DEPARTMENT OF EACH EMPLOYEE

QUERY

```
SELECT department, COUNT(*) AS total_employees
FROM Employees
GROUP BY department;
```

OUTPUT

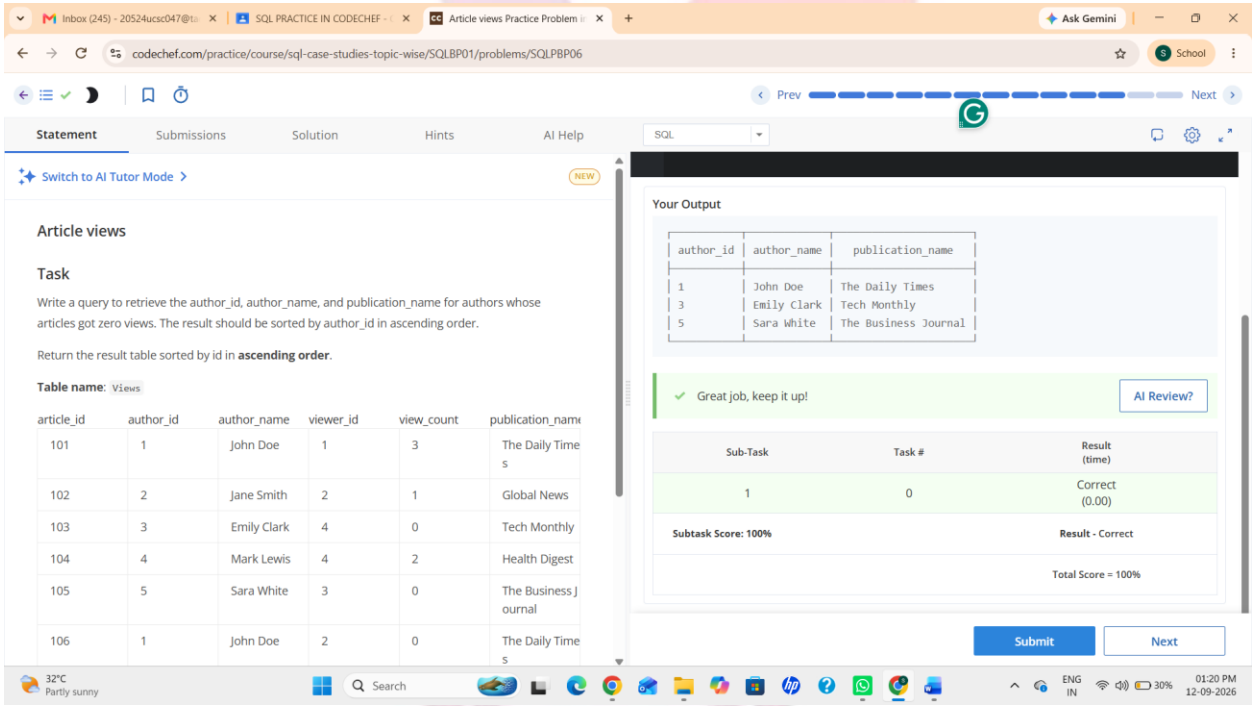


11.ARTICLE VIEWS

QUERY

```
SELECT author_id, author_name, publication_name
FROM Views
WHERE view_count = 0
ORDER BY author_id ASC;
```

OUTPUT

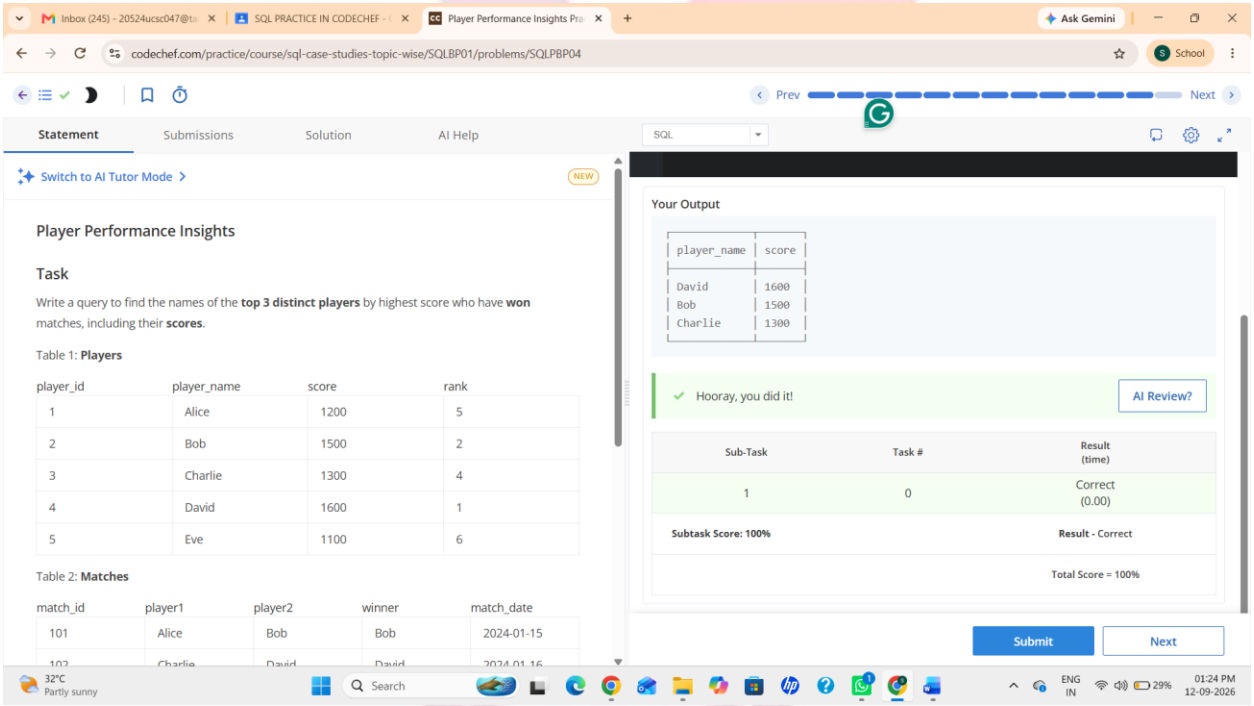


12.PLAYER PERFORMANCE INSIGHTS

QUERY

```
SELECT DISTINCT p.player_name, p.score
FROM Players p
JOIN Matches m
ON p.player_name = m.winner
ORDER BY p.score DESC
LIMIT 3;
```

OUTPUT

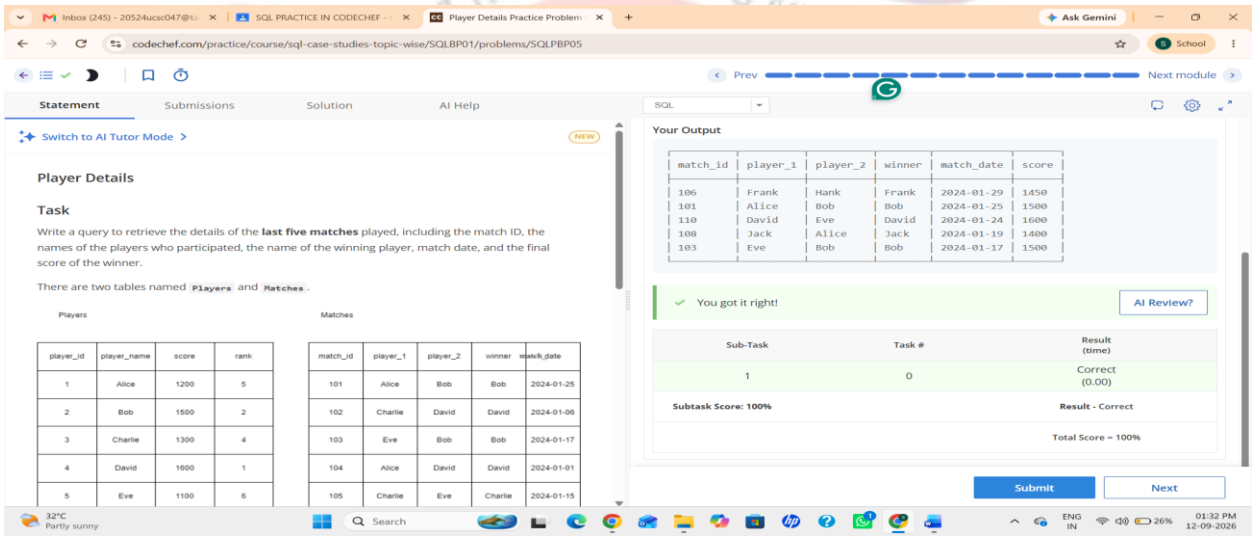


13.PLAYER DETAILS

QUERY

```
SELECT m.match_id,
    m.player_1,
    m.player_2,
    m.winner,
    m.match_date,
    p.score
FROM Matches m
JOIN Players p
ON m.winner = p.player_name
ORDER BY m.match_date DESC
LIMIT 5;
```

OUTPUT



EX. NO:13 **MINI PROJECT**

DATE: **SUPERMARKET SALES AND BILLING MANAGEMENT**

AIM

To develop a Supermarket Sales and Billing Management system using SQL and PL/SQL to store, manage, process, calculate customer bills, and analyse product-wise sales revenue.

Software Used

- GitHub Codespaces
- SQLite
- Oracle Database
- SQL
- PL/SQL

Description

The Supermarket Sales and Billing Management system is developed to manage customer purchases and billing details.
It stores invoice number, customer type, product line, quantity, unit price, tax, and total amount.
SQL and PL/SQL are used to analyze sales, generate bills, classify purchases, and handle exceptions.

Database Tables

1.SUPERMARKET_SALES

Column Name	Data Type	Constraint
INVOICE_NUMBER	VARCHAR2(10)	PRIMARY KEY
CUSTOMER_TYPE	VARCHAR2(20)	NOT NULL
PRODUCT_LINE	VARCHAR2(50)	NOT NULL
QUANTITY	NUMBER(5)	NOT NULL
UNIT_PRICE	NUMBER(10,2)	NOT NULL

TOTAL_AMOUNT	NUMBER(12,2)	NOT NULL
TAX_AMOUNT	NUMBER(12,2)	NOT NULL
FINAL_AMOUNT	NUMBER(12,2)	NOT NULL

TABLE CREATION

```
CREATE TABLE supermarket_sales (
    invoice_number VARCHAR2(10) PRIMARY KEY,
    customer_type VARCHAR2(20) NOT NULL,
    product_line VARCHAR2(50) NOT NULL,
    quantity NUMBER(5) NOT NULL,
    unit_price NUMBER(10,2) NOT NULL,
    total_amount NUMBER(12,2) NOT NULL,
    tax_amount NUMBER(12,2) NOT NULL,
    final_amount NUMBER(12,2) NOT NULL,
    CONSTRAINT chk_quantity CHECK (quantity > 0),
    CONSTRAINT chk_unit_price CHECK (unit_price > 0),
    CONSTRAINT chk_total_amount CHECK (total_amount >= 0),
    CONSTRAINT chk_tax_amount CHECK (tax_amount >= 0),
    CONSTRAINT chk_final_amount CHECK (final_amount >= 0)
);
```

Table Created

The SUPERMARKET_SALES table was created successfully with primary key, NOT NULL constraints, and CHECK constraints.

DATASET INSERTION

```
INSERT INTO supermarket_sales VALUES
('INV001','Member','Health and beauty',5,500,2500,125,2625);
```

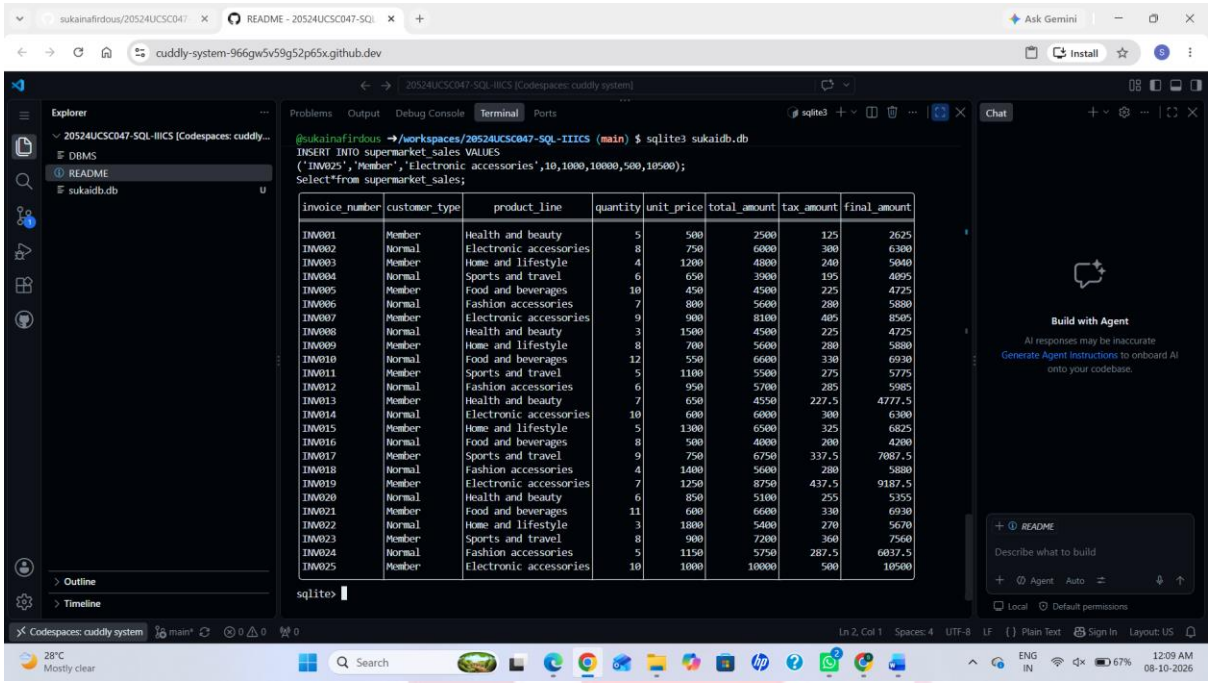
```
INSERT INTO supermarket_sales VALUES
('INV002','Normal','Electronic accessories',8,750,6000,300,6300);
INSERT INTO supermarket_sales VALUES
('INV003','Member','Home and lifestyle',4,1200,4800,240,5040);
INSERT INTO supermarket_sales VALUES
('INV004','Normal','Sports and travel',6,650,3900,195,4095);
INSERT INTO supermarket_sales VALUES
('INV005','Member','Food and beverages',10,450,4500,225,4725);
INSERT INTO supermarket_sales VALUES
('INV006','Normal','Fashion accessories',7,800,5600,280,5880);
INSERT INTO supermarket_sales VALUES
('INV007','Member','Electronic accessories',9,900,8100,405,8505);
INSERT INTO supermarket_sales VALUES
('INV008','Normal','Health and beauty',3,1500,4500,225,4725);
INSERT INTO supermarket_sales VALUES
('INV009','Member','Home and lifestyle',8,700,5600,280,5880);
INSERT INTO supermarket_sales VALUES
('INV010','Normal','Food and beverages',12,550,6600,330,6930);
INSERT INTO supermarket_sales VALUES
('INV011','Member','Sports and travel',5,1100,5500,275,5775);
INSERT INTO supermarket_sales VALUES
('INV012','Normal','Fashion accessories',6,950,5700,285,5985);
INSERT INTO supermarket_sales VALUES
('INV013','Member','Health and beauty',7,650,4550,227.50,4777.50);
INSERT INTO supermarket_sales VALUES
('INV014','Normal','Electronic accessories',10,600,6000,300,6300);
```

```
INSERT INTO supermarket_sales VALUES
('INV015','Member','Home and lifestyle',5,1300,6500,325,6825);
INSERT INTO supermarket_sales VALUES
('INV016','Normal','Food and beverages',8,500,4000,200,4200);
INSERT INTO supermarket_sales VALUES
('INV017','Member','Sports and travel',9,750,6750,337.50,7087.50);
INSERT INTO supermarket_sales VALUES
('INV018','Normal','Fashion accessories',4,1400,5600,280,5880);
INSERT INTO supermarket_sales VALUES
('INV019','Member','Electronic accessories',7,1250,8750,437.50,9187.50);
INSERT INTO supermarket_sales VALUES
('INV020','Normal','Health and beauty',6,850,5100,255,5355);
INSERT INTO supermarket_sales VALUES
('INV021','Member','Food and beverages',11,600,6600,330,6930);
INSERT INTO supermarket_sales VALUES
('INV022','Normal','Home and lifestyle',3,1800,5400,270,5670);
INSERT INTO supermarket_sales VALUES
('INV023','Member','Sports and travel',8,900,7200,360,7560);
INSERT INTO supermarket_sales VALUES
('INV024','Normal','Fashion accessories',5,1150,5750,287.50,6037.50);
INSERT INTO supermarket_sales VALUES
('INV025','Member','Electronic accessories',10,1000,10000,500,10500);
Select*from supermarket_sales;
COMMIT;
```

Records Inserted

A total of **25 supermarket sales records** were inserted into the SUPERMARKET_SALES table.

OUTPUT

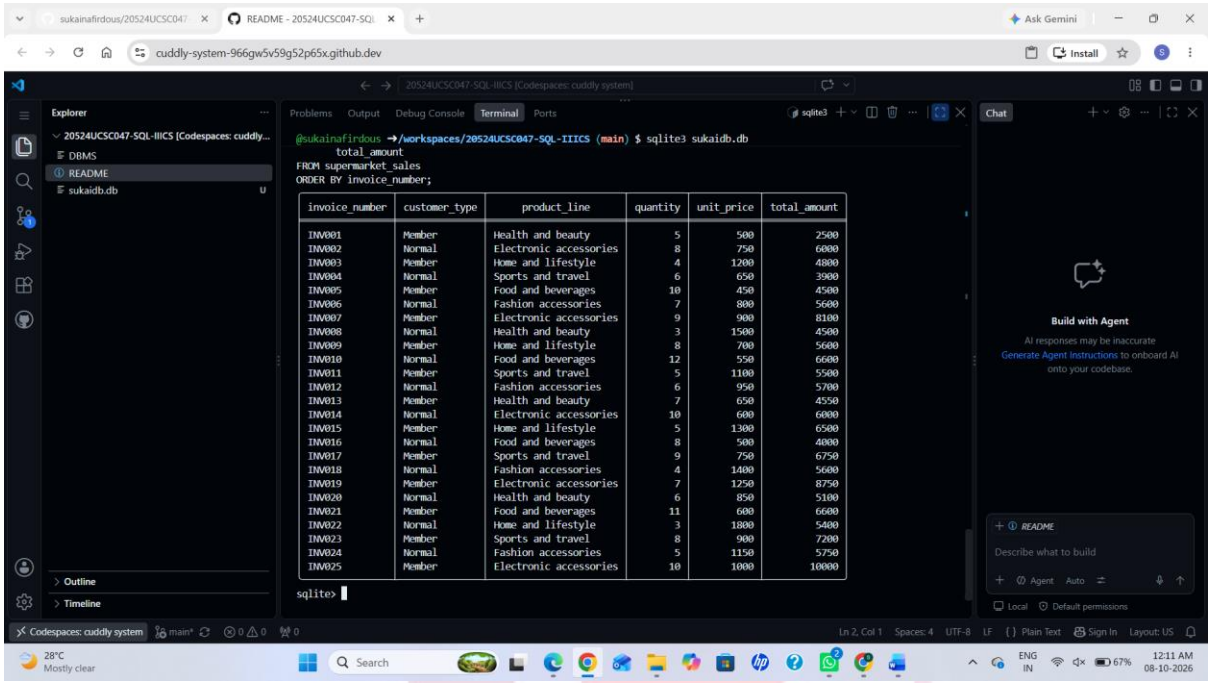


SQL QUERY 1: Display Sales Transaction Records

Query

```
SELECT invoice_number,
    customer_type,
    product_line,
    quantity,
    unit_price,
    total_amount
FROM supermarket_sales
ORDER BY invoice_number;
```

OUTPUT



SQL QUERY 2: Total Sales Revenue for Each Product Line

Query

```
SELECT product_line,
    SUM(final_amount) AS total_sales_revenue
FROM supermarket_sales
GROUP BY product_line
ORDER BY product_line;
```

OUTPUT

```
sqlite> SELECT product_line,
       SUM(final_amount) AS total_sales_revenue
FROM supermarket_sales
GROUP BY product_line
ORDER BY product_line;
```

product_line	total_sales_revenue
Electronic accessories	40792.5
Fashion accessories	23782.5
Food and beverages	22785
Health and beauty	17482.5
Home and lifestyle	23415
Sports and travel	24517.5

SQL QUERY 3: Five Highest Purchase Amounts

Query

```
SELECT invoice_number,
    customer_type,
    product_line,
    final_amount
FROM supermarket_sales
ORDER BY final_amount DESC
LIMIT 5;
```

OUTPUT

```
sqlite> SELECT invoice_number,
       customer_type,
       product_line,
       final_amount
FROM supermarket_sales
ORDER BY final_amount DESC
LIMIT 5;
```

invoice_number	customer_type	product_line	final_amount
INV025	Member	Electronic accessories	10500
INV019	Member	Electronic accessories	9187.5
INV007	Member	Electronic accessories	8505
INV023	Member	Sports and travel	7560
INV017	Member	Sports and travel	7087.5

SQL QUERY 4: Average Purchase Amount for Each Customer Type

Query

```
SELECT customer_type,
    ROUND(AVG(final_amount),2) AS average_purchase_amount
FROM supermarket_sales
GROUP BY customer_type
ORDER BY customer_type;
```

OUTPUT

```
sqlite> SELECT customer_type,
       ROUND(AVG(final_amount),2) AS average_purchase_amount
FROM supermarket_sales
GROUP BY customer_type
ORDER BY customer_type;
```

customer_type	average_purchase_...
Member	6570.58
Normal	5613.13

SQL QUERY 5: Display Invoices Exceeding ₹5,000

Query

SELECT invoice_number,

customer_type,

product_line,

final_amount

FROM supermarket_sales

WHERE final_amount > 5000

ORDER BY final_amount DESC;

OUTPUT

```
sqlite> SELECT invoice_number,
       customer_type,
       product_line,
       final_amount
FROM supermarket_sales
WHERE final_amount > 5000
ORDER BY final_amount DESC;
```

invoice_number	customer_type	product_line	final_amount
INV025	Member	Electronic accessories	10500
INV019	Member	Electronic accessories	9187.5
INV007	Member	Electronic accessories	8505
INV023	Member	Sports and travel	7560
INV017	Member	Sports and travel	7087.5
INV010	Normal	Food and beverages	6930
INV021	Member	Food and beverages	6930
INV015	Member	Home and lifestyle	6825
INV002	Normal	Electronic accessories	6300
INV014	Normal	Electronic accessories	6300
INV024	Normal	Fashion accessories	6037.5
INV012	Normal	Fashion accessories	5985
INV006	Normal	Fashion accessories	5880
INV009	Member	Home and lifestyle	5880
INV018	Normal	Fashion accessories	5880
INV011	Member	Sports and travel	5775
INV022	Normal	Home and lifestyle	5670
INV020	Normal	Health and beauty	5355
INV003	Member	Home and lifestyle	5040

PL/SQL PROCEDURE 1: Generate Customer Bill

This procedure accepts an invoice number and displays the quantity, unit price, tax amount and final bill amount.

PROGRAM

```
SET SERVEROUTPUT ON;


CREATE OR REPLACE PROCEDURE generate_bill
(
   p_invoice_number IN VARCHAR2
)
IS
   v_customer_type supermarket_sales.customer_type%TYPE;
   v_product_line  supermarket_sales.product_line%TYPE;
   v_quantity      supermarket_sales.quantity%TYPE;
   v_unit_price    supermarket_sales.unit_price%TYPE;
   v_tax_amount    supermarket_sales.tax_amount%TYPE;
   v_final_amount  supermarket_sales.final_amount%TYPE;
BEGIN

   SELECT customer_type,
          product_line,
          quantity,
          unit_price,
          tax_amount,
          final_amount
   INTO   v_customer_type,
          v_product_line,
```

```
        v_quantity,
        v_unit_price,
        v_tax_amount,
        v_final_amount
    FROM supermarket_sales
    WHERE invoice_number = p_invoice_number;

    DBMS_OUTPUT.PUT_LINE('----- CUSTOMER BILL -----');
    DBMS_OUTPUT.PUT_LINE('Invoice Number : ' || p_invoice_number);
    DBMS_OUTPUT.PUT_LINE('Customer Type  : ' || v_customer_type);
    DBMS_OUTPUT.PUT_LINE('Product Line   : ' || v_product_line);
    DBMS_OUTPUT.PUT_LINE('Quantity       : ' || v_quantity);
    DBMS_OUTPUT.PUT_LINE('Unit Price     : ' || v_unit_price);

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Error: Invoice Number Not Found.');

    WHEN TOO_MANY_ROWS THEN
        DBMS_OUTPUT.PUT_LINE('Error: Multiple records found.');

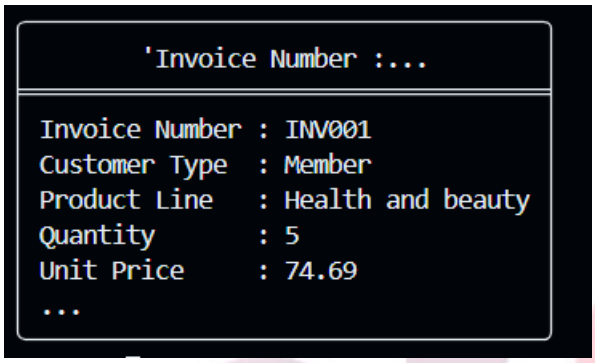
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);

END generate_bill;
/
```

EXECUTION

```
BEGIN
   generate_bill('INV001');
END;
/
```

OUTPUT



PL/SQL FUNCTION 2: CLASSIFY PURCHASE

PROGRAM

```
CREATE OR REPLACE FUNCTION classify_purchase
(
   p_final_amount IN NUMBER
)
RETURN VARCHAR2
IS
BEGIN

   IF p_final_amount IS NULL THEN

      RAISE_APPLICATION_ERROR(
         -20001,
         'Purchase amount cannot be NULL'
```

```
        );

    ELSIF p_final_amount < 0 THEN

        RAISE_APPLICATION_ERROR(
            -20002,
            'Invalid Purchase Amount'
        );

    ELSIF p_final_amount >= 5000 THEN

        RETURN 'Premium Purchase';

    ELSE

        RETURN 'Regular Purchase';

    END IF;

END classify_purchase;
/
SELECT invoice_number,
       final_amount,
       classify_purchase(final_amount) AS purchase_category
FROM supermarket_sales
WHERE invoice_number IN ('INV001')
ORDER BY invoice_number;
```

OUTPUT

invoice_number	final_amount	purchase_category
INV001	78.42	Regular Purchase

PL/SQL PROCEDURE 3: GENERATE SALES SUMMARY

PROGRAM

```
CREATE OR REPLACE PROCEDURE sales_summary
IS
BEGIN

  DBMS_OUTPUT.PUT_LINE(
    '----- PRODUCT LINE SALES SUMMARY -----'
  );

  FOR r IN
  (
    SELECT product_line,
        COUNT(*) AS total_invoices,
        SUM(final_amount) AS total_sales
    FROM supermarket_sales
    GROUP BY product_line
    ORDER BY product_line
  )
  LOOP

    DBMS_OUTPUT.PUT_LINE(
      'Product Line: ' || r.product_line ||
```

```
            ' | Invoices: ' || r.total_invoices ||
            ' | Total Sales: ' || r.total_sales
        );

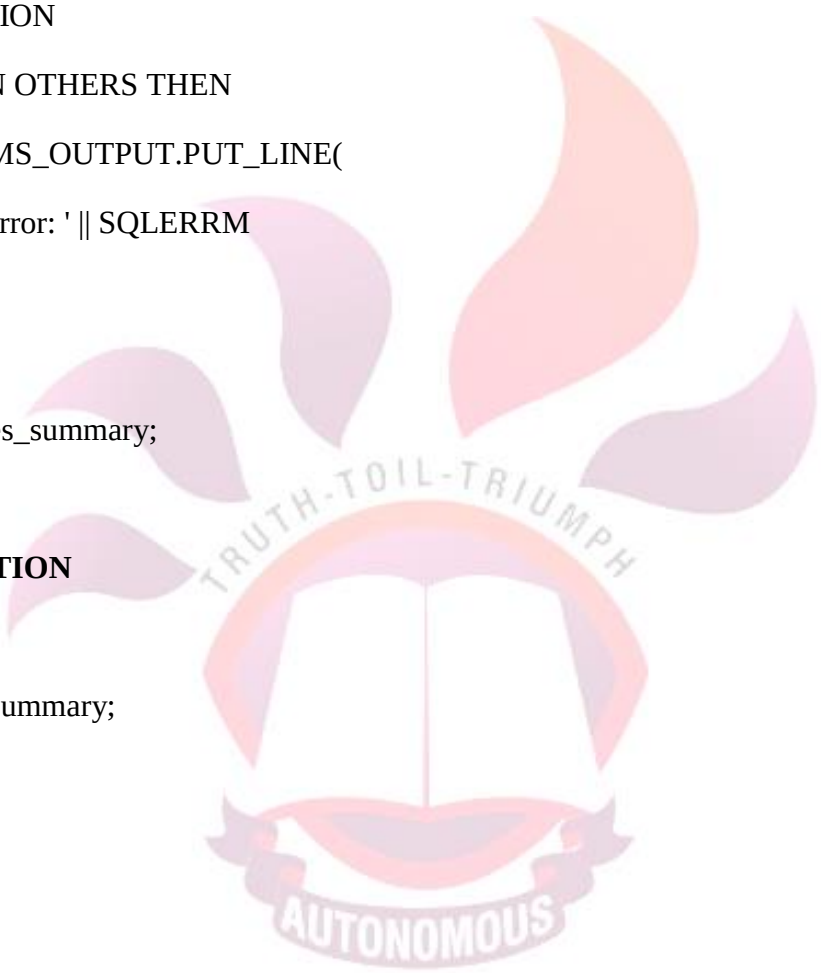
    END LOOP;

EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE(
            'Error: ' || SQLERRM
        );

END sales_summary;
/
```

EXECUTION

```
BEGIN
    sales_summary;
END;
/
```



OUTPUT

```
----- PRODUCT LINE SALES SUMMARY -----

Product Line: Electronic accessories
Invoices: 5
Total Sales: 39292.5
----- PRODUCT LINE SALES SUMMARY -----

Product Line: Fashion accessories
Invoices: 4
Total Sales: 18782.5
----- PRODUCT LINE SALES SUMMARY -----

Product Line: Food and beverages
Invoices: 4
Total Sales: 17805.0
----- PRODUCT LINE SALES SUMMARY -----

Product Line: Health and beauty
Invoices: 4
Total Sales: 17482.5
----- PRODUCT LINE SALES SUMMARY -----

Product Line: Home and lifestyle
Invoices: 4
Total Sales: 17415.0
----- PRODUCT LINE SALES SUMMARY -----

Product Line: Sports and travel
Invoices: 4
Total Sales: 18717.5
```

EXCEPTION HANDLING

Test Case 1 – Invalid Invoice Number

EXECUTION

```
CREATE TABLE invoices (
    Invoice_ID TEXT PRIMARY KEY,
    Customer_Type TEXT,
    Product_Line TEXT,
    Quantity INTEGER,
    Unit_Price REAL,
    Tax_Amount REAL,
```

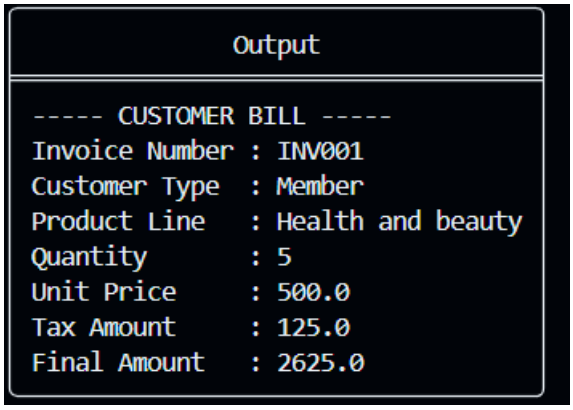
```
    Total_Amount REAL
);


INSERT INTO invoices VALUES ('INV001', 'Member', 'Health and beauty', 5, 500, 125,
2625);
SELECT '----- CUSTOMER BILL -----' AS Output
UNION ALL
SELECT 'Invoice Number : ' || Invoice_ID FROM invoices WHERE Invoice_ID = 'INV001'
UNION ALL
SELECT 'Customer Type  : ' || Customer_Type FROM invoices WHERE Invoice_ID =
'INV001'
UNION ALL
SELECT 'Product Line   : ' || Product_Line FROM invoices WHERE Invoice_ID = 'INV001'
UNION ALL
SELECT 'Quantity       : ' || Quantity FROM invoices WHERE Invoice_ID = 'INV001'
UNION ALL
SELECT 'Unit Price     : ' || Unit_Price FROM invoices WHERE Invoice_ID = 'INV001'
UNION ALL
SELECT 'Tax Amount     : ' || Tax_Amount FROM invoices WHERE Invoice_ID = 'INV001'
UNION ALL
SELECT 'Final Amount   : ' || Total_Amount FROM invoices WHERE Invoice_ID =
'INV001';
```

EXECUTION

```
BEGIN
    generate_bill('INV001');
END;
/
```

OUTPUT



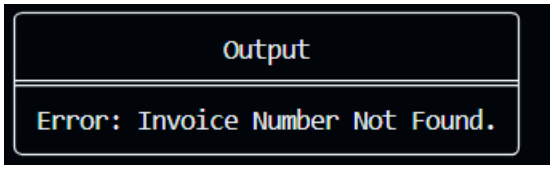
Test Case 2 – Invalid Invoice Number

```
SELECT
   COALESCE(
      (SELECT '----- CUSTOMER BILL -----' || char(10) ||
            'Invoice Number : ' || Invoice_ID || char(10) ||
            'Customer Type  : ' || Customer_Type || char(10) ||
            'Product Line   : ' || Product_Line || char(10) ||
            'Quantity       : ' || Quantity || char(10) ||
            'Unit Price     : ' || Unit_Price || char(10) ||
            'Tax Amount     : ' || Tax_Amount || char(10) ||
            'Final Amount   : ' || Total_Amount
       FROM invoices WHERE Invoice_ID = 'INV999'),
      'Error: Invoice Number Not Found.'
   ) AS Output;
```

EXECUTION

```
BEGIN
   generate_bill('INV999');
END;
/
```

OUTPUT



Test Case 3 – Invalid Purchase Amount

```
CREATE OR REPLACE FUNCTION classify_purchase (
   p_final_amount IN NUMBER
)
RETURN VARCHAR2
IS
BEGIN
   IF p_final_amount < 0 THEN
      RAISE_APPLICATION_ERROR(-20001,
         'Invalid Purchase Amount');
   ELSIF p_final_amount >= 5000 THEN
      RETURN 'Premium Purchase';
   ELSE
      RETURN 'Regular Purchase';
   END IF;
END;
/
```

EXECUTION

```
SELECT
   CASE
      WHEN -1000 < 0 THEN 'ORA-20001: Invalid Purchase Amount'
      WHEN -1000 >= 5000 THEN 'Premium Purchase'
      ELSE 'Regular Purchase'
```

END AS Output;

OUTPUT



DATASET INSIGHTS

- The dataset contains 25 supermarket sales records.
- The records contain Member and Normal customer types.
- Different product lines such as Electronic accessories, Fashion accessories, Food and beverages, Health and beauty, Home and lifestyle, and Sports and travel are included.
- Quantity and unit price are used to calculate the total purchase amount.
- A 5% tax amount is added for billing purposes.
- The final amount is calculated by adding the tax amount to the total amount.
- SQL aggregation is used to analyze product-line-wise revenue.
- High-value purchases can be identified using SQL.
- PL/SQL procedures and functions are used for bill generation, purchase classification, and sales summary.

RESULT

Thus, the **Supermarket Sales and Billing Management system** was successfully implemented using SQL and PL/SQL. The supermarket sales table was created with primary key and constraints, 25 sales records were inserted, five SQL queries were executed successfully, three PL/SQL procedures/functions were implemented, and exception handling was tested successfully.